测向误差下的交会定位几何与覆盖搜索清除模型

摘　要

机器狗携带测向机在半径 1800 米的圆域内搜索个数未知的干扰源，测向误差不超过 1°，走到源 20 米以内才能清除，任务时间越短越好。本文针对测向交会定位与自动搜索清除问题进行了研究。

针对问题一，建立了**扇形交会定位模型**。首先把每条示向度化为张角 2° 的扇形，证明定位区域是若干半平面之交，为必含真实位置的凸多边形；然后用**旋转卡壳直径算法**求直径，与暴力枚举逐组一致；最后由泰勒斯定理与阿波罗尼奥斯定理建立覆盖判据。最终结论为：**以直径为直径的圆不一定覆盖定位区域**，三个及以上检测点时最远顶点到圆心的距离**最坏为半径的 $\sqrt{3}$ 倍**；两个检测点时只在**交会角与直角相差不到 2°** 时盖不住，且**超出半径不到 3.6%**。

针对问题二，建立了**期望耗时选点模型**。首先以第一检测点为原点、示向度为轴化简坐标，由接收半径条件得到源距的后验分布；然后推出定位区域直径的闭式，按定位效果口径得到**最优第二点为前进 908 米、横移 578 米，期望直径 50.2 米**；接下来计入走到第二点的时间，以清除该源的期望耗时为目标，用**蒙特卡洛网格搜索算法**求解。最终结果为：**最优第二点为前进 350 米、横移 125 米，期望完成时间 207.3 秒**，比定位效果口径快 42%；**候选区域为示向度前方 75 至 600 米、两侧 50 至 200 米**。

针对问题三，建立了**六边形覆盖清除模型**。首先分析代价结构，行走 25 米与一次检测同价，时间主要花在行走上；然后证明原点加半径 1150 米正六边形顶点共 7 个扫描点即可覆盖整个目标区域；接下来把扫描点与已发现的源排成开放最短路并逐步重排，用**滚动路径决策算法**在试清除与补测之间取舍。最终结果为：1000 局离线案例**清除比例 1.000、平均定位清除时间 238.7 秒**，比逐步贪心缩短 33.1%；模拟器演练 104 局清除比例 1.000、平均定位清除时间 232.5 秒；正式测试结果见表 9。

针对问题四，建立了**凸包覆盖清除模型**。首先证明任意朝向的源都能被发现，当且仅当其位置落在周围扫描点的凸包内；然后把扫描点设计写成路线长度与扫描耗时之和最小的集合覆盖问题，用**模拟退火布点算法**得到 25 个扫描点、路线 17.2 公里，五千万个随机源零漏检；接下来在运行中按独占样本对扫描点剪枝与合并。最终结果为：500 局离线案例**清除比例 1.000、平均定位清除时间 483.7 秒**，比逐步贪心缩短 51.3% 且不漏定向源；模拟器演练 100 局清除比例 1.000；正式测试结果见表 14。

关键词：交会定位　旋转卡壳　蒙特卡洛网格搜索　滚动路径决策　模拟退火布点

1. 问题重述

1.1 问题背景

无线电干扰源的定位与清除是频谱管理的重要任务[1]，分初步监测、区域性排查与精准定位三个阶段。前两个阶段用固定监测站、监测车或无人机大范围采集，圈定干扰源的分布区域；精准定位仍要靠技术人员携带便携式测向机徒步排查，耗时最长、风险也最高，因此有公司拟研发搭载于机器狗的自动定位清除系统代替人工。

系统由测向机、光学探测仪与激光枪构成：测向机在一个检测点给出干扰源的示向度，光学探测仪在 20 米内完成精确定位，激光枪完成清除。目标区域是半径 1800 米的圆域，以圆心为原点、正东为 x 轴正向、正北为 y 轴正向，区域内干扰源的个数未知，每个源占 20 个频道中的一个。任务场景如图 1 所示：全向源的接收范围是一个整圆，定向源只有朝向一侧的半圆，圆外与半圆背面都收不到信号。

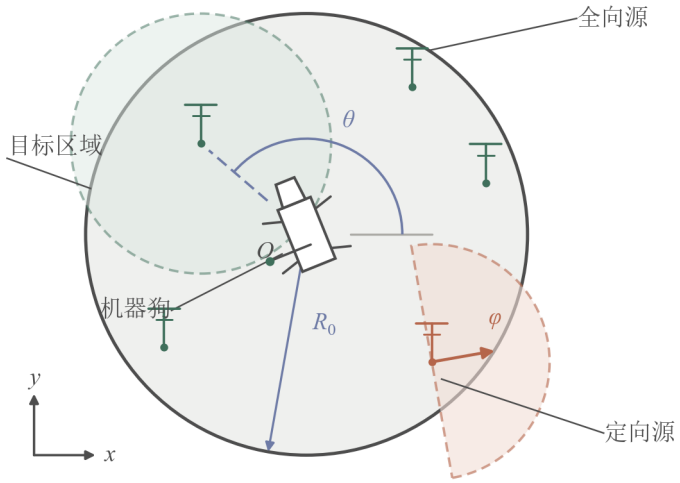


图 1 目标区域中的机器狗与两类干扰源

1.2 问题要求

本文需要解决以下四个问题。

问题 1. 已知若干个检测点的坐标及某干扰源关于这些检测点的示向度，给出用交会定位法计算多边形定位区域直径的算法，并回答以该直径为直径的圆能否覆盖定位区域。

问题 2. 已知某全向干扰源在一个检测点处测得的示向度，给出第二个检测点的选择策略使定位效果较好，并给出候选区域。

问题 3. 干扰源均为全向，总数在 10 到 16 之间但个数未知。机器狗从原点出发、初始频道为 1，在确保全部清除的前提下使完成时间尽量短。制定策略并设计算法，通过模拟器演练测试给出被清除个数的比例与平均定位清除时间，并完成三次正式测试。

问题 4. 全向源与定向源并存，总数仍在 10 到 16 之间，定向源的个数与方向均未知，其余条件同问题三。给出此条件下的检测与清除策略及算法，并完成演练测试与三次正式测试。

2. 问题分析

2.1 问题一的分析

问题一要求由检测点坐标与示向度算出交会定位区域的直径，并回答覆盖性。

首先看单个检测点给出什么。读数与真实方位角相差不超过 1°，所以一个检测点把干扰源限制在以该点为顶点、张角 2° 的扇形内，这正是题目图 2 中两条虚线夹出的区域[2,3]。

然后把扇形写成半平面。张角小于 180° 的扇形等于两个半平面之交，于是定位区域是有限个半平面的交，必为凸多边形，交会定位法化成一个半平面交问题，建立扇形交会定位模型。

最后求直径并回答覆盖性。凸多边形的最远点对必在顶点取到，用旋转卡壳算法线性时间求出；由泰勒斯定理，顶点在圆内当且仅当它对直径的张角不小于 90°，覆盖性归结为逐顶点的角度判断，再由阿波罗尼奥斯定理给出距离上界，并用随机统计与反例确认盖不住的情形确实出现。

2.2 问题二的分析

问题二要求在只有一条示向度时给出第二个检测点的选择策略与候选区域。

首先做坐标化简。取第一个检测点为原点、示向度方向为 t 轴正向，任何构型都化到同一个平面上讨论。源距未知，但该源确实被测到，距离不超过它的有效接收半径，由此得到源距的后验密度。

然后分析第二点怎样影响定位。两条示向度各对应一条窄条带，交出近似平行四边形，由此得到定位区域直径的闭式：交会角越接近直角、两个检测点离源越近，区域越小。但只按区域大小选点，机器狗会为撑开交会角横走很远，而清除落空只耗 3 秒、补测一次只耗 5 秒，所以第二点的好坏应当用把这个源清掉所需的期望时间来衡量，据此建立期望耗时选点模型。

最后求解并给出候选区域。期望时间写不出闭式，用蒙特卡洛网格搜索算法求最优点，把期望时间不超过最优值 5% 的点集作为候选区域，再改变样本数重算检验稳定性。目标区域边界不作约束，附件允许机器狗到区域外检测，它只用来截断源距的取值范围。

2.3 问题三的分析

问题三要求在全向源条件下制定完整的搜索定位清除策略，目标是在全部清除的前提下时间最短。

首先辨清代价结构。一次检测只相当于走 25 米，时间主要花在走路上：走到每个源 20 米以内的路不可避免，证明频道为空要到足够多的地方扫描，定位还要偏离路线补测。所以本问的实质是带覆盖约束的路径规划问题[4]，而不是测量次数的节约问题。

然后分处理覆盖、路线与定位清除。覆盖上，全向源在 1000 米内必被测到，只要扫描点的 1000 米圆盘盖满目标区域，没测到的频道一定为空，取原点加正六边形 6 个顶点即可。路线上，把还要去的扫描点与已发现的源排成一条从当前位置出发的最短开放路，每做完一件事重排一次。定位清除上，沿用问题一的扇形可行域与问题二的期望耗时思想，每一步比较试清除与补测哪个更快。由此建立六边形覆盖清除模型。

最后求解与检验。用最短路启发式与蒙特卡洛估计逐步求解，在离线复刻环境上与逐步贪心等策略同批对比，再到模拟器上批量演练，检验两者是否一致。

2.4 问题四的分析

问题四在问题三的基础上加入定向干扰源，定向源只向定向方向两侧各 90° 辐射，方向未知。

首先看变化出在哪里。检测点落在定向源背面时，1000 米内也收不到信号。因此六边形扫描点不再能证明频道为空，无信号也不能排除附近的位置，沿用问题三会漏掉定向源。

然后重新推导覆盖条件。一个位置上的源不论朝哪个方向都能被发现，当且仅当它落在周围 1000 米内扫描点的凸包里，这比圆盘覆盖严得多。据此把扫描点设计写成路线长度与扫描耗时之和最小的抽样集合覆盖问题，用模拟退火压缩点数与路线；运行中

按独占样本剪枝、顶替与合并。对已发现的频道，每个可能位置再附带定向方向的可行圆弧，并把无信号给出的接收半径上界带进收到信号的概率，免得在背面反复白测。由此建立方向覆盖混合源清除模型。

最后求解与检验。算法沿用问题三，只换扫描点与可行域；先拆时间构成找到大头，再比较点阵、贪心与退火三种覆盖方案，用五千万个随机源独立核验，并在离线环境与模拟器上演练。

2.5　总体思路

四问的承接关系如图 2 所示。第一问把交会定位化为半平面交，得到必含真值的凸可行域，这是后三问共用的定位基础；第二问选第二个检测点，得出的期望耗时取舍原样用于后两问的每一次补测与试清除；第三问补上覆盖约束与全局路线；第四问只替换覆盖约束与可行域，其余沿用第三问。图中每一行从上一问算完的那个框接进下一行的首框。

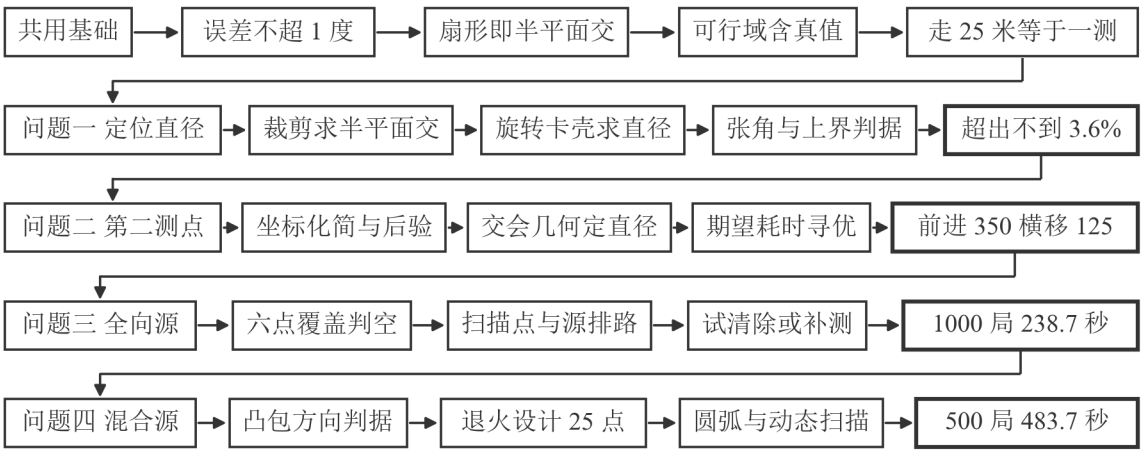


图 2　全文的建模与求解思路

3.　模型假设

(1) 忽略测向机与干扰源的高度差异，全部建模在二维平面上进行。

(2) 机器狗沿直线行进且速度恒为 5 米每秒，忽略起步、转向与地形造成的额外耗时。

(3) 示向度误差的绝对值不超过 1°，这是硬上界而非统计分布，因此真实位置必然落在示向度扇形之内。

(4) 同一地点同一频道的电磁环境固定，重复检测不改变误差，只有换地点误差才重新出现。

(5) 不同干扰源的信号互不影响，每个频道至多对应一个干扰源，且频道不随时间变化。

(6) 干扰源位置在目标区域内均匀分布，有效接收半径在 1000 到 1500 米之间均匀分布，二者独立。

4.　符号说明

其它符号在文中具体说明。

表 1　符号说明

符号	说明	单位
$\mathbf{S}_i$	第 i 个检测点的位置	米
$\mathbf{G}$	干扰源的真实位置	米
θ_i	测向机在 $\mathbf{S}_i$ 处测得的示向度	度
ε	示向度误差上界，取 1°	度
Ω	交会定位区域	—
$d(\Omega)$	定位区域的直径	米
R_0	目标区域半径，取 1800	米
$R_{\min}, R_{\max}$	有效接收半径的下界与上界，取 1000 与 1500	米
r_c	清除半径，取 20	米
ρ	干扰源到第一个检测点的距离	米
L	干扰源到第二个检测点的距离	米
γ	两条示向度在干扰源处的交会角	度
v	机器狗行进速度，取 5	米每秒
t_m, t_s	一次检测与一次切换频道的耗时，取 5 与 1	秒
t_c, t_l	清除成功与清除落空的耗时，取 5 与 3	秒
$\mathcal{F}_c, \mathcal{Q}$	频道 c 的可行位置集合，覆盖扫描点的集合	—
$\varphi, \Psi(\mathbf{X})$	定向方向与位置 $\mathbf{X}$ 处定向方向的可行圆弧	度
a	问题三外圈扫描点到原点的距离，取 1150	米
$\mathbf{X}^*, p^*$	后验下的最优清除点与该点清除成功的概率	—

5.　模型的准备

本章给出四问共用的三件事：角度约定与误差、虚拟时间的代价结构、三类观测对可行位置的作用。

5.1　角度约定与示向度误差

全文方位角以正东为 0°、逆时针为正。记干扰源 $\mathbf{G}$ 相对检测点 $\mathbf{S}$ 的真实方位角为 $\alpha = \arg(\mathbf{G}-\mathbf{S})$、读数为 θ，则由题目条件

$$|\alpha - \theta| \le \varepsilon, \qquad \varepsilon = 1^\circ. \tag{5-1}$$

测向机的工作方式如图 3(a) 所示。机器狗停下转动天线，场强最大的朝向就是读数 θ；真实方位 α 落在读数两侧各 ε 的一小段扇面内。式 (5-1) 是硬约束而不是置信区间，因此后文所有可行域都必然包含真实位置。同一地点重复检测得到同一个读数，要缩小这段弧只能换地方再测。

5.2　虚拟时间的代价结构

一次检测指令的耗时由三部分组成，一次清除指令由两部分组成：

$$\begin{aligned} T_{\text{measure}} &= \frac{\|\mathbf{p}_{\text{new}} - \mathbf{p}_{\text{old}}\|}{v} + t_s \mathbf{1}\{c_{\text{new}} \neq c_{\text{old}}\} + t_m, \\ T_{\text{clear}} &= \frac{\|\mathbf{p}_{\text{new}} - \mathbf{p}_{\text{old}}\|}{v} + \begin{cases} t_c = 5 \text{ 秒}, & \text{清除成功} \\ t_l = 3 \text{ 秒}, & \text{未发现} \end{cases} \end{aligned} \tag{5-2}$$

其中 $v = 5$ 米每秒，$t_m = 5$ 秒，$t_s = 1$ 秒。

把式 (5-2) 的各项折算成等价的行走距离，就得到图 3(b) 的时间尺：一次检测等于走 25 米，切换频道等于走 5 米，清除成功等于走 25 米，落空等于走 15 米；而走完目标区域直径要 720 秒，是一次检测的 144 倍。**行走是主要代价**，这是后文策略的出发点：宁可多测几次，也不要多走几百米。

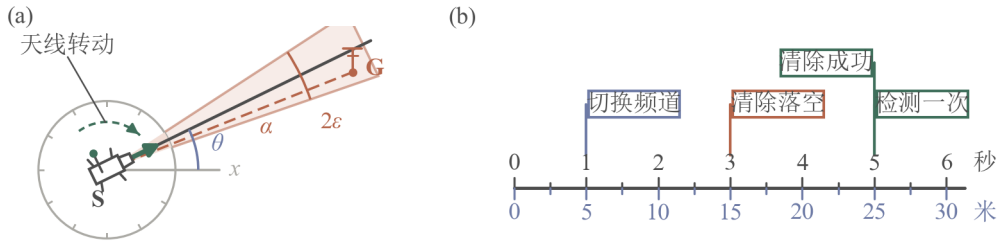


图 3　两件共用的基础：(a) 示向度的定义与误差上界；(b) 各项动作的耗时与等效行走距离

5.3　三类观测对可行位置的约束

设频道 c 的可行位置集合为 $\mathcal{F}_c$，初始为整个目标区域。在 $\mathbf{p}$ 处对频道 c 检测一次，三种返回值分别给出：返回示向度 θ 时干扰源落在以 $\mathbf{p}$ 为顶点、张角 2ε 的扇形内，且距离大于 5 米、不超过 $R_{\max}$；返回信号过强时干扰源距 $\mathbf{p}$ 不超过 5 米，可直接清除；返回无信号时，全向源距 $\mathbf{p}$ 大于 $R_{\min}$，定向源只说明指向不朝这一侧，见 6.4 节。此外，清除指令返回未发现时，该频道的干扰源距该点大于 $r_c = 20$ 米。

四类约束的作用如图 4 所示，扇形张角放大画出，浅色是留下的可行域，斜线是被切掉的部分。图 4(a) 测到示向度，只剩扇形与 $R_{\max}$ 圆盘之交；(b) 无信号，挖去 $\mathbf{p}$ 周围半径 $R_{\min}$ 的圆盘；(c) 在 $\mathbf{p}$ 处测到而在 $\mathbf{p}'$ 处没测到，两点接收半径相同，故源离 $\mathbf{p}$ 更近，再交上中垂线靠 $\mathbf{p}$ 的一侧；(d) 清除落空，挖去落空点周围半径 r_c 的小圆。四类约束都只让可行域变小，真实位置始终留在其中。

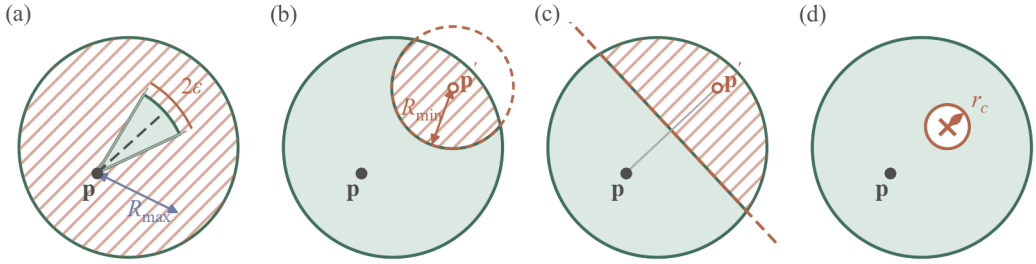


图 4　四类观测对可行位置的作用：(a) 测到示向度；(b) 无信号；(c) 中垂线；(d) 清除落空

6.　模型的建立与求解

6.1　问题一：交会定位区域的直径算法与覆盖性

6.1.1　思路框架

本问的推理链如图 5 所示：由误差上界得到扇形，由扇形得到半平面交与凸性，在凸多边形上求直径，再由张角判据与距离上界回答覆盖性，最后用暴力枚举、随机统计与反例三面核对。

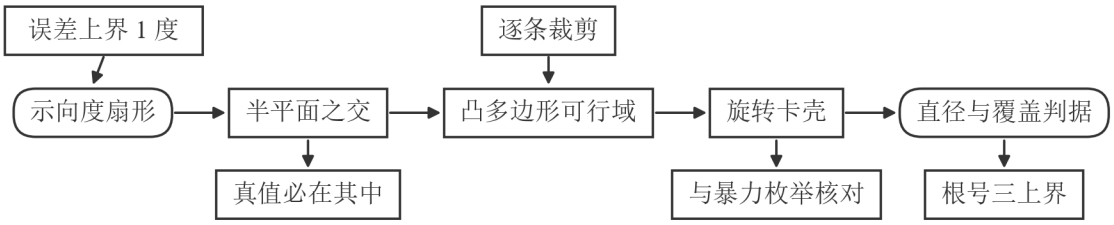


图 5　问题一的思路框架

6.1.2　扇形与半平面

记 $\theta_i^{\pm} = \theta_i \pm \varepsilon$、边界射线方向 $\boldsymbol{e}_i^{\pm} = (\cos\theta_i^{\pm}, \sin\theta_i^{\pm})$，二维叉积记 $\boldsymbol{a} \times \boldsymbol{b} = a_x b_y - a_y b_x$。

引理 1. 设 $0<\varepsilon<90^\circ$，则检测点 $\mathbf{S}_i$ 给出的扇形

$$W_i=\left\{\mathbf{p}:\ \boldsymbol{e}_i^{-}\times(\mathbf{p}-\mathbf{S}_i)\ge 0,\ \ \boldsymbol{e}_i^{+}\times(\mathbf{p}-\mathbf{S}_i)\le 0\right\} \tag{6-1}$$

是两个半平面之交，且真实位置 $\mathbf{G}\in W_i$。

证明. 两个条件各是关于 $\mathbf{p}$ 的一个线性不等式，故 W_i 是两个半平面之交。设 $\mathbf{p}\neq\mathbf{S}_i$，记 $\beta=\arg(\mathbf{p}-\mathbf{S}_i)$，则 $\boldsymbol{e}_i^{-}\times(\mathbf{p}-\mathbf{S}_i)=\|\mathbf{p}-\mathbf{S}_i\|\sin(\beta-\theta_i^{-})$，$\boldsymbol{e}_i^{+}\times(\mathbf{p}-\mathbf{S}_i)=\|\mathbf{p}-\mathbf{S}_i\|\sin(\beta-\theta_i^{+})$。两式分别要求 $\beta-\theta_i^{-}\in[0^\circ,180^\circ]$ 与 $\beta-\theta_i^{+}\in[-180^\circ,0^\circ]$，即 $\beta\in[\theta_i^{-},\theta_i^{-}+180^\circ]\cap[\theta_i^{+}-180^\circ,\theta_i^{+}]$。由 $\theta_i^{+}-\theta_i^{-}=2\varepsilon<180^\circ$，这个交集恰为 $[\theta_i^{-},\theta_i^{+}]$，反方向的那一瓣不会被包进来。再由式 (5-1)，$\alpha=\arg(\mathbf{G}-\mathbf{S}_i)$ 满足 $|\alpha-\theta_i|\le\varepsilon$，故 $\mathbf{G}\in W_i$。 □

引理 1 说明把扇形写成半平面之交不是近似而是恒等：两个半平面各占过 $\mathbf{S}_i$ 的一条直线的一侧，交出来只剩夹角为 2ε 的那一瓣，真实位置 $\mathbf{G}$ 落在其中。

6.1.3 扇形交会定位模型

综上所述，最终得到**扇形交会定位模型**为

$$\begin{cases}\Omega=\displaystyle\bigcap_{i=1}^{n}W_i=\bigcap_{j=1}^{2n}\left\{\mathbf{p}:\ \boldsymbol{u}_j\cdot\mathbf{p}+c_j\le 0\right\}\\ W_i=\left\{\mathbf{p}:\ \boldsymbol{e}_i^{-}\times(\mathbf{p}-\mathbf{S}_i)\ge 0,\ \boldsymbol{e}_i^{+}\times(\mathbf{p}-\mathbf{S}_i)\le 0\right\},\quad i=1,\dots,n\\ d(\Omega)=\displaystyle\max_{\mathbf{p},\boldsymbol{Q}\in\Omega}\|\mathbf{p}-\boldsymbol{Q}\|\end{cases} \tag{6-2}$$

由引理 1，每个 W_i 都含真实位置，故 $\mathbf{G}\in\Omega$；Ω 是半平面之交，为凸多边形，直径在顶点处取到。若各条示向度近乎平行，Ω 可能无界，此时需要补测；问题三、四另加接收半径上界与目标区域两个圆盘约束，保证可行域有界。

6.1.4 旋转卡壳直径算法

求 Ω 分两步：先用逐条裁剪把 $2n$ 个半平面交出来，再在结果的凸包上用旋转卡壳求最远点对。

两步的做法如图 6 所示。图 6(a) 是裁剪：以一个边长足够大的正方形起手，把 $2n$ 条半平面边界逐条画上去，每条只保留法向指定的一侧[5,6]，深色就是 Ω；若某条线把多边形整个切空，说明这组示向度互相矛盾，若最后仍有顶点贴在起手正方形的边上，说明 Ω 无界。图 6(b) 是旋转卡壳[8,9]：一对平行的卡钳臂夹住凸多边形，一条贴着某条边，另一条贴在离它最远的顶点上；卡钳转一圈，贴边的臂依次走过每条边，对臂的顶点只单调前进，所以总代价是线性的。图中虚线一对是中途的位置，实线一对取到直径，端点为 P、Q。

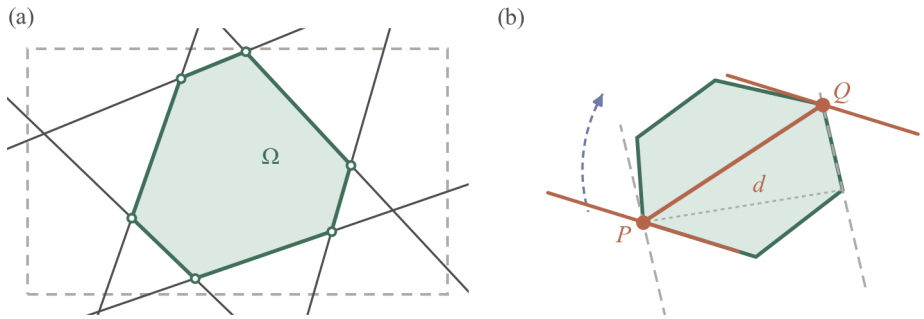


图 6 求定位区域与直径：(a) 逐条裁剪求半平面交；(b) 旋转卡壳求最远点对

求解步骤如下。

Step1 按式 (6-1) 为每个检测点生成两个半平面，共 $2n$ 个。

Step2 以一个边长足够大的正方形为初始多边形逐个裁剪，得到 Ω 的顶点序列，并判断是否为空或无界。

Step3　用单调链算法[7] 求顶点的凸包，去掉共线点。
Step4　用旋转卡壳求最远点对，输出直径 d 与端点 P、Q。

多边形顶点数不超过 $2n$，故 Step2 为 $O(n^2)$，Step3 为 $O(n\log n)$，Step4 为 $O(n)$；检测点很多时 Step2 可换成按极角排序的半平面交算法，总复杂度降为 $O(n\log n)$。整个流程如图 7 所示，两个判断框分别对应可行域为空与无界这两种要单独处理的情形。

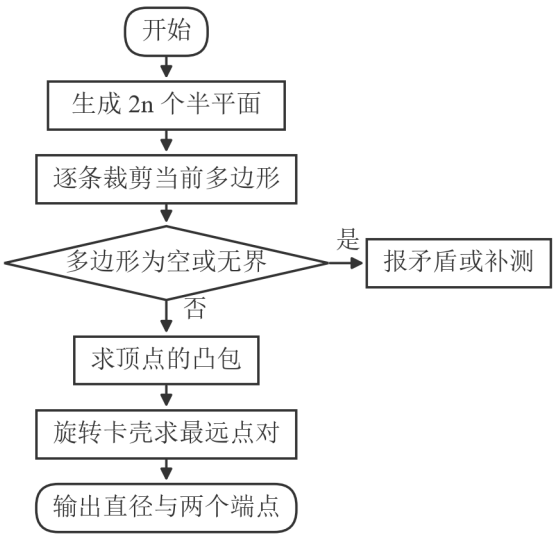


图 7　定位区域直径算法的流程

6.1.5　覆盖性判据

以直径两端点 P、Q 的连线为直径的圆，圆心为中点 M，半径为 $d/2$。凸多边形被这个圆覆盖，当且仅当每个顶点都在圆内。

定理 1 (张角判据)**.** *顶点 V 落在以 PQ 为直径的闭圆盘内，当且仅当 $\angle PVQ \geq 90^\circ$。*

证明. 记 M 为 PQ 的中点。由中线长公式 $\|VM\|^2 = \frac{1}{2}(\|VP\|^2 + \|VQ\|^2) - \frac{1}{4}\|PQ\|^2$，再由余弦定理 $\|PQ\|^2 = \|VP\|^2 + \|VQ\|^2 - 2\|VP\|\|VQ\|\cos\angle PVQ$，两式相减得

$$\|VM\|^2 - \frac{d^2}{4} = -\|VP\|\,\|VQ\|\cos\angle PVQ. \tag{6-3}$$

故 $\|VM\| \leq d/2$ 当且仅当 $\cos\angle PVQ \leq 0$，即 $\angle PVQ \geq 90^\circ$。 □

定理 2 (距离上界)**.** *凸多边形的任一顶点 V 到直径中点 M 的距离满足*

$$\|VM\| \leq \frac{\sqrt{3}}{2}\,d, \qquad \text{即} \quad \frac{\|VM\|}{d/2} \leq \sqrt{3}, \tag{6-4}$$

等号当且仅当 $\|VP\| = \|VQ\| = d$，即 PQV 为边长 d 的正三角形时取到。

证明. P, Q 是直径端点，故任一顶点满足 $\|VP\| \leq d$ 与 $\|VQ\| \leq d$。代入中线长公式得 $\|VM\|^2 = \frac{1}{2}(\|VP\|^2 + \|VQ\|^2) - \frac{1}{4}d^2 \leq d^2 - \frac{1}{4}d^2 = \frac{3}{4}d^2$。等号要求两个不等式同时取等，即 $\|VP\| = \|VQ\| = d$。 □

由定理 1，式 (6-3) 并不对所有顶点自动成立，所以**以定位区域直径为直径的圆不一定能覆盖该定位区域**；由定理 2，最坏时最远顶点到圆心的距离是圆半径的 $\sqrt{3}$ 倍。

定理 1 的三种情形如图 8 所示。三幅图的 PQ 与虚线半圆相同，只有第三个顶点的位置不同：(a) 张角大于 90°，顶点在圆内；(b) 恰为 90°，顶点落在圆周上；(c) 小于 90°，顶点跑到圆外。判断覆盖性只要对每个顶点量一次这个角。

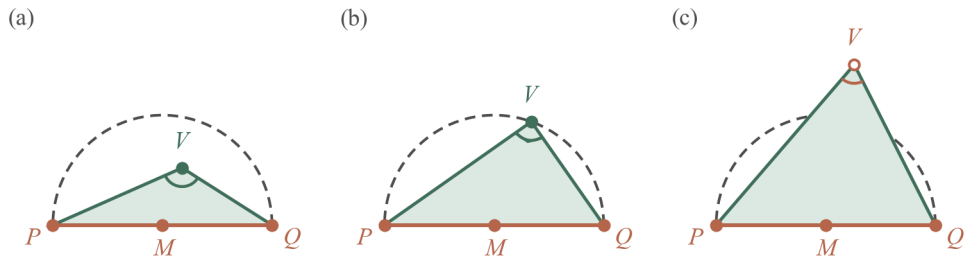


图 8　张角判据的三种情形：(a) 张角大于直角；(b) 张角为直角；(c) 张角小于直角

6.1.6　两个检测点时的解析结论

两个检测点是交会定位最常用的情形，这时覆盖性有解析判据。

定理 3 (两点判据)**.** 两个检测点交出的定位区域是四边形，记两条示向度的交会角为 γ，取 $0 < \gamma \le 90^\circ$ 一侧。则圆盖不住定位区域蕴含

$$90^\circ - 2\varepsilon < \gamma \le 90^\circ, \tag{6-5}$$

且最远顶点到圆心的距离不超过半径的 $\cot(45^\circ - \varepsilon)$ 倍。

证明. 四条扇形边界两两相交得到四边形的四个顶点，每个内角等于过该顶点的两条边界线的夹角。两个检测点的边界线方向分别为 $\theta_1 \pm \varepsilon$ 与 $\theta_2 \pm \varepsilon$，故四个内角分别等于 $|\theta_1 - \theta_2|$ 加上 0、$\pm 2\varepsilon$ 这几个偏移：一组对角落在 γ 的 2ε 邻域内，另一组落在 $180^\circ - \gamma$ 的 2ε 邻域内。直径连接两个锐角顶点，其余两个顶点对直径的张角就是它们的内角，取值不小于 $90^\circ - 2\varepsilon$。由定理 1，盖不住要求某个内角小于 90°，于是 $\gamma + 2\varepsilon > 90^\circ$ 且 $\gamma \le 90^\circ$，即式 (6-5)。又对直径张角为 ϕ 的点落在一段圆弧上，它到直径中点的最远距离为 $(d/2)\cot(\phi/2)$，取 $\phi \ge 90^\circ - 2\varepsilon$ 即得倍数上界。 □

取 $\varepsilon = 1^\circ$，式 (6-5) 说明盖不住只发生在交会角处于 88° 到 90° 之间，倍数上界为 $\cot 44^\circ \approx 1.0355$，即最远顶点超出半径不到 3.6%。

定理 3 的内容如图 9 所示，三幅图只改变交会角。(a) 的 $\gamma = 50^\circ$，四边形细长，直径连的是两个锐角顶点，另两个顶点的张角是钝角，圆盖得住；(b) 的 $\gamma = 75^\circ$，四边形方正一些，两个张角仍在直角以上；(c) 的 $\gamma = 88^\circ$，两个张角落到直角两侧，小于直角的那个顶点落在虚线圆外。可见盖不住只发生在交会角贴近直角的一个 2ε 宽的窄带里。

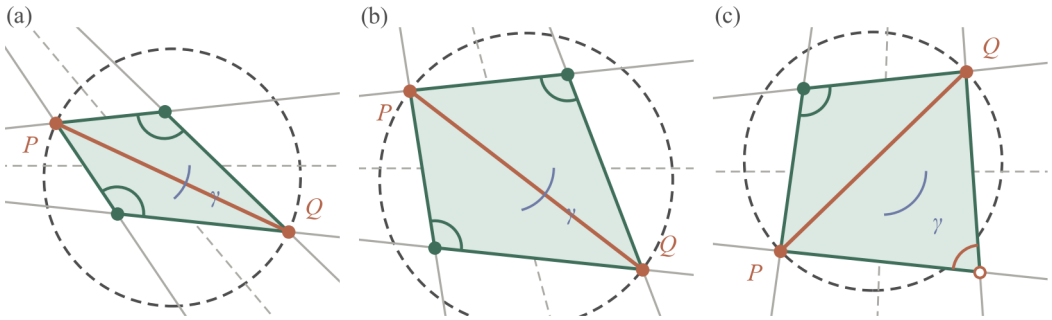


图 9　交会角对四边形与覆盖性的影响：(a) $\gamma = 50^\circ$；(b) $\gamma = 75^\circ$；(c) $\gamma = 88^\circ$

6.1.7　取到上界的反例

三个及以上检测点时不再有式 (6-5) 这样的窄带，定位区域可以接近正三角形。

构造如下：三个检测点位于半径 900 米的圆周上彼此相隔 120°，三条示向度中线都指向圆心但各带相同的小偏置 β。扫描 β 后在 $\beta = -0.85^\circ$ 处定位区域近似正三角形，直径 $d = 8.162$ 米，以它为直径的圆半径 4.081 米，第三个顶点到圆心 7.069 米，比值 1.7321，恰好取到定理 2 的 $\sqrt{3}$。

推论 1. 圆心固定在直径中点时，要保证覆盖必须把圆的直径放大到 $\sqrt{3}d$，这个倍数不能再小；若允许把圆平移到任意位置，由 *Jung* 定理[11] 只需放大到 $2d/\sqrt{3} \approx 1.155d$，同样在正三角形处取到。

两个倍数的差别如图 10 所示，扇形张角放大画出。(a) 中三个扇形交出的区域恰为正三角形；(b) 把它放大并画出两个圆：以 PQ 为直径的小圆只盖住 PQ 附近，顶点 V 落在圆外，而圆心挪到外心后，半径 4.712 米的最小覆盖圆[10] 刚好盖住三个顶点。可见两个倍数的差别只在圆心允不允许挪动。

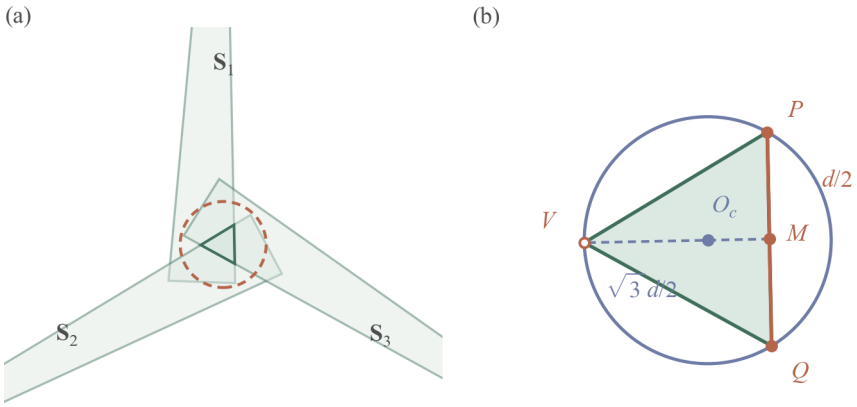


图 10 取到上界的正三角形反例：(a) 三个扇形交出正三角形；(b) 两种圆的对照

6.1.8 求解结果与检验

为确认盖不住的情形确实出现，随机生成构型统计：干扰源在目标区域内随机取位，检测点在其周围 100 到 1400 米处随机取位，误差在 $[-1°, 1°]$ 内均匀取值，每种检测点数约 6 万组，剔除无界的情形，结果见表 2。

表 2 以直径为直径的圆对定位区域的覆盖情况，比值指最远顶点到直径中点的距离与半径之比

检测点数	有界构型数	盖不住的构型数	盖不住的占比	比值最大值
2	59340	685	1.15%	1.0344
3	59996	8537	14.23%	1.7185
4	60002	13449	22.41%	1.7283
5	60002	16938	28.23%	1.7139

由表 2 可以看出三点。其一，两个检测点时盖不住的只占 1.15%，比值最大 1.0344，与定理 3 的 1.0355 相符，最坏一例的两个张角为 88.06° 与 92.06°，超出半径 3.44%。其二，检测点由三个增至五个，盖不住的比例由 14.23% 升至 28.23%。其三，比值最大值已接近定理 2 的上界 $\sqrt{3}$。

比值的分布如图 11 所示。(a) 是四种检测点数下比值的经验分布，曲线在比值为 1 处的高度就是盖得住的比例：两个检测点几乎一步到顶，检测点越多起跳越低、拖尾越长，但所有拖尾都止于 $\sqrt{3}$ 之前。(b) 每行一种检测点数，哑铃的空心端是中位数、实心端是 99% 分位：中位数都贴着左侧，而右端随检测点数迅速右移并逼近 $\sqrt{3}$。原因是两个扇形交出的四边形接近平行四边形，直径两侧的顶点对称地贴在圆上；检测点越多，交出三角形这类没有对称结构的区域的机会越大，第三个顶点就可能离直径中点很远。

模型用三种方式检验。第一，与暴力枚举比对：在表 2 的 23.9 万组有界构型上，旋转卡壳与全顶点枚举所得直径逐组比较，偏差全部为 0。第二，真值包含性：这些构型中真实干扰源落在定位区域之外的次数为 0，与引理 1 一致。第三，理论与数值互证：比值最大值始终不超过 $\sqrt{3}$，反例恰好取到该上界。这一问也说明了后两问为什么不追求

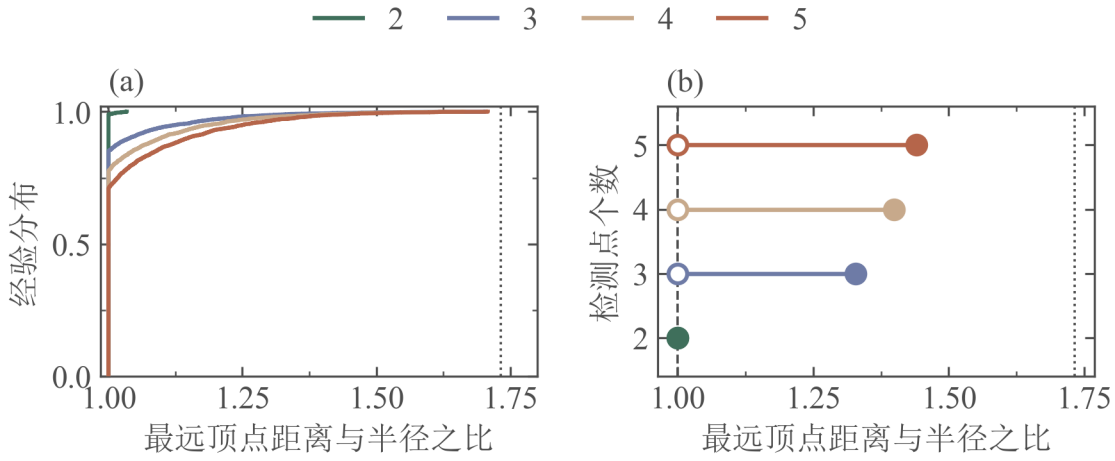


图 11　覆盖情况的统计：(a) 比值的经验分布；(b) 按边数统计的盖不住占比

一次必中：要保证必定清除成功，可行域的最小覆盖圆半径须不超过 $r_c = 20$ 米，为此要多测多走，而清除落空只多耗 3 秒。

问题一的答案：定位区域直径按生成半平面、逐条裁剪、求凸包、旋转卡壳四步算出。以定位区域直径为直径的圆**不一定**能覆盖该定位区域，判据是其余顶点对直径的张角是否都不小于 90°。两个检测点时盖不住只发生在交会角与直角相差不到 2° 时，最远顶点超出半径不到 3.6%；三个及以上检测点时最坏超出 $\sqrt{3}$ 倍，由正三角形取到。要保证覆盖，圆心取直径中点时须把直径放大到 $\sqrt{3}d$，允许平移圆心时须放大到 $2d/\sqrt{3}$。

6.2　问题二：第二个检测点的选择策略

6.2.1　思路框架

本问的推理链如图 12 所示：先化到统一坐标并给出源距后验，再由交会几何得到直径的闭式，据此按定位效果作答；然后改以清除该源的期望时间为目标建立模型，用共同随机数网格寻优求解，给出两套口径的候选区域。

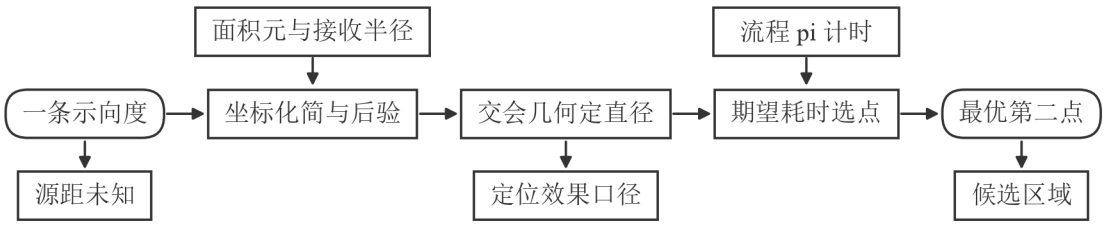


图 12　问题二的思路框架

6.2.2　坐标化简与源距的后验分布

取第一个检测点为原点、示向度方向为 t 轴正向、逆时针转 90° 为 h 轴，第二个检测点记 $S_2 = (t, h)$，干扰源近似位于 t 轴上距离 ρ 处。ρ 未知但有信息：该源在第一个检测点被测到，故 $R \geq \rho$；该源在目标区域内，故 ρ 不超过沿示向度走出目标区域的距离 ρ_{geo}。于是

$$
\begin{aligned}
&f(\rho) \propto \rho \cdot \Pr\{R \geq \rho\}, \qquad 5 < \rho \leq \min\{R_{\max}, \rho_{\text{geo}}\}, \\
&\Pr\{R \geq \rho\} = \min\left\{1,\ \max\left\{0,\ \frac{R_{\max} - \rho}{R_{\max} - R_{\min}}\right\}\right\},
\end{aligned}
\tag{6-6}
$$

其中因子 ρ 来自位置均匀分布的面积元，下限取 5 米是因为更近时测向机返回信号过强，可直接清除。

化简后的坐标与式 (6-6) 的后验如图 13 所示。示向度方向拉成一把尺子，尺上的深浅是 ρ 的后验密度：$R_{\min}$ 以内密度随 ρ 线性上升，因为面积元正比于 ρ；越过 $R_{\min}$ 后接收半径够长的概率下降，到 $R_{\max}$ 处归零。第一个检测点取在原点时 $\rho_{\text{geo}} = 1800$ 米，ρ 的 5%、50%、95% 分位数为 281、890、1331 米，图中用三道刻线标出。可见源距的不确定范围有一公里宽，第二点必须对整段都管用。

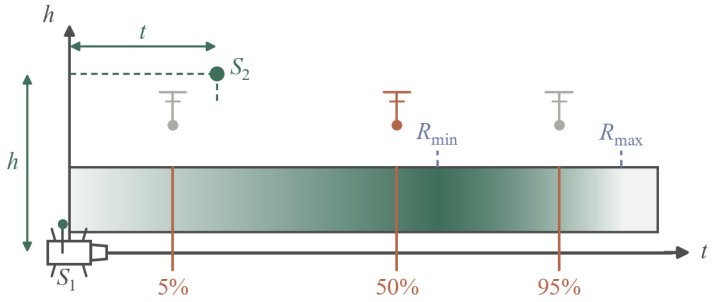


图 13　化简坐标与源距的后验密度

6.2.3　交会几何与定位区域直径

设第二点到源的距离为 L、两条示向度在源处的交会角为 γ。两条示向度把源限制在宽 $2\rho\tan\varepsilon$ 与 $2L\tan\varepsilon$ 的条带内，两条带以 γ 相交成平行四边形，较长对角线即定位区域直径

$$D(\rho, t, h) \approx \frac{2\varepsilon\sqrt{\rho^2 + L^2 + 2\rho L|\cos\gamma|}}{\sin\gamma}, \tag{6-7}$$

式中取了 $\tan\varepsilon \approx \varepsilon$。式 (6-7) 说明交会角要接近 90°，且两个检测点到干扰源的距离都要小。

式 (6-7) 里各量的几何意义如图 14 所示，三幅图的条带宽度相同，只改变交会角。(a) 的 $\gamma = 25^\circ$，两条带几乎重叠，平行四边形沿视线拉得很长，长对角线 D 远大于条带宽度；(b) 的 $\gamma = 55^\circ$ 收短；(c) 的 $\gamma = 90^\circ$ 两条带正交，成为矩形，D 取到最小。这正是式 (6-7) 分母上 $\sin\gamma$ 的来历。

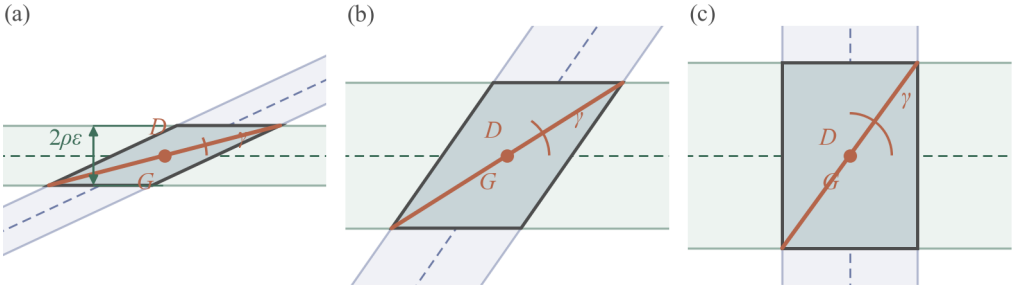


图 14　交会角与定位区域的形状：(a) $\gamma = 25^\circ$；(b) $\gamma = 55^\circ$；(c) $\gamma = 90^\circ$

定理 4 (垂直平移的最优横移)**.** *设第二点只在过 S_1 且垂直于示向度的直线上取，即 $S_2 = (0, b)$，则在式 (6-7) 下*

$$b^* = \sqrt{2}\,\rho, \qquad D_{\min} = 6\varepsilon\rho. \tag{6-8}$$

证明. 此时 $L = \sqrt{\rho^2 + b^2}$，$\cos\gamma = \rho/L$，$\sin\gamma = b/L$。代入式 (6-7) 得

$$D^2 = \frac{4\varepsilon^2}{\sin^2\gamma}\left(\rho^2 + L^2 + 2\rho^2\right) = \frac{4\varepsilon^2 L^2}{b^2}\left(4\rho^2 + b^2\right) = 4\varepsilon^2\left(\frac{4\rho^4}{b^2} + 5\rho^2 + b^2\right).$$

令 $u = b^2 > 0$，则 $D^2 = 4\varepsilon^2(4\rho^4 u^{-1} + 5\rho^2 + u)$，对 u 求导置零得 $u = 2\rho^2$，二阶导为正，故取最小值。回代得 $b^* = \sqrt{2}\rho$ 与 $D_{\min} = 2\varepsilon\sqrt{2\rho^2 + 5\rho^2 + 2\rho^2} = 6\varepsilon\rho$。 □

定理 4 的内容如图 15 所示。第二点沿竖直滑轨自下而上移动，右侧一列按同一放大倍数画出四个位置对应的定位区域，区域内的线段是长对角线 D：横移太小时两条视线接近共线，区域是细长的斜条；横移到 $\sqrt{2}\rho$ 时 D 最短；再往上第二点离源变远，D 又长回去。可见最优点不在交会角恰为 90° 处，因为撑开交会角的同时 L 也在变大。

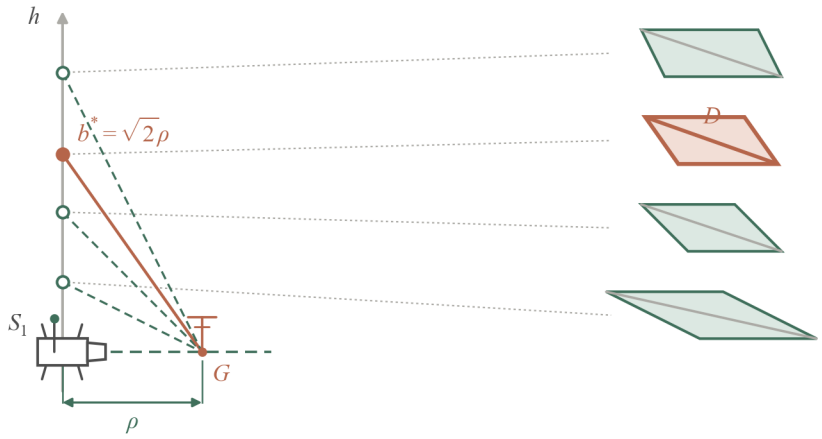


图 15　第二点沿垂直方向滑动时定位区域的变化

定理 4 的数值验证见表 3。

表 3　只作垂直平移时的最优横移量与最小定位区域直径，最小直径按式 (6-7) 数值寻优，精确直径按问题一的算法计算

ρ 米	最优横移米	$\sqrt{2}\rho$ 米	最小直径米	$6\varepsilon\rho$ 米	精确直径米
300	424.3	424.3	31.42	31.42	31.45
600	848.5	848.5	62.84	62.83	62.91
1000	1414.2	1414.2	104.73	104.72	104.85
1500	2121.3	2121.3	157.10	157.08	157.27

由表 3 可以看出，数值寻优的最优横移与 $\sqrt{2}\rho$ 逐行相同，闭式直径与精确直径相差不到 0.2%，式 (6-7) 的近似可用。

6.2.4　定位效果口径下的第二点与候选区域

题目要求第二点使定位效果较好，故先按定位效果作答。以式 (6-7) 的直径为评价量对 ρ 的后验取期望，在收得到信号即 $L \le R_{\min}$ 的点中求最小，得**最优第二点为沿示向度前进 908 米、横移 578 米**，离第一点 1076 米，期望直径 50.2 米。若改取一次交会后直径不超过 $2r_c = 40$ 米的概率最大，最优点为前进 705 米、横移 458 米，该概率 0.407。两个点都落在交会角接近 90° 处，与式 (6-7) 一致。

候选区域取这样的点集：无论源距 ρ 落在 5% 到 95% 可信区间 $[281, 1331]$ 米的哪个位置，都能收到信号且交会角不低于下限 γ_0。构成如图 16 所示。(a) 是收得到信号的条件：源距取区间两端时第二点都要落在以该位置为圆心、$R_{\min}$ 为半径的圆盘内，两圆盘之交就是允许范围。(b) 是交会角条件：对每个源位置，交会角不低于 γ_0 的点集是以源为顶点的 V 形区域之外，源距取遍区间后公共部分形成一条斜带。(c) 是二者之交，即候选区域，星号为最优点。区域关于示向度对称，图中只画上半侧。

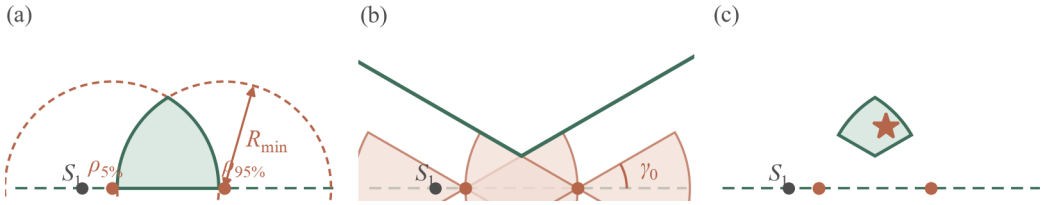


图 16　定位效果口径下候选区域的构成：(a) 收得到信号；(b) 交会角不低于下限；(c) 二者之交

交会角下限取 20°、30°、45° 时，候选区域面积分别为 6.45、4.14、1.23 乘 10^5 平方米，占目标区域的 6.34%、4.07%、1.21%，依次位于示向度前方 398 到 1215 米、473 到 1140 米、630 到 983 米，两侧 195 到 848 米、308 到 848 米、533 到 848 米。下限取 60° 时区域为空，说明对全部可能的源距都保证大交会角做不到。

这个口径把第二点推到离第一个检测点 800 到 1100 米处。由表 3，$\rho = 1000$ 米时须横走 1414 米、耗时 283 秒，而直径仍有 104.7 米，远大于 $2r_c = 40$ 米。机器狗最终都要走到源 20 米以内，朝源前进的路不浪费，横走的路却要额外付出，而清除落空只耗 3 秒、补测只耗 5 秒。因此第二点选得好不好，应当用把这个源清掉总共要花多少时间来衡量。

6.2.5　期望耗时选点模型

1）决策变量

决策变量为第二个检测点在化简坐标下的位置 $S_2 = (t, h)$。

2）目标函数

目标取从第一个检测点出发、到该干扰源被清除为止的期望完成时间。第二点测完之后怎么清除要有固定流程才能计时，本文取与问题三一致的流程 π：把可行域上的可能位置按后验加权，求出使 20 米清除圆内概率最大的点 $\mathbf{X}^*$ 与该概率 p^*；$p^* \ge \tau$ 就走到 $\mathbf{X}^*$ 清除，否则走到 $\mathbf{X}^*$ 补测一次，取 $\tau = 0.5$；落空时从可行域去掉该点周围 20 米的圆盘，按同样规则继续直到成功。于是

$$T(S_2; \mathbf{G}, \boldsymbol{e}) = \frac{\|S_2 - S_1\|}{v} + t_m + C_\pi(S_2; \mathbf{G}, \boldsymbol{e}), \tag{6-9}$$

其中 C_π 为按流程 π 从 S_2 到清除成功的耗时，由移动、补测、试清除三部分累加，$\boldsymbol{e}$ 为各次检测的示向度误差。$\mathbf{G}$ 按式 (6-6) 的后验分布取值，示向度误差在 $[-\varepsilon, \varepsilon]$ 内取值，接收半径在 $[\max\{R_{\min}, \rho\}, R_{\max}]$ 内均匀取值。

流程 π 的三步如图 17 所示。(a) 中灰点是按后验加权的可能位置，圆是半径 r_c 的清除圆，圆心 $\mathbf{X}^*$ 取圆内后验概率最大的位置；(b) 是落空后挖去清除圆，可行域中间出现一个洞；(c) 是在当前位置补测一次示向度，只有落在新扇形内的位置留下来，可行域大幅收窄。问题三与问题四在单个源上用的也是这套流程。

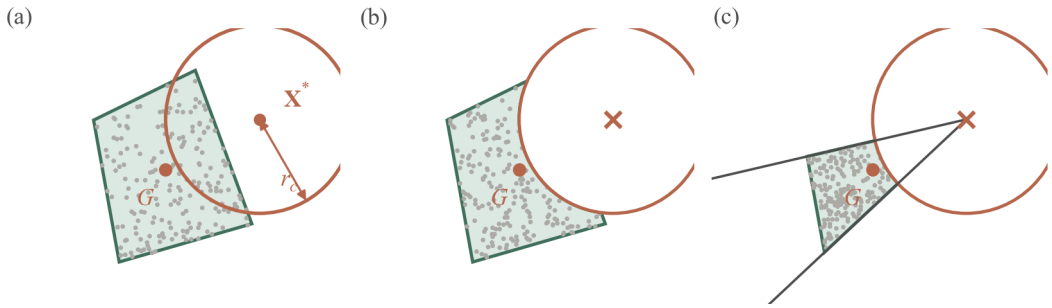


图 17　流程 π 的三步：(a) 后验最密处试清除；(b) 落空后挖去清除圆；(c) 补测后可行域收窄

3）约束条件

约束只有一条：第二点必须收得到这个源的信号，即 S_2 到源的可能位置不超过 $R_{\min}$；目标区域边界不作约束。为同时反映定位效果，另记两个评价量：第一次清除就成功的

概率 P_{hit} 与定位区域直径 D。

综上所述，最终得到**期望耗时选点模型**为

$$\begin{cases} \min\limits_{(t,h)} \ \mathbb{E}\big[T(S_2;\mathbf{G},\boldsymbol{e})\big] \\ s.t. \quad \begin{cases} T = \|S_2 - S_1\|/v + t_m + C_\pi(S_2;\mathbf{G},\boldsymbol{e}) \\ \mathbf{G} = S_1 + \rho\,(\cos\phi, \sin\phi), \quad f(\rho) \propto \rho \Pr\{R \geq \rho\}, \quad |\phi| \leq \varepsilon \\ \|S_2 - \mathbf{G}\| \leq R_{\min} \\ C_\pi \text{ 按流程 } \pi \text{ 计时，} \ p^* \geq \tau \text{ 时清除，否则在 } \mathbf{X}^* \text{ 补测} \end{cases} \end{cases} \tag{6-10}$$

第二检测点的候选区域取期望完成时间不超过最优值 $1+\delta$ 倍的点集

$$\mathcal{C}_\delta = \Big\{(t,h) \ \Big|\ \mathbb{E}[T(t,h)] \leq (1+\delta) \min_{(t',h')} \mathbb{E}[T(t',h')],\ \|S_2 - \mathbf{G}\| \leq R_{\min}\Big\}, \quad \delta = 5\%. \tag{6-11}$$

换回题目的坐标系：第一个检测点为 (x_1, y_1)、示向度为 θ_1 时，第二个检测点

$$x = x_1 + t\cos\theta_1 - h\sin\theta_1, \qquad y = y_1 + t\sin\theta_1 + h\cos\theta_1, \tag{6-12}$$

h 取正负两侧，候选区域是关于示向度对称的两块。

6.2.6 模型的求解

C_π 写不出闭式，故用**蒙特卡洛网格搜索算法**求解：所有候选点共用同一批样本与误差，比较时抽样噪声大部分相互抵消。步骤如下。

Step1　按式 (6-6) 抽取 800 个源距 ρ，方位偏差在 $\pm1^\circ$ 内均匀抽取，接收半径按条件分布抽取，得到 800 个干扰源样本。

Step2　在 $t \in [-100, 1100]$、$h \in [0, 600]$ 米上取步长 25 米的网格作为候选点，共 1225 个；候选区域关于 t 轴对称，只算上半平面。

Step3　对每个候选点与每个样本，在问题三的离线复刻环境中完整模拟一遍：从 S_1 走到 S_2 检测，再按流程 π 直到清除成功，记录耗时、第一次清除是否成功与定位区域直径，共 98 万次。

Step4　按样本平均得到每个候选点的 $\mathbb{E}[T]$ 与 P_{hit}，取最优点，按式 (6-11) 统计候选区域。

6.2.7 求解结果与检验

把只看定位效果时的最优点、以及不横移只沿示向度前进的点也放在同一批样本上评估，结果见表 4。

表 4　不同第二检测点在同一批 800 个样本上的表现，D 取中位数

第二检测点	$(t, \lvert h\rvert)$ 米	$\mathbb{E}[T]$ 秒	P_{hit}	D 米
期望完成时间最短	(350, 125)	207.3	0.853	221.3
第一次清除成功概率最大	(875, 600)	358.5	0.920	45.9
交会几何口径：一次交会可清除概率最大	(705, 458)	293.8	0.891	45.6
交会几何口径：期望定位区域直径最小	(908, 578)	355.8	0.909	46.2
沿示向度前进 300 米不横移	(300, 0)	269.5	0.799	300.0
沿示向度前进 600 米不横移	(600, 0)	253.0	0.806	503.4
沿示向度前进 900 米不横移	(900, 0)	292.7	0.758	750.0

由表 4 可以看出三点。其一，期望完成时间最短的第二点在示向度方向前进 350 米、横移 125 米，期望完成时间 207.3 秒。其二，只追求定位效果代价很大：命中概率最大的点离第一点 1061 米，直径缩到 45.9 米，但要 358.5 秒，多出 73%。其三，横移不能省：

同样前进 600 米不横移要 253.0 秒，多 22%，因为两条视线几乎共线，直径超过 500 米。所以第二点应放在走向干扰源的路上，偏出一小段即可。

结果如图 18 所示。(a) 是期望完成时间在 (t, h) 平面上的分布，颜色越深时间越长，白线围出 $\delta = 5\%$ 的候选区域，星号为最优点：低谷在示向度前方 100 到 600 米、离轴 50 到 200 米处，离轴越远时间涨得越快。(b) 把表 4 的七个点画成坡度图，左端是期望时间与最优值之比，右端是定位区域直径与最小值之比：时间口径的最优点左端为 1.00 而右端接近 5，定位口径的点恰好相反，线越交叉说明两个目标越冲突。没有一个点能同时取到两个口径的最优。

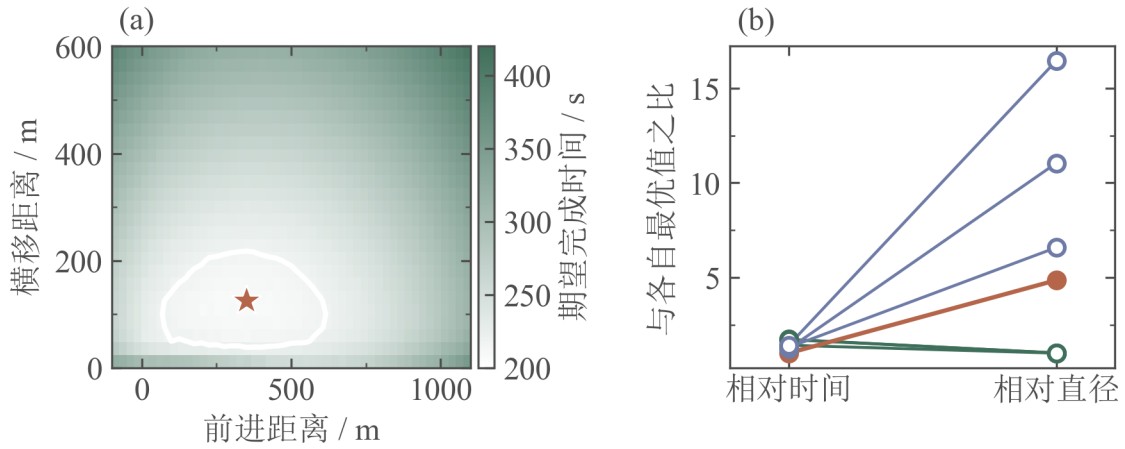


图 18　第二检测点的两个口径：(a) 期望完成时间与候选区域；(b) 七个候选点的相对时间与相对直径

候选区域的范围见表 5。

表 5　不同容差下的第二检测点候选区域，面积为两侧之和

容差 δ	面积平方米	t 的范围米	$\lvert h\rvert$ 的范围米	区域内 P_{hit} 的均值
5%	1.56×10^5	75 到 600	50 到 200	0.842
10%	2.85×10^5	0 到 700	25 到 250	0.845

由表 5 可以看出，$\delta = 5\%$ 时候选区域位于示向度前方 75 到 600 米、两侧 50 到 200 米，面积 1.56×10^5 平方米，放宽到 10% 时各扩出一圈。区域紧贴示向度而不在其正上，正是前进为主、横移不为零的几何体现。

模型用两种方式检验。其一，抽样稳定性：样本数减到 200、网格步长放大到 50 米重算，最优点为 (350, 100)、期望完成时间 206.1 秒，与 800 个样本相差 0.6%，候选区域基本重合。其二，与问题三一致：问题三在每个源上用的是同一流程 π，1000 局平均每局只有 1.3 次清除落空，说明用它评价第二点是合理的。

问题二的答案：按题目字面的定位效果口径，第二点走到交会角接近 90° 的位置，最优第二点为沿示向度前进 908 米、向任一侧横移 578 米，期望定位区域直径为 50.2 米；候选区域取源距在 5% 到 95% 可信区间内都能收到信号且交会角不低于 30° 的点集，面积为 4.14×10^5 平方米，位于示向度前方 473 到 1140 米、两侧 308 到 848 米。按本题代价结构下更实用的时间口径，第二点放在走向干扰源的路上并偏出一小段，最优第二点为沿示向度前进 350 米、向任一侧横移 125 米，期望完成时间为 207.3 秒，第一次清除成功的概率为 0.853，比定位效果口径的点快 42%；候选区域为示向度前方 75 到 600 米、两侧 50 到 200 米，面积为 1.56×10^5 平方米。两套答案都由式 (6-12) 换回任意第一点与示向度的坐标。

6.3 问题三：全向干扰源的自动搜索定位与清除

6.3.1 思路框架

本问的推理链如图 19 所示：由代价结构判明时间由行走主导，于是拆成覆盖、路线与定位清除三件事，分别给出正六边形扫描点、滚动最短路与期望耗时取舍，合成模型后用多策略对照与模拟器演练检验。

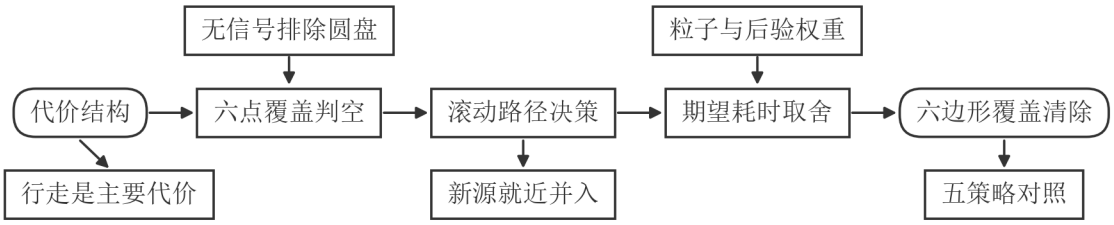


图 19 问题三的思路框架

6.3.2 可行域与后验权重

为每个频道 $c \in \{1,\dots,20\}$ 维护可行位置集合 $\mathcal{F}_c$，初始为整个目标区域 $\mathcal{D}_0$。一次在 $\mathbf{p}$ 处对频道 c 的检测把 $\mathcal{F}_c$ 更新为

$$\mathcal{F}_c \leftarrow \mathcal{F}_c \cap \begin{cases} \{\mathbf{X} : |\arg(\mathbf{X}-\mathbf{p}) - \theta| \le \varepsilon,\ 5 < \|\mathbf{X}-\mathbf{p}\| \le R_{\max}\}, & \text{返回示向度}\theta \\ \{\mathbf{X} : \|\mathbf{X}-\mathbf{p}\| \le 5\}, & \text{返回信号过强} \\ \{\mathbf{X} : \|\mathbf{X}-\mathbf{p}\| > R_{\min}\}, & \text{返回无信号} \end{cases} \tag{6-13}$$

清除返回未发现时再交上 $\{\mathbf{X} : \|\mathbf{X}-\mathbf{p}\| > r_c\}$；若在 $\mathbf{p}$ 处测到而在 $\mathbf{p}'$ 处没测到，还要交上两点中垂线靠 $\mathbf{p}$ 的半平面。这四条正是 5.3 节的四类约束。可行域内各位置并不等可能，接收半径越勉强的位置越不可能，于是给位置 $\mathbf{X}$ 赋后验权重

$$w_c(\mathbf{X}) = \Pr\Big\{ \max_{\mathbf{p}\in\mathcal{P}_c} \|\mathbf{X}-\mathbf{p}\| \le R < \min_{\mathbf{p}'\in\mathcal{P}'_c} \|\mathbf{X}-\mathbf{p}'\| \Big\}, \tag{6-14}$$

其中 $\mathcal{P}_c$ 与 $\mathcal{P}'_c$ 分别为频道 c 测到与未测到信号的检测点，R 在 $[R_{\min}, R_{\max}]$ 上均匀分布。计算时在 $\mathcal{F}_c$ 内均匀撒 900 个粒子按此加权。

6.3.3 覆盖约束

未测到信号的频道 c 要被判为空，必须保证整个目标区域都在某个无信号检测点的 $R_{\min}$ 之内，即

$$\mathcal{D}_0 \subseteq \bigcup_{\mathbf{q}\in\mathcal{P}'_c} B(\mathbf{q}, R_{\min}). \tag{6-15}$$

扫描点取原点加外圈正六边形的 6 个顶点 $\mathbf{q}_k = a\,(\cos(k\pi/3),\ \sin(k\pi/3))$。

定理 5 (六点覆盖)**.** *原点加半径为 a 的正六边形顶点这 7 个点的 $R_{\min}$ 圆盘盖满 $\mathcal{D}_0$，当且仅当*

$$\sqrt{R_0^2 + a^2 - \sqrt{3}R_0 a} \le R_{\min} \quad \text{且} \quad a \le \sqrt{3}\,R_{\min}. \tag{6-16}$$

外圈换成 5 个等分点时，无论 a 取何值都盖不满。

证明. 外圈相邻两个顶点的角平分方向上，边界点 $W = R_0(\cos 30^\circ, \sin 30^\circ)$ 到最近扫描点 $\mathbf{q}_0$ 的距离最大，由余弦定理即得式 (6-16) 的第一式；圆盘并集在内圈的最薄处出现在相邻两顶点的角平分方向、距原点 $R_{\min}$ 处，要求它被原点的圆盘或两个外圈圆盘盖住，化简得 $a \le \sqrt{3}R_{\min}$。若外圈只有 5 个等分点，边界上相邻两点的角平分方向上的点到最近顶点的距离至少为 $R_0 \sin 36^\circ = 1058$ 米，超过 $R_{\min} = 1000$ 米，故必留缺口。 □

实际判定用边长 15 米的网格，整格落入圆盘才算盖住，相当于把 $R_{\min}$ 收紧 10.6 米，由式 (6-16) 解得 $1148 \le a \le 1714$ 米；外圈越小路线越短，取 $a = 1150$ 米，最紧处的距离为 988.7 米。另外，测到过示向度的频道必有干扰源，总数不超过 16，所以已清除与已发现未清除之和达到 16 时其余频道直接判空。

定理 5 的两个结论如图 20 所示。(a) 是取 $a = 1150$ 米的六点方案，七个圆盘把目标区域盖满，最紧的位置 W 在边界上两个外圈顶点的正中；(b) 把外圈换成五个等分点、半径不变，边界上出现五块盖不住的缺口。可见六点不是凑出来的数，而是覆盖条件允许的最少点数。

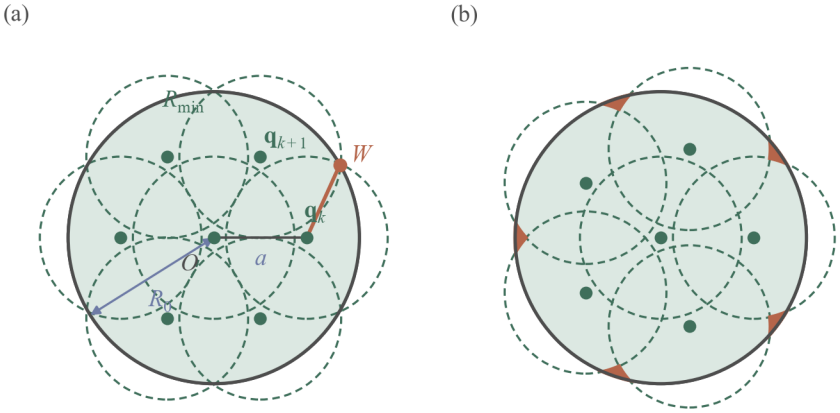


图 20　外圈扫描点个数与覆盖：(a) 六个顶点盖满；(b) 五个顶点留下缺口

6.3.4　路线

记当前位置为 $\mathbf{p}$，还要去的扫描点与已发现未清除的源构成节点集合 $\mathcal{N}$，源用后验均值代表。下一个目标取从 $\mathbf{p}$ 出发经过 $\mathcal{N}$ 全部节点的最短开放路径上的第一个节点：

$$\pi^{*} = \arg\min_{\pi} \sum_{i=1}^{|\mathcal{N}|} \big\| \boldsymbol{n}_{\pi(i)} - \boldsymbol{n}_{\pi(i-1)} \big\|, \qquad \boldsymbol{n}_{\pi(0)} = \mathbf{p}. \tag{6-17}$$

每做完一件事重新求一次，新发现的源随时并入。扫描点周围 $R_{\min}$ 内没有任何频道的未排除位置时就删去。

滚动重排的效果如图 21 所示。空心圆是外圈扫描点，天线记号是陆续发现的源，三条线是同一局里三个时刻的路线：细虚线是开局只有扫描点时的路线，点划线是发现第一个源之后重排的路线，粗实线是再发现两个源之后的路线。新发现的源不排到队尾，而是按距离插进最近的两个节点之间。正是这种就近并入使扫描顺路完成，不必先扫完一圈再回头清除。

6.3.5　定位清除决策

走到一个源时要在直接试清除与先补测之间选择。记后验下使 20 米清除圆内概率最大的点为 $\mathbf{X}^{*}$、该概率为 p^{*}，两种做法的期望耗时为

$$\begin{aligned} J_{\text{clear}} &= \frac{\|\mathbf{p} - \mathbf{X}^{*}\|}{v} + p^{*} t_c + (1 - p^{*})\big(t_l + Q(\mathbf{X}^{*})\big), \\ J_{\text{meas}}(\mathbf{Y}) &= \frac{\|\mathbf{p} - \mathbf{Y}\|}{v} + Q(\mathbf{Y}), \\ Q(\mathbf{Y}) &= t_m + t_s \mathbf{1}\{c \neq c_0\} + \mathbb{E}\Big[\frac{\|\mathbf{Y} - \mathbf{X}^{*}_{\mathbf{Y}}\|}{v} + t_c + (1 - p^{*}_{\mathbf{Y}})\kappa\Big], \end{aligned} \tag{6-18}$$

其中 $\mathbf{X}^{*}_{\mathbf{Y}}$ 与 $p^{*}_{\mathbf{Y}}$ 为在 $\mathbf{Y}$ 处补测之后的最优清除点与命中概率，κ 为再次落空后的兜底耗时。问题二的结论正是这一决策在第二个检测点上的特例。

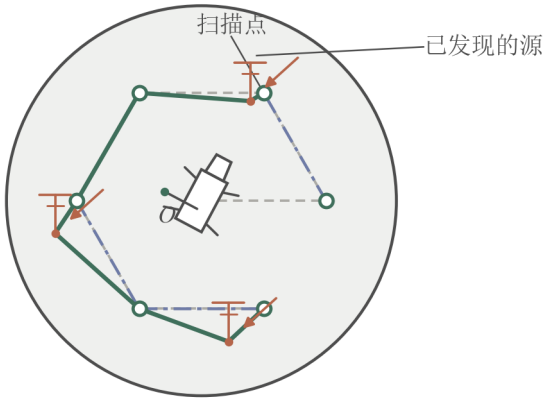


图 21　滚动最短路的三次重排

式 (6-18) 两支的取舍如图 22 所示，两种做法画成两条动作链，每一节的长度正比于耗时。上面一条是直接试清除：走过去，成功就结束，落空则分叉接上一条走、测、走、清的长尾巴。下面一条是先补测：走到补测点切频道测一次，可行域收窄后再走一小段清除，链更长但没有长尾巴。可行域细长时命中概率低，长尾巴出现得频繁，补测更短；可行域收小后直接试清除更划算。式 (6-18) 就是比这两条链的期望长度。

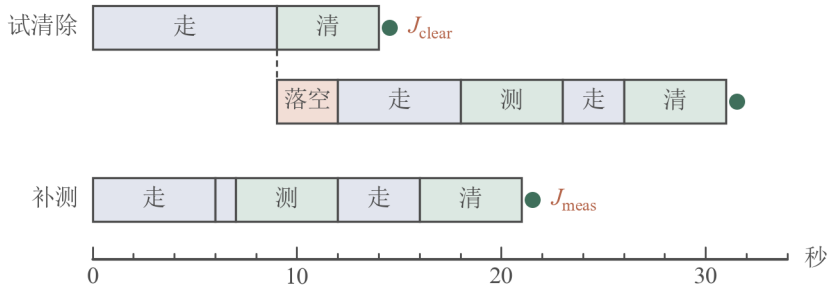


图 22　试清除与补测两条动作链的耗时对照

6.3.6　六边形覆盖清除模型

综上所述，最终得到**六边形覆盖清除模型**为

$$
\begin{cases}
\min\ \bar{T} = \dfrac{T_{\text{total}}}{N_{\text{cleared}}} \\
s.t. \quad \begin{cases}
\mathcal{F}_c \text{ 按式 (6-13) 与中垂线约束更新，权重按式 (6-14)}, \quad c = 1, \ldots, 20 \\
\text{频道} c \text{ 判空} \iff \text{式 (6-15) 成立，或} N_{\text{cleared}} + |\{\text{已发现未清除}\}| \geq 16 \\
\text{扫描点} \{\boldsymbol{O}\} \cup \{\mathbf{q}_k\}_{k=0}^{5} \text{ 满足式 (6-16)} \\
\text{下一目标取式 (6-17) 的首节点} \\
\text{对目标频道执行 } \arg\min\{J_{\text{clear}},\ \min_{\mathbf{Y}} J_{\text{meas}}(\mathbf{Y})\} \\
N_{\text{cleared}} = N, \qquad T_{\text{real}} \leq 1200 \text{ 秒}, \qquad T_{\text{total}} \leq 360000 \text{ 秒}
\end{cases}
\end{cases}
\tag{6-19}
$$

6.3.7　滚动路径决策算法

模型 (6-19) 的动作序列取决于此前全部观测，无法事先整体求出，故用**滚动路径决策算法**逐步求解。步骤如下。

Step1　在原点对 20 个频道各检测一次，耗时 119 秒，发现原点附近的源并排除原点周

围的位置。

Step2　更新各频道的粒子与权重，按式 (6-15) 与源数上限判定哪些频道为空。

Step3　以还需要去的扫描点和已发现源的后验均值为节点，用最近邻法得初始路径，再用 2-opt 与 Or-opt 交换改进[12,13]，取第一个节点为目标。

Step4　目标是扫描点时走过去对周围仍有未排除位置的频道各检测一次，并对已发现但命中概率 $p^{*}<0.7$、距离不超过 1300 米、交会角不小于 20° 的频道顺带测一次示向度。

Step5　目标是源时按式 (6-18) 在直接试清除与 43 个候选补测点之间取期望耗时最小者执行，期望用 20 个真值假设的蒙特卡洛平均估计，直到清除成功。

Step6　重复 Step2 到 Step5，直到全部频道清除或判空，调用退出指令。

算法流程如图 23 所示。第一个判断框决定是否结束，第二个按最短路首节点的类型把下一件事分成扫描与定位清除两支，两支做完都回到更新可行域，所以整个算法是一个以更新可行域为枢纽的循环。

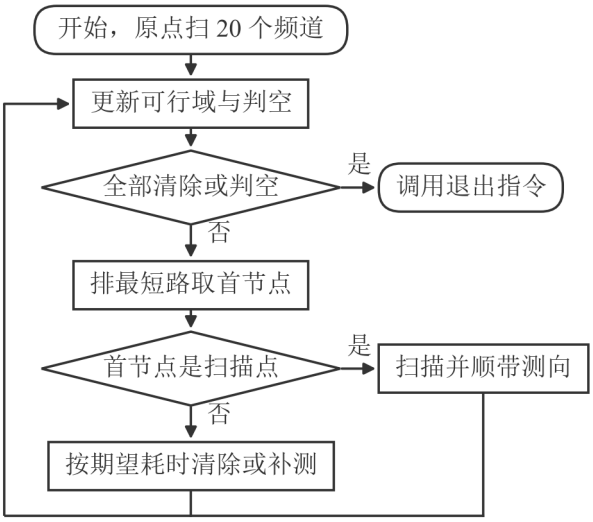


图 23　滚动路径决策算法的流程

6.3.8　多策略对照

把五种方法放在种子 70000 起的同一批 800 局案例上对比：逐步贪心每步只挑单位时间收益最大的动作；扫描点优先先排扫描点，源只在绕路不超过 400 米时顺路清除；虚拟节点在路线里加入尚未发现源的虚拟节点；最短路把扫描点与源一起排最短路、外圈 1200 米；本文策略在其上再加 Or-opt、外圈 1150 米与清除后顺带测向。结果见表 6。

表 6　问题三五种决策方法在同一批 800 局案例上的对比

指标	逐步贪心	扫描点优先	虚拟节点	最短路	本文策略
清除比例	1.000	1.000	1.000	1.000	1.000
平均定位清除时间均值/秒	356.7	276.2	240.3	244.2	**238.6**
平均定位清除时间中位数/秒	357.8	270.8	237.3	241.3	**235.5**
逐局用时最短的局数占比	0.2%	0.6%	40.2%	18.2%	**43.1%**
程序运行时间均值/秒	0.37	0.51	0.56	0.53	0.53

由表 6 可以看出，五种方法都清除了全部干扰源，差别在时间。本文策略最短，为 238.6 秒，比逐步贪心缩短 33.1%，比扫描点优先缩短 13.6%。逐步贪心每步只看眼前，最后还要花近两成时间收尾扫描；扫描点优先把扫描排在前面，发现的源要等绕完一圈再回头清。虚拟节点的均值只多 0.7%，但有 52.8% 的局比本文策略慢，因为虚拟节点撒在随机位置，会把路线引向其实没有源的方向。五种方法的单局运行时间都不到 1 秒。

所以问题三采用本文策略。

6.3.9　求解结果与检验

在种子 50000 起、从未参与任何比较与调整的 1000 局离线案例上评估最终策略，结果见表 7。

表 7　问题三最终策略在 1000 局离线案例上的结果

指标	结果
被清除干扰源个数的比例	1.000
全部清除的局数占比	100%
平均定位清除时间的均值	238.7 秒
全部局总时间除以总清除个数	234.2 秒
平均定位清除时间的中位数	235.9 秒
平均定位清除时间的九成分位	286.3 秒
最后一次清除之后的收尾时间占比	1.8%
平均每局检测次数与清除落空次数	100 次，1.3 次
单局程序运行时间上限	0.8 秒

由表 7 可以看出，1000 局每一局都清除了全部干扰源，平均定位清除时间为 238.7 秒。收尾时间只占 1.8%，原因是扫描点与源排在同一条路线上，扫描顺路完成，最后一个源清除时覆盖通常也已完成。平均每局只有 1.3 次清除落空，说明式 (6-18) 的决策很少在可行域还大时贸然试清除。

为说明各项设计的作用，每次只改一项在同一批案例上配对比较，清除比例都是 1.000。外圈半径应取覆盖允许的最小值，但必须与覆盖判定的精度匹配：网格为 30 米时收紧量为 21.2 米，1150 米的外圈判为盖不满，程序只能补扫描点，反而慢 30.0%；网格改为 15 米后判为盖满，快 0.9%。扫描要少而顺路：清除后顺带测向快 0.9%，而清除后原地扫描全部未发现频道要多测十来个频道，慢 2.3%；路线加 Or-opt 改进快 0.7%。

为判断还有多少压缩余地，对 400 局按动作拆分时间并与全知路线对照。每局 3032 秒中检测 100 次占 601 秒，清除占 69 秒，行走 11.8 公里占其余 2360 秒，其中定位测向真正多绕的路只有每局 363 米。假如开局就知道全部源的位置，走完扫描点与全部源的最短开放路线均值为 10.5 公里，实际行走比它多 12.8%。按全知路线折算，平均定位清除时间的下限约 212 秒，现有策略的 238.7 秒离它 13%，多出的主要是源陆续发现带来的回头路。

时间构成与各策略的差距如图 24 所示。(a) 按干扰源个数把每个源摊到的时间拆成行走、检测与清除三层：总高度由 10 个源时的 289 秒降到 16 个源时的 198 秒，降的几乎全是行走与检测，清除那一层始终很薄。原因是七个扫描点的路线与检测次数基本固定，源越多摊得越薄。(b) 是五种方法在各个干扰源个数上的平均定位清除时间，颜色越深越长：本文策略整行最浅，逐步贪心整行最深，且各行都随源数增加而变淡，说明策略排序与源数无关。

在模拟器上用本文策略连续做了 106 局问题三演练，其中 2 局因本机内存不足提前退出、与策略无关，予以剔除，其余 104 局的统计与离线结果对照列于表 8。

由表 8 可以看出，104 局演练共 1326 个干扰源全部清除，比例为 1.000；平均定位清除时间按题目定义取总时间除以被清除个数，为 232.5 秒，与离线复刻的 234.2 秒只差 0.7%。这说明离线复刻环境对模拟器的刻画可信，离线得出的对比与灵敏度结论可以直接用于模拟器。单局程序运行时间最长 5.1 秒，多出的部分是网络通信。三次正式测试的结果按表 9 填写，从模拟器导出的加密日志放入支撑材料。

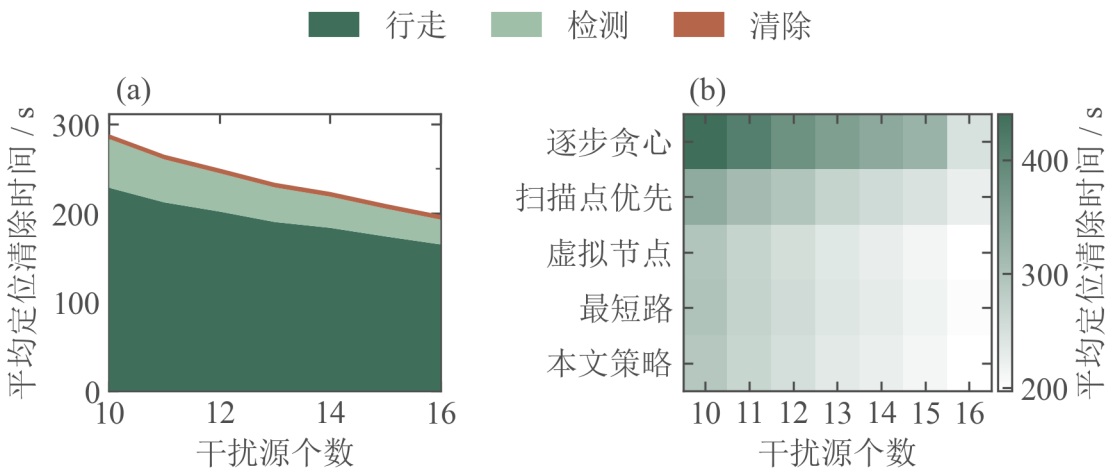


图 24　问题三的离线演练结果：(a) 按干扰源个数分组的时间构成；(b) 五种方法的平均定位清除时间

表 8　问题三模拟器演练测试的统计结果

指标	模拟器演练 104 局	离线复刻 1000 局
干扰源总数与被清除个数	1326，1326	—
被清除干扰源个数的比例	1.000	1.000
平均定位清除时间	232.5 秒	234.2 秒
逐局平均定位清除时间的均值	236.0 秒	238.7 秒
逐局平均定位清除时间的中位数	232.6 秒	235.9 秒
单局程序运行时间的均值与最长	2.0 秒，5.1 秒	0.5 秒，0.8 秒

表 9　问题三正式测试结果

序号	测试案例编码	清除干扰源个数	平均定位清除时间/秒	程序运行时间/秒
测试 1				
测试 2				
测试 3				

问题三的答案：原点加半径 1150 米正六边形的七个扫描点保证覆盖，扫描点与已发现的源排成滚动更新的最短开放路，每步按期望耗时在试清除与补测之间取舍。1000 局离线案例被清除干扰源个数的比例为 1.000、平均定位清除时间为 238.7 秒，比逐步贪心缩短 33.1%；模拟器演练 104 局比例为 1.000、平均定位清除时间为 232.5 秒。

6.4　问题四：全向与定向干扰源混合时的检测与清除

6.4.1　思路框架

问题四与问题三的差别只在无信号这条观测的含义，改动的是覆盖约束与可行域，路线与决策沿用。推理链如图 25 所示：先由半圆辐射推出凸包覆盖判据，据此把扫描点设计写成集合覆盖问题并用退火求解，再给可行域补上定向方向的圆弧，运行中按独占样本动态改动扫描点。

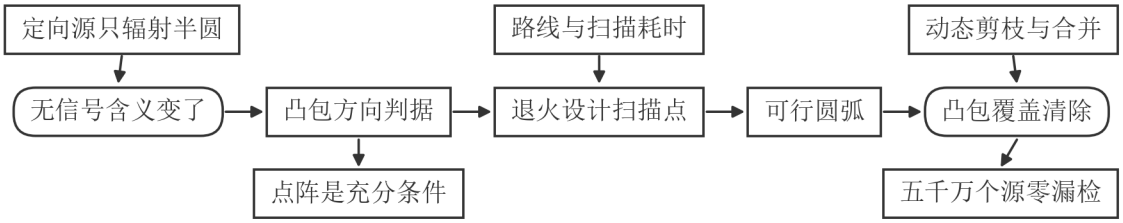


图 25　问题四的思路框架

6.4.2　方向覆盖判据

定向源位于 $\mathbf{G}$、方向为 φ 时，检测点 $\mathbf{q}$ 收得到信号当且仅当 $\|\mathbf{q}-\mathbf{G}\| \le R$ 且 $|\arg(\mathbf{q}-\mathbf{G})-\varphi| \le 90^\circ$，即 $\mathbf{q}$ 落在以 $\mathbf{G}$ 为圆心、朝向 φ 的半圆内。要让 $\mathcal{Q}$ 不论源在哪里、朝向哪里都能发现它，有如下判据。

定理 6 (凸包判据). *对目标区域内的位置 $\mathbf{G}$，任意朝向的定向源都能被 $\mathcal{Q}$ 发现，当且仅当*

$$\mathbf{G} \in \operatorname{conv}\big(\mathcal{Q} \cap B(\mathbf{G}, R_{\min})\big). \tag{6-20}$$

证明. 记 $\mathcal{Q}_{\mathbf{G}} = \mathcal{Q} \cap B(\mathbf{G}, R_{\min})$，$\mathbf{u}$ 为朝向 φ 的单位向量。朝向 φ 的源被发现，等价于存在 $\mathbf{q} \in \mathcal{Q}_{\mathbf{G}}$ 使 $(\mathbf{q}-\mathbf{G})\cdot\mathbf{u} \ge 0$。若 $\mathbf{G}$ 在凸包内而某个 $\mathbf{u}$ 使所有 $(\mathbf{q}-\mathbf{G})\cdot\mathbf{u} < 0$，则凸包整个落在过 $\mathbf{G}$ 的开半平面内，与 $\mathbf{G}$ 属于凸包矛盾。反之若 $\mathbf{G}$ 不在凸包内，由分离超平面定理存在 $\mathbf{u}$ 使所有 $(\mathbf{q}-\mathbf{G})\cdot\mathbf{u} < 0$，朝向 $\mathbf{u}$ 的源就测不到。 □

推论 2. *边长 $s \le R_{\min}$ 的三角点阵满足式 (6-20)。*

证明. 任一位置落在某个小三角形内，三个顶点离它都不超过 $s \le R_{\min}$，且它在这三个顶点的凸包里，由定理 6 即得。 □

定理 6 的两个方向如图 26 所示，扇形为定向源的辐射半圆。(a) 中 $\mathbf{G}$ 落在四个扫描点的凸包里，图上叠画的三个不同朝向的半圆各罩住至少一个扫描点，所以无论方向转到哪里都能被发现；(b) 中 $\mathbf{G}$ 落在凸包之外，存在一条分离直线把全部扫描点甩到一侧，朝相反方向的半圆一个也罩不住。可见方向覆盖要求扫描点从各个方向把每个位置围住，比式 (6-15) 的圆盘覆盖严格得多。

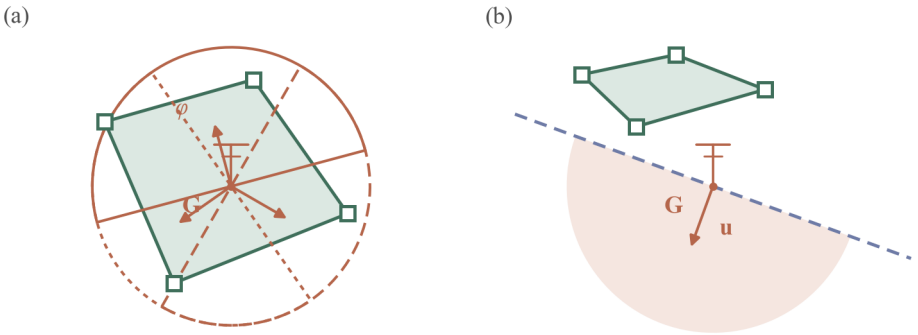


图 26　凸包判据：(a) 位置在凸包内，任意朝向都被发现；(b) 位置在凸包外，存在漏检朝向

由推论 2 三角点阵可行，但它在边界外要多铺一圈，路线长达 25 公里。故把扫描点的选择写成优化问题：在满足式 (6-20) 的点集中使扫描路线长度与扫描耗时之和最小，

$$\min_{\mathcal{Q}} \; L_{\mathrm{TSP}}(\mathcal{Q}) + |\mathcal{Q}|\, c_s, \qquad \text{s.t. } \forall(\mathbf{G}_j,\varphi_j):\ \exists \mathbf{q}\in\mathcal{Q},\ \|\mathbf{q}-\mathbf{G}_j\| \le r_d,\ |\arg(\mathbf{q}-\mathbf{G}_j)-\varphi_j| \le \alpha_d, \tag{6-21}$$

其中 $(\mathbf{G}_j,\varphi_j)$ 为区域内均匀抽取的 12 万个源样本，并在边界上加抽朝外的样本；c_s 为每个扫描点检测约 8 个频道的耗时折算成的距离；判定时取余量 $r_d = 970$ 米、$\alpha_d = 89^\circ$。运行时判空用同一批样本与同样的余量，设计与运行口径一致。

6.4.3　联合可行域

对已发现的频道，每个可能位置 $\mathbf{X}$ 还附带类型与定向方向。在 $\mathbf{p}$ 处测到信号要求 φ 与 $\arg(\mathbf{p}-\mathbf{X})$ 相差不超过 90°，在 $\mathbf{p}'$ 处无信号且 $\|\mathbf{X}-\mathbf{p}'\| \le R$ 要求相差大于 90°。这些都是宽 180° 的圆弧，交出来仍是一段圆弧 $\Psi(\mathbf{X})$，粒子只存它的中心与半宽。权重改为

$$w_c(\mathbf{X}) = \Pr\{R \ge \max_{\mathbf{p}\in\mathcal{P}_c}\|\mathbf{X}-\mathbf{p}\|\}\Big[\eta\,\mathbf{1}\{\text{全向时与全部无信号相容}\} + (1-\eta)\frac{|\Psi(\mathbf{X})|}{360^\circ}\Big], \tag{6-22}$$

其中 η 为源是全向的先验概率，取 0.5。

圆弧的收窄过程如图 27 所示，定向方向画在一个罗盘上，最内一圈代表初始时所有可能的朝向。在 $\mathbf{p}_1$ 处测到信号，只剩朝 $\mathbf{p}_1$ 一侧的半圈；在 $\mathbf{p}_2$ 处无信号且距离在接收半径以内，去掉朝 $\mathbf{p}_2$ 一侧的半圈；在 $\mathbf{p}_3$ 处又测到信号，再去掉一块，剩下最外一圈的那段短弧。观测越多圆弧越短，弧长占整圈的比例就是式 (6-22) 中定向那一项的权重。

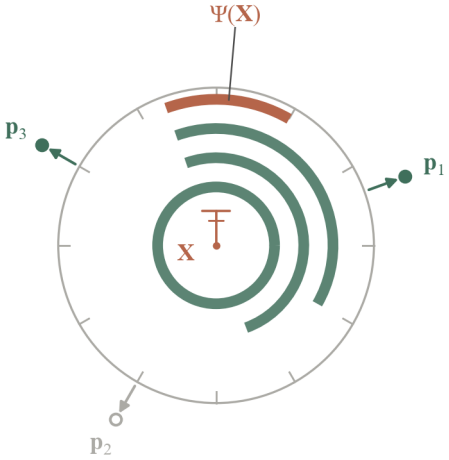


图 27　三次观测后定向方向的可行圆弧

式 (6-18) 中收到信号的概率也按圆弧算：真值在 $\mathbf{Y}$ 处收得到信号的概率等于 Ψ 中朝向 $\mathbf{Y}$ 一侧所占的比例。无信号还给出接收半径的上界：全向假设下任何无信号检测点到 $\mathbf{X}$ 的距离都不小于 R，于是每个粒子记下 R 的区间 $[R_{\text{lo}}, R_{\text{hi}}]$，在距离 d 处收到信号的概率取 $(R_{\text{hi}}-d)/(R_{\text{hi}}-R_{\text{lo}})$ 并截断到 $[0,1]$。不带上界时模型会高估收到信号的概率，机器狗在定向源背面连测七八次才试清除；带上界后概率低于 0.3 的候选检测点直接不考虑。

6.4.4　动态扫描点

设计出的 $\mathcal{Q}$ 有重叠，运行中又不断有新的检测点排除位置，所以不是每个设计点都非去不可。记未发现频道仍未排除的样本集合为 $\mathcal{S}$，扫描点 $\mathbf{q}_k$ 独占的样本为

$$U_k = \{\,j \in \mathcal{S} :\ \mathbf{q}_k \text{ 盖住} j,\ \text{其余未扫描的点都盖不住} j\,\}, \tag{6-23}$$

只有 U_k 才必须由 $\mathbf{q}_k$ 来盖。每一步按三条规则改动：$U_k=\varnothing$ 的点剪掉；当前位置能盖住某个 U_k 就原地扫描并放弃 $\mathbf{q}_k$；路线上相邻的两个扫描点若存在一点能盖住二者合起来独占的样本，且合并后路线增加的长度小于一次扫描折算的距离 c_s，就合并成一个。候选位置取 700 米内间距 100 米的网格，判覆盖时余量收紧到 950 米与 88.5°。三条规则都保持未扫描点的覆盖并集包含 $\mathcal{S}$，覆盖保证不变。

三条规则的效果如图 28 所示。小点是仍未排除的样本，按归属染色：与某个扫描点同色的点只有那个扫描点能盖住，灰点被两个以上扫描点同时盖住。$\mathbf{q}_2$ 周围一个同色点也没有，说明任务已被别的点接管，于是打叉剪掉；$\mathbf{q}_3$ 与 $\mathbf{q}_4$ 各有一小撮同色点且靠得很近，二者之间的新点能一起盖住，于是合并。剩下的扫描点仍盖住全部样本，覆盖没有被削弱。

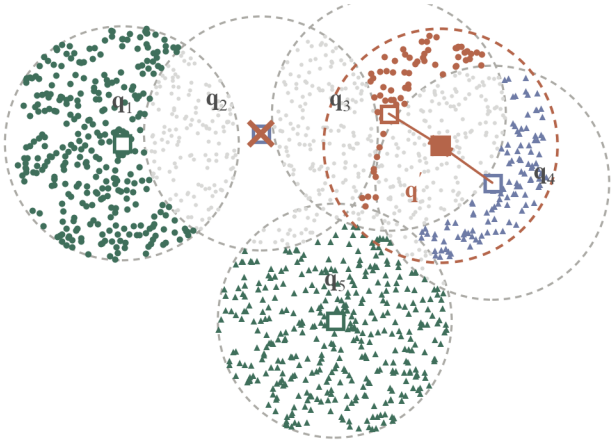


图 28　按独占样本剪掉与合并扫描点

6.4.5　凸包覆盖清除模型

综上所述，最终得到**凸包覆盖清除模型**为

$$
\begin{cases}
\min \ \ \bar{T} = T_{\text{total}} / N_{\text{cleared}} \\
s.t. \quad \begin{cases}
\mathcal{F}_c \text{ 按式 (6-13) 前两行更新，权重按式 (6-22)} \\
\text{频道} c \text{ 判空} \iff \text{无信号检测点按式 (6-20) 盖住全部样本，或源数达到16} \\
\mathcal{Q} \text{ 取式 (6-21) 的解，运行中按式 (6-23) 剪枝、顶替与合并} \\
\text{下一目标取式 (6-17) 的首节点，定位清除按式 (6-18)} \\
N_{\text{cleared}} = N, \qquad T_{\text{real}} \le 1200 \text{ 秒}, \qquad T_{\text{total}} \le 360000 \text{ 秒}
\end{cases}
\end{cases}
\tag{6-24}
$$

6.4.6　模拟退火布点算法

扫描点设计离线完成一次，运行时的算法与问题三的 Step1 到 Step6 相同，只把扫描点、判空条件与粒子权重换成式 (6-21)、式 (6-20)、式 (6-22)。式 (6-21) 是带路线目标的集合覆盖问题，属于 NP 难问题，用**模拟退火布点算法**求解，步骤如下。

Step1　在区域及其外侧 1000 米内取间距 100、150、200 米的三角点阵作候选点并随机平移，共 48 个变体；每个变体从原点出发贪心选点，盖满后按贡献从少到多试删冗余点，取总代价最小的一组作退火起点，得 32 个点、路线 17.4 公里。

Step2　模拟退火。每步随机删一个点、删一个再加一个，或把一个点按标准差 30、80、200 米的正态扰动挪开；覆盖不破才评估，代价下降就接受，上升按 Metropolis 准则以 $\exp(-\Delta/T)$ 接受，温度由 400 线性降到 1。16 条链并行各走 12 万步，每点折算的频道数在 7 到 12 之间轮换，取代价最小的一条。

Step3　用另抽的 200 万个源、接收半径取 1000 米整、角度上限取 90° 独立核验；有漏检就只为漏检的源追加最少的点，换一批样本再验，直到零漏检；再用一千万个新抽的源终检。

Step4　终检通过后再抽三千万个源复核，把最近的一个扫描点挪动 50 米补上 3 个漏检源，另抽两千万个源复核零漏检，最后按最短路排出扫描顺序。

三角点阵与退火设计的差别如图 29 所示，两套点画在同一幅俯视图上：空心圆是间距 950 米的三角点阵共 27 个，均匀铺满区域并在边界外多铺一圈；实心点是退火设计的 25 个点，内部稀疏、边界附近密集，粗线是它们的扫描路线。可见退火不是把点阵稀释，而是把点从内部搬到边界：内部位置四面都有扫描点、容易满足凸包判据，边界位置只有内侧有点。

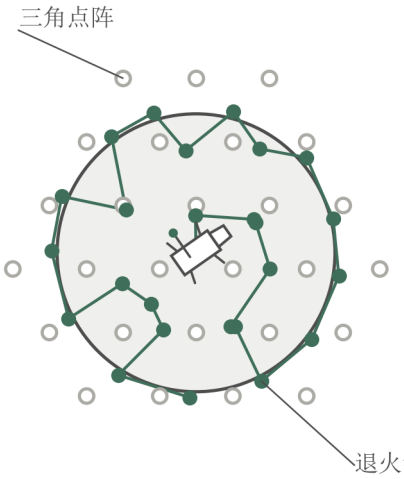


图 29　三角点阵与退火设计的扫描点对照

不同覆盖设计的对比见表 10。前四行是固定扫描点版本在同一批 300 局案例上的结果，后两行是退火设计配合动态扫描在另一批同种子 300 局案例上的结果。

表 10　问题四覆盖扫描点的不同设计，平均时间指平均定位清除时间

设计	点数	路线/公里	漏检个数与核验样本	平均时间/秒
三角点阵 950 米，网格判定	27	25.2	0，2×10^{6}	600.6
贪心覆盖，未补漏	27	19.2	7，2×10^{6}	531.2
贪心挪点补漏，余量 990 米	34	17.6	0，10^{7}	527.4
贪心挪点补漏，余量 970 米	32	17.4	0，10^{7}	527.8
同上，加动态扫描	32	17.4	0，10^{7}	509.3
模拟退火补漏，加动态扫描	**25**	**17.2**	**0，5×10^{7}**	**467.4**

由表 10 可以看出四点。其一，扫描点个数相同时路线长度差别很大：三角点阵与未补漏的贪心设计都是 27 个点，路线却相差 6 公里，时间相差 11.6%，说明扫描点的选择必须把路线纳入目标。其二，只按抽样样本设计会在样本稀疏处留下漏洞，未补漏时两百万个源里漏检 7 个；补漏只追加点，两轮就收敛。其三，余量由 990 米加大到 970 米后点数反而少 2 个、时间几乎相同，对真实条件偏差的容忍更大。其四，点数比路线更要紧：退火把 32 个点压到 25 个而路线只短 0.2 公里，时间却由 509.3 秒降到 467.4 秒，缩短 8.2%。退火从 32 点与 25 点起步都收敛到 21 到 23 个点，补漏追加后都是 25 个点上下，24 点与 27 点设计在同批案例上分别为 476.0 秒与 468.8 秒，所以最终取 25 个点、路线 17.2 公里。

时间花在哪里也拆得很清楚：固定 32 点版本每局平均检测 340 次，其中扫描点上对未发现频道的检测就占 324 次，定位测向只有 12 次；每局 6353 秒中行走 4234 秒、检测 2040 秒。扫描点上的检测次数等于扫描点个数乘每个点上未排除的频道数，后者平均约 10 个。所以减少扫描点个数是最大的杠杆，每少一个约省 60 秒，而路线每短 300 米才省同样的时间。动态扫描在同一批 40 局上的配对比较也印证这一点：32 点设计冗余多，剪枝与合并省 35 次检测、缩短 2.5%；25 点设计本就紧凑，只省 12 次、缩短 1.5%，两种情形清除比例都是 1.000。

6.4.7　多策略对照

把五种方案放在种子 70000 起的同一批 400 局案例上对比：逐步贪心沿用问题三早期的逐步决策并加入定向源的背面处理；三角点阵覆盖用间距 950 米的点阵；扫描点优先换用问题三的扫描点优先顺序；固定点版是贪心设计的 32 个扫描点加最短路决策；本

文策略换成退火设计的 25 个点并加动态扫描。结果见表 11。

表 11　问题四五种方案在同一批 400 局案例上的对比，平均时间指平均定位清除时间

指标	逐步贪心	三角点阵覆盖	扫描点优先	固定点版	本文策略
清除比例	0.9987	1.000	1.000	1.000	**1.000**
全清局数占比	98.2%	100%	100%	100%	**100%**
平均时间均值/秒	974.4	579.7	550.3	508.5	**474.8**
平均时间中位数/秒	1026.7	576.7	539.5	506.2	**468.2**
用时最短局数占比	3.2%	2.2%	1.0%	16.8%	**76.8%**
程序时间均值/秒	29.89	1.38	2.21	2.22	44.24

由表 11 可以看出，本文策略的平均定位清除时间为 474.8 秒，在 76.8% 的局中用时最短。与逐步贪心相比缩短 51.3%，且逐步贪心漏掉了 7 个定向源；与三角点阵覆盖缩短 18.1%，与扫描点优先缩短 13.7%，与固定点版缩短 6.6%。代价是单局运行时间由 2 秒增到 44 秒，仍远低于限制。所以问题四采用退火设计的覆盖扫描点加动态扫描与最短路决策。

6.4.8　求解结果与检验

在种子 50000 起、从未参与比较与调整的 500 局离线案例上评估最终策略，定向源个数随机，结果与问题三对照列于表 12。

表 12　问题三与问题四最终策略的离线结果

指标	问题三 1000 局	问题四 500 局
被清除干扰源个数的比例	1.000	1.000
全部清除的局数占比	100%	100%
平均定位清除时间的均值	238.7 秒	483.7 秒
全部局总时间除以总清除个数	234.2 秒	469.4 秒
平均定位清除时间的中位数	235.9 秒	479.6 秒
平均定位清除时间的九成分位	286.3 秒	614.6 秒
最后一次清除之后的收尾时间占比	1.8%	6.8%
平均每局检测次数与清除落空次数	100 次，1.3 次	284 次，4.2 次
单局程序运行时间上限	0.8 秒	61.3 秒

由表 12 可以看出，问题四的 500 局同样全部清除，平均定位清除时间 483.7 秒，约为问题三的 2.0 倍。多出的时间来自方向覆盖：扫描点由 7 个增至 25 个，每局检测次数由 100 次增至 284 次。另取只有 1 个定向源的 300 局评估，为 447.1 秒，只少 7.6%，说明即使定向源很少也必须完成方向覆盖。判空频道实际测过的位置按真实规则再核验，500 局共 2.0 亿个随机源里漏检 38 个，漏检率 1.9×10^{-7}。

结果的分布如图 30 所示。(a) 按定向源个数分组，糖头是该组中位数，细竖线连到全体中位数的基准线：四组在 445.7 到 499.9 秒之间高低交错，并不随定向源增多而上升，定向源 9 到 15 个的一组因为干扰源总数平均更多而中位数最低。可见问题四的时间主要由方向覆盖决定。(b) 是问题三与问题四逐局时间的经验分布：问题四整条阶梯在问题三右侧，中位数 479.6 秒是问题三 235.9 秒的 2.0 倍，最长一局 900.0 秒对 335.5 秒。

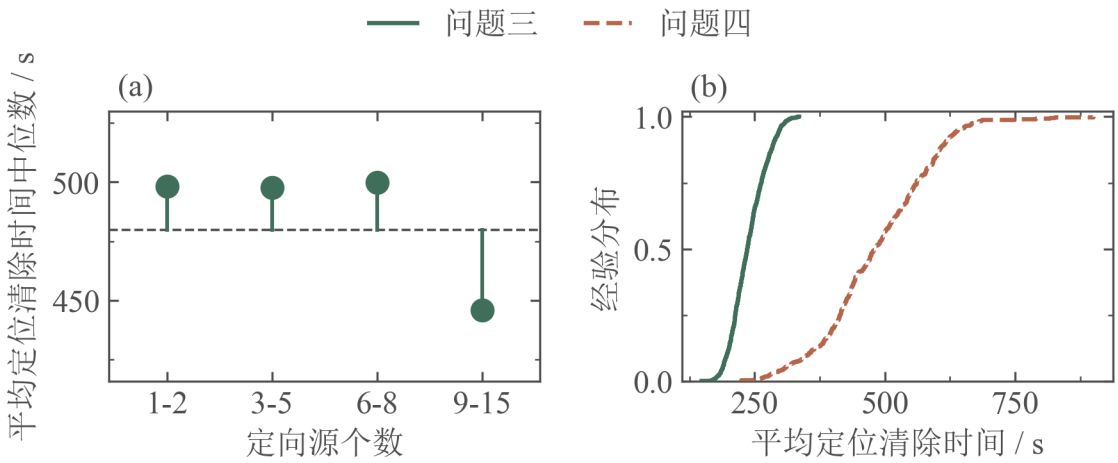


图 30　问题四的离线演练结果：(a) 按定向源个数分组的中位数；(b) 问题三与问题四的经验分布

在模拟器上用固定 32 点版本连续做了 100 局问题四演练，统计与该版本的离线结果对照列于表 13；换上退火设计与动态扫描后又做了 1 局，15 个源全部清除，平均定位清除时间 354.4 秒。

表 13　问题四模拟器演练测试的统计结果，演练与离线均为固定 32 点扫描点版本

指标	模拟器演练 100 局	离线复刻 1000 局
干扰源总数与被清除个数	1325，1325	—
被清除干扰源个数的比例	1.000	1.000
平均定位清除时间	503.0 秒	500.7 秒
逐局平均定位清除时间的均值	515.8 秒	517.0 秒
逐局平均定位清除时间的中位数	510.5 秒	508.3 秒
单局程序运行时间的均值与最长	6.5 秒，14.8 秒	2.3 秒，7.4 秒

由表 13 可以看出，100 局演练共 1325 个干扰源全部清除，比例为 1.000，平均定位清除时间 503.0 秒，与离线复刻的 500.7 秒只差 0.5%。模拟器中定向源的个数与朝向同样未知，策略没有漏掉任何一个。三次正式测试的结果按表 14 填写，从模拟器导出的加密日志放入支撑材料。

表 14　问题四正式测试结果

序号	测试案例编码	清除干扰源个数	平均定位清除时间/秒	程序运行时间/秒
测试 1				
测试 2				
测试 3				

问题四的答案：按凸包判据设计 25 个扫描点、路线 17.2 公里，五千万个随机源零漏检，运行中按独占样本剪枝、顶替与合并；可行域给每个粒子附带定向方向的可行圆弧，其余沿用问题三。500 局离线案例被清除干扰源个数的比例为 1.000、平均定位清除时间为 483.7 秒，比逐步贪心缩短 51.3% 且不漏定向源；模拟器演练 100 局比例为 1.000、平均定位清除时间为 503.0 秒。

7.　灵敏度分析

策略对干扰源个数最敏感，对示向度误差上界不敏感，只有接收半径下界被突破时才会漏源。下面分两类检验：第一类改变干扰源个数与定向源个数；第二类是稳健性检

验，策略仍按题目给定的误差上界 1° 与接收半径下界 1000 米推理，而案例的真实条件更差。问题三每个取值在同一批 400 局上重跑，问题四在 300 局上重跑，策略参数取第 6 章的最终值。第一类结果见表 15。

表 15　干扰源个数与定向源个数改变时的表现，所有情形清除比例均为 1.000

问题	改变的量	取值	平均定位清除时间/秒	平均每局检测次数
问题三	干扰源个数	10	289.5	110
		13	235.7	101
		16	196.9	88
问题四	干扰源个数	10	628.9	334
		16	334.8	210
	定向源个数	1	447.1	256
		3	456.5	265
		6	478.0	275

由表 15 可以看出两点。其一，干扰源越少，平均定位清除时间越长：问题三由 16 个减到 10 个时增加 47.0%，问题四增加 87.8%，因为覆盖扫描的路线与检测次数几乎与源数无关，源越少摊得越多；源为 16 个时源数上限推理还能省掉剩余频道的扫描。其二，问题四对定向源个数不敏感：由 1 个增至 6 个只增加 6.9%，因为方向覆盖不论定向源多少都要完成。稳健性检验的结果见表 16。

表 16　真实条件比策略假设更差时的表现，策略假设误差上界为 1°、接收半径下界为 1000 米

问题	不一致的量	真实取值	清除比例	全部清除局数占比	平均定位清除时间/秒
问题三	误差上界	1.1°	1.000	100.0%	239.9
		1.25°	1.000	100.0%	240.7
		1.5°	1.000	100.0%	242.3
	接收半径下界	950 米	1.000	100.0%	241.0
		900 米	0.9996	99.5%	243.1
		800 米	0.9953	94.2%	247.9
问题四	误差上界	1.1°	1.000	100.0%	479.6
		1.25°	1.000	100.0%	484.0
	接收半径下界	900 米	0.9998	99.7%	481.4

两类参数的影响如图 31 所示。图 31(a) 是平均定位清除时间随干扰源个数的变化，两条曲线分别为问题三与问题四，空心圆为基准取值：两条都单调下降且问题四下降得更陡，因为它的扫描代价更大。图 31(b) 把两类不一致的量画成横条，条长为清除比例与 1 的差距：误差上界那几根短到看不出，接收半径下界那几根随下界降低迅速变长，问题三真实下界为 800 米时清除比例降到 0.9953。可以看出两类参数的地位完全不同，一类只影响时间，另一类直接决定能不能全部清除。

由表 16 与图 31 可以看出两点。其一，策略对误差上界估小非常稳健：真实误差上界放大到 1.5° 时问题三仍全部清除，时间只增加约 1%。原因有二，一是清除决策按后验概率试清除，落空后在原地补测，补测距离近，误差带来的偏差只有几米；二是可行域一旦因误差越界而变空，程序自动把扇形放宽后重建，不会在错误的可行域上反复推理，代价只是平均每局清除落空由 1.4 次增到 5.1 次。其二，策略对接收半径下界估大较为敏感：真实下界为 800 米时，问题三的清除比例降到 0.9953，漏掉的都是恰好落在六边形扫描点覆盖边缘、接收半径又偏小的全向源。问题四的方向覆盖设计留有 30 米

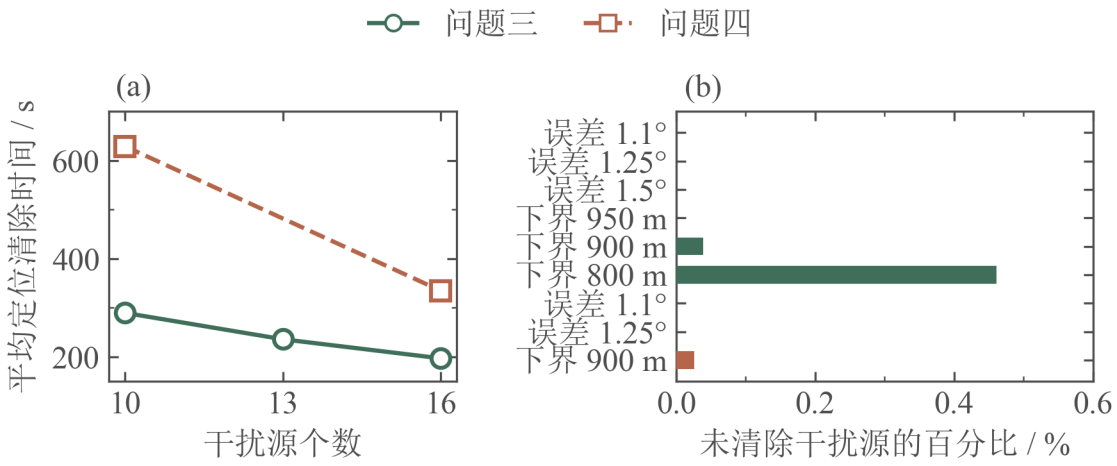


图 31　灵敏度分析：(a) 干扰源个数的影响；(b) 真实条件偏离假设时的漏检

距离余量，真实下界为 900 米时 300 局只漏掉 1 个定向源，清除比例 0.9998；退火把扫描点压到 25 个后重叠比 32 点时少，对下界偏差的容忍也随之变小。所以接收半径下界是覆盖设计的关键参数，实际使用时宁可按偏小的值设计扫描点。

8.　模型的评价、改进与推广

8.1　模型的优点

(1) 覆盖有严格保证。定理 5 证明正六边形七点盖满目标区域且六点是最少点数；定理 6 给出任意朝向都能被发现的充要条件，据此设计的扫描点经五千万个随机源核验零漏检，运行中的剪枝与合并都保持覆盖并集不变。离线评估中问题三 1000 局、问题四 500 局全部清除。

(2) 全局路线显著省时。扫描点与源排在同一条最短路上，收尾时间占比只有 1.8% 与 6.8%；同批案例上比逐步贪心缩短 33% 与 51%，且问题四不再漏掉定向源。

(3) 四问环环相扣。问题一的扇形可行域与张角判据是后三问的定位基础，问题二的期望耗时决策原样用于后两问，问题四只替换覆盖约束与可行域。

(4) 稳健且计算量小。真实误差上界放大到 1.5° 仍全部清除；单局程序运行时间最长 0.8 秒与 61 秒，远低于 20 分钟的限制。

(5) 时间构成看得清。问题三拆出全知路线下限约 212 秒，现有 238.7 秒已接近；问题四据此把扫描点由 32 个压到 25 个，同批案例缩短 8.2%。

8.2　模型的缺点

(1) 覆盖保证依赖接收半径下界。真实下界比假设小 200 米时，问题三的清除比例降到 0.9953。

(2) 问题四为保证不漏定向源付出了较大代价。扫描点由 7 个增至 25 个，时间是问题三的 2.0 倍，即使只有 1 个定向源也要完成全部方向覆盖，覆盖路线加扫描检测本身就要 360 到 400 秒。

(3) 离线复刻环境的示向度误差按每 25 米一格独立均匀抽取，模拟器的误差规律无法读取，只能间接核对。

8.3　模型的改进

针对第一条缺点，可按偏小的接收半径下界设计扫描点，例如按 900 米设计，外圈可能要增加点、时间略增，但能容忍下界偏差；两种错误的代价极不对称，漏掉一个源

比多走几百米严重得多。针对第二条缺点，40 万个样本重新退火可得 23 个点、路线 17.3 公里、一千万个源零漏检，按每点约 60 秒估计可再省 2%。

8.4 模型的推广

(1) 模型只用到两条前提：测向给出误差有界的方位，近到一定距离可以直接处置。可推广到其他依靠测向寻找目标的任务，例如无人机搜寻遇险信标与野生动物无线电项圈追踪。

(2) 凸包形式的方向覆盖判据与抽样集合覆盖设计，可推广到任何只在某个扇区内可被探测的目标，例如定向声源与定向天线基站的排查。

人工智能工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语法检查与文字润色，详细使用情况见支撑材料。

参考文献

[1] International Telecommunication Union. Handbook on Spectrum Monitoring[M]. Geneva: ITU, 2011.

[2] Stansfield R G. Statistical theory of d.f. fixing[J]. Journal of the Institution of Electrical Engineers, Part IIIA: Radiocommunication, 1947, 94(15): 762-770.

[3] Gavish M, Weiss A J. Performance analysis of bearing-only target location algorithms[J]. IEEE Transactions on Aerospace and Electronic Systems, 1992, 28(3): 817-828.

[4] Stone L D. Theory of Optimal Search[M]. New York: Academic Press, 1975.

[5] de Berg M, Cheong O, van Kreveld M, et al. Computational Geometry: Algorithms and Applications[M]. 3rd ed. Berlin: Springer, 2008.

[6] Sutherland I E, Hodgman G W. Reentrant polygon clipping[J]. Communications of the ACM, 1974, 17(1): 32-42.

[7] Andrew A M. Another efficient algorithm for convex hulls in two dimensions[J]. Information Processing Letters, 1979, 9(5): 216-219.

[8] Shamos M I. Computational Geometry[D]. New Haven: Yale University, 1978.

[9] Toussaint G T. Solving geometric problems with the rotating calipers[C]//Proceedings of IEEE MELECON'83. Athens, 1983: A10.02/1-4.

[10] Welzl E. Smallest enclosing disks (balls and ellipsoids)[M]//Maurer H. New Results and New Trends in Computer Science. Berlin: Springer, 1991: 359-370.

[11] Jung H. Über die kleinste Kugel, die eine räumliche Figur einschließt[J]. Journal für die reine und angewandte Mathematik, 1901, 123: 241-257.

[12] Croes G A. A method for solving traveling-salesman problems[J]. Operations Research, 1958, 6(6): 791-812.

[13] Or I. Traveling salesman-type combinatorial problems and their relation to the logistics of regional blood banking[D]. Evanston: Northwestern University, 1976.

附录

附录一　支撑材料文件清单

支撑材料压缩包内的目录与文件如下。画图脚本不参与求解，未放入。

- 代码/：附录二逐字列出的 23 个源程序，其中 kernel/ 下另含问题四扫描点设计的 json 数据文件
- 结果/q1/：问题一的随机构型统计与两点构型核对，summary.json、examples.json、twopoint.json
- 结果/q2/：问题二两个口径的候选点网格，grids.npz 与 summary.json 是定位效果口径，time_grids.npz 与 time_summary.json 是时间口径
- 结果/q3/：问题三各策略与灵敏度的逐局记录，plan_*.jsonl、batch_c3old.jsonl 与对应汇总
- 结果/q4/：问题四各方案与灵敏度的逐局记录，plan_*.jsonl、batch_c4old.jsonl、q4_verify.json
- AI 工具使用详情.pdf

三次正式测试结束后，从模拟器导出的加密行为日志按原文件名放入 正式测试日志/。

附录二　源程序

运行环境为 Python 3.10 以上，依赖 numpy；批量演练在多核机器上用多进程。各程序的用法写在文件开头的说明里。

kernel/rules.py　题目常数

```
# -*- coding: utf-8 -*-
""" 题目与附件给定的全部常数。改规则只改这一处。"""

ARENA_R = 1800.0          # 目标区域半径，米
R_MIN, R_MAX = 1000.0, 1500.0   # 有效接收半径的范围，米
ERR_DEG = 1.0             # 示向度误差上界，度
SPEED = 5.0               # 行进速度，米每秒
T_MEASURE = 5.0           # 一次检测耗时，秒
T_SWITCH = 1.0            # 切换频道耗时，秒
T_LOCATE = 3.0            # 清除指令未发现目标时只有精确定位，秒
T_CLEAR = 5.0             # 精确定位加清除，秒
NEAR_R = 5.0              # 信号过强无法测向的距离，米
CLEAR_R = 20.0            # 清除半径，米
HALF_COVER_DEG = 90.0     # 定向源覆盖定向方向两侧各 90 度
CH_MIN, CH_MAX = 1, 20    # 频道编号
N_MIN, N_MAX = 10, 16     # 干扰源个数范围
MAX_VIRTUAL_S = 360000.0  # 虚拟世界限时
MAX_REAL_S = 1200.0       # 程序运行时间上限
COORD_LIMIT = 2000000.0   # 接口坐标分量绝对值上限
```

kernel/geometry.py　平面几何：半平面交、凸包、旋转卡壳、最小覆盖圆

```
# -*- coding: utf-8 -*-
""" 平面几何算法。问题 1 的交会定位区域和问题 3、问题 4 的可行域共用这一份。

半平面统一写成三元组 (a, b, c)，表示 a*x + b*y + c <= 0。
点写成二元组 (x, y)，多边形写成逆时针顶点列表。
```

```
"""
import math
import random

EPS = 1e-9


def cross(o, a, b):
    """ 向量 oa 与 ob 的叉积，正值表示 b 在 oa 左侧。"""
    return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0] - o[0])


def convex_hull(points):
    """ 单调链算法求凸包，返回逆时针顶点，去掉共线点。复杂度 O(n log n)。"""
    pts = sorted(set((float(p[0]), float(p[1])) for p in points))
    if len(pts) <= 2:
        return pts
    # 共线判据用相对容差。区域只有厘米级大小时叉积本身就小于 1e-9，
    ↪ 绝对容差会把真顶点当共线删掉
    span = max(pts[-1][0] - pts[0][0], max(p[1] for p in pts) - min(p[1] for p in
    ↪ pts)) or 1.0
    tol = 1e-12 * span * span
    lower, upper = [], []
    for p in pts:
        while len(lower) >= 2 and cross(lower[-2], lower[-1], p) <= tol:
            lower.pop()
        lower.append(p)
    for p in reversed(pts):
        while len(upper) >= 2 and cross(upper[-2], upper[-1], p) <= tol:
            upper.pop()
        upper.append(p)
    return lower[:-1] + upper[:-1]


def diameter(points):
    """ 点集直径，即任意两点间距离的最大值，返回 (直径，端点 p，端点 q)。

    先求凸包，再用旋转卡壳：对凸包的每条边找离它最远的顶点，最远顶点随边的推进单调前进，
    所以扫一圈就够，这一步 O(h)，h 为凸包顶点数。
    ↪ 直径一定在某条边的端点与其对踵点之间取到。
    """
    h = convex_hull(points)
    if len(h) > 2:
        # 半平面裁剪会在边界上留下几乎重合的顶点，两点到当前边的距离相等，
        # 指针会在它们前面停住而漏掉真正的对踵点，所以先按相对容差把它们合并
        scale = max(max(abs(p[0]), abs(p[1])) for p in h) or 1.0
        tol = 1e-9 * scale
        merged = []
        for p in h:
            if not merged or math.dist(p, merged[-1]) > tol:
                merged.append(p)
        if len(merged) > 1 and math.dist(merged[0], merged[-1]) <= tol:
            merged.pop()
        h = merged
    m = len(h)
    if m == 0:
        return 0.0, None, None
    if m == 1:
```

```
        return 0.0, h[0], h[0]
    if m == 2:
        return math.dist(h[0], h[1]), h[0], h[1]
    best = [-1.0, None, None]

    def take(a, b):
        d = math.dist(a, b)
        if d > best[0]:
            best[0], best[1], best[2] = d, a, b

    j = 1
    for i in range(m):
        i2 = (i + 1) % m
        steps = 0
        # 距离不减就继续推进；推进途经的每个点都与边的两个端点配对比较，
        # 这样边与对边平行、两个顶点同样远时也不会漏掉
        while steps < m and abs(cross(h[i], h[i2], h[(j + 1) % m])) >=
        ↪  abs(cross(h[i], h[i2], h[j])):
            j = (j + 1) % m
            steps += 1
            take(h[i], h[j])
            take(h[i2], h[j])
        take(h[i], h[j])
        take(h[i2], h[j])
    return tuple(best)


def brute_diameter(points):
    """ 暴力枚举所有点对，只用来核对旋转卡壳。"""
    best = (0.0, None, None)
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            d = math.dist(points[i], points[j])
            if d > best[0]:
                best = (d, points[i], points[j])
    return best


def _circumcircle(a, b, c):
    d = 2.0 * (a[0] * (b[1] - c[1]) + b[0] * (c[1] - a[1]) + c[0] * (a[1] - b[1]))
    if abs(d) < 1e-12:
        return None
    a2, b2, c2 = a[0] ** 2 + a[1] ** 2, b[0] ** 2 + b[1] ** 2, c[0] ** 2 + c[1] ** 2
    ux = (a2 * (b[1] - c[1]) + b2 * (c[1] - a[1]) + c2 * (a[1] - b[1])) / d
    uy = (a2 * (c[0] - b[0]) + b2 * (a[0] - c[0]) + c2 * (b[0] - a[0])) / d
    return (ux, uy), math.dist((ux, uy), a)


def min_enclosing_circle(points, seed=0):
    """ 最小覆盖圆，Welzl 随机增量算法，期望 O(n)。返回 (圆心, 半径)。"""
    pts = list(points)
    if not pts:
        return None, 0.0
    random.Random(seed).shuffle(pts)
    c, r = pts[0], 0.0
    for i in range(1, len(pts)):
        p = pts[i]
        if math.dist(c, p) <= r + EPS:
```

```
            continue
        c, r = p, 0.0
        for j in range(i):
            q = pts[j]
            if math.dist(c, q) <= r + EPS:
                continue
            c, r = ((p[0] + q[0]) / 2.0, (p[1] + q[1]) / 2.0), math.dist(p, q) / 2.0
            for k in range(j):
                s = pts[k]
                if math.dist(c, s) <= r + EPS:
                    continue
                cc = _circumcircle(p, q, s)
                if cc is None:
                    # 三点共线时覆盖圆由最远的两点决定
                    far = max(((p, q), (p, s), (q, s)), key=lambda t: math.dist(*t))
                    cc = (((far[0][0] + far[1][0]) / 2.0, (far[0][1] + far[1][1]) /
                    ↪  2.0),
                          math.dist(*far) / 2.0)
                c, r = cc
    return c, r


def clip(poly, hp):
    """ 用一个半平面裁剪凸多边形，Sutherland-Hodgman 算法。"""
    a, b, c = hp
    out = []
    n = len(poly)
    for i in range(n):
        p, q = poly[i], poly[(i + 1) % n]
        fp = a * p[0] + b * p[1] + c
        fq = a * q[0] + b * q[1] + c
        if fp <= EPS:
            out.append(p)
        if (fp <= EPS) != (fq <= EPS):
            t = fp / (fp - fq)
            out.append((p[0] + t * (q[0] - p[0]), p[1] + t * (q[1] - p[1])))
    return out


def intersect_halfplanes(hps, box=1e6):
    """ 若干半平面的交。从边长 2*box 的正方形出发逐个裁剪。

    返回 (凸多边形，是否有界)。结果仍有顶点贴在正方形边上，说明真实的交在某个方向上无界。
    """
    poly = [(-box, -box), (box, -box), (box, box), (-box, box)]
    for hp in hps:
        poly = clip(poly, hp)
        if len(poly) < 3:
            return [], True
    bounded = all(abs(x) < 0.999 * box and abs(y) < 0.999 * box for x, y in poly)
    return poly, bounded


def wedge_halfplanes(s, theta_deg, eps_deg):
    """ 检测点 s 处示向度 theta 对应的扇形，顶点在 s，张角 2*eps。

    扇形等于两个半平面之交：方向在下边界射线左侧，同时在上边界射线右侧。
    张角小于 180 度时这两个条件不会把反方向那一瓣也包进来。
```

```
    """
    lo = math.radians(theta_deg - eps_deg)
    hi = math.radians(theta_deg + eps_deg)
    dlo = (math.cos(lo), math.sin(lo))
    dhi = (math.cos(hi), math.sin(hi))
    sx, sy = s
    h1 = (dlo[1], -dlo[0], -dlo[1] * sx + dlo[0] * sy)      # cross(dlo, P-s) >= 0
    h2 = (-dhi[1], dhi[0], dhi[1] * sx - dhi[0] * sy)       # cross(dhi, P-s) <= 0
    return [h1, h2]


def disk_halfplanes(center, radius, m=96):
    """ 圆盘的外切正 m 边形，写成 m 个半平面。外切是为了宁可放大也不把真值切掉。"""
    rc = radius / math.cos(math.pi / m)
    cx, cy = center
    out = []
    for k in range(m):
        ang = 2.0 * math.pi * k / m
        nx, ny = math.cos(ang), math.sin(ang)
        out.append((nx, ny, -(nx * cx + ny * cy) - rc))
    return out


def point_in_convex(p, poly, tol=1e-7):
    n = len(poly)
    if n < 3:
        return False
    for i in range(n):
        if cross(poly[i], poly[(i + 1) % n], p) < -tol:
            return False
    return True


def polygon_area(poly):
    s = 0.0
    for i in range(len(poly)):
        x1, y1 = poly[i]
        x2, y2 = poly[(i + 1) % len(poly)]
        s += x1 * y2 - x2 * y1
    return 0.5 * s


def polygon_centroid(poly):
    a = polygon_area(poly)
    if abs(a) < 1e-12:
        return (sum(p[0] for p in poly) / len(poly), sum(p[1] for p in poly) /
        ↪  len(poly))
    cx = cy = 0.0
    for i in range(len(poly)):
        x1, y1 = poly[i]
        x2, y2 = poly[(i + 1) % len(poly)]
        f = x1 * y2 - x2 * y1
        cx += (x1 + x2) * f
        cy += (y1 + y2) * f
    return cx / (6.0 * a), cy / (6.0 * a)


def bearing_deg(frm, to):
```

```
    """ 从 frm 指向 to 的方位角，x 轴正向逆时针为正，范围 [0, 360)。"""
    return math.degrees(math.atan2(to[1] - frm[1], to[0] - frm[0])) % 360.0


def ang_diff(a, b):
    """ 两个角度之差，化到 [-180, 180)。"""
    return (a - b + 180.0) % 360.0 - 180.0
```

kernel/sim.py　模拟器的离线复刻

```
# -*- coding: utf-8 -*-
""" 模拟器的离线复刻。计时与检测规则逐条照附件 1、附件 2 实现，接口与真模拟器同形。

用途有两个：一是在服务器上成百上千局地批量演练、调参数；二是让同一份策略代码
既能打离线复刻也能打真模拟器。真模拟器只监听本机回环地址，服务器上只能用这份复刻。
"""
import math
import random
import hashlib

from kernel import rules as R
from kernel.geometry import ang_diff


class Source:
    __slots__ = ('x', 'y', 'ch', 'radius', 'face', 'cleared')

    def __init__(self, x, y, ch, radius, face=None):
        self.x, self.y, self.ch, self.radius, self.face = x, y, ch, radius, face
        self.cleared = False

    def covers(self, px, py):
        """检测点是否在有效覆盖角度范围内。全向源恒为真，定向源看定向方向两侧各 90 度，
        ↪ 含边界。"""
        if self.face is None:
            return True
        b = math.degrees(math.atan2(py - self.y, px - self.x))
        return abs(ang_diff(b, self.face)) <= R.HALF_COVER_DEG + 1e-9


class Arena:
    """ 一局案例。给定 seed 就完全可复现。

    n      干扰源个数，不给就在 10 到 16 之间随机
    n_dir  定向源个数，0 为问题 3；-1 表示随机，且全向与定向各至少一个，对应问题 4
    """

    def __init__(self, seed=0, n=None, n_dir=0, err_cell=25.0, sources=None,
    ↪ r_range=None, err_deg=None):
        # r_range 与 err_deg 是这一局的真实条件，默认取题目给定值。
        ↪ 灵敏度分析里单独改它们，
        # 而策略仍按题目给定值推理，用来检验真实条件比假设更差时策略是否还稳
        rng = random.Random(seed)
        self.seed = seed
        self.err_cell = err_cell
        self.err_deg = R.ERR_DEG if err_deg is None else err_deg
        r_lo, r_hi = (R.R_MIN, R.R_MAX) if r_range is None else r_range
```

```
        if sources is None:
            n = n if n is not None else rng.randint(R.N_MIN, R.N_MAX)
            if n_dir is not None and n_dir < 0:
                n_dir = rng.randint(1, n - 1)
            chans = rng.sample(range(R.CH_MIN, R.CH_MAX + 1), n)
            dir_idx = set(rng.sample(range(n), n_dir or 0))
            sources = []
            for i in range(n):
                rad = R.ARENA_R * math.sqrt(rng.random())      # 面积上均匀
                ang = rng.uniform(0.0, 2.0 * math.pi)
                face = rng.uniform(0.0, 360.0) if i in dir_idx else None
                sources.append(Source(rad * math.cos(ang), rad * math.sin(ang),
                ↪  chans[i],
                                      rng.uniform(r_lo, r_hi), face))
        self.sources = sources
        self.by_ch = {s.ch: s for s in sources}
        self.n = len(sources)
        self.n_dir = sum(1 for s in sources if s.face is not None)
        self.pos = (0.0, 0.0)
        self.ch = R.CH_MIN
        self.t = 0.0
        self.entered = False
        self.done = False
        self.n_cleared = 0
        self.t_last_clear = 0.0
        self.log = []

    def _err(self, px, py, ch):
        """ 同一地点同一频道误差固定，换地点重新抽，落在正负 1 度内。"""
        cx = math.floor(px / self.err_cell)
        cy = math.floor(py / self.err_cell)
        h = hashlib.md5(('%d|%d|%d|%d' % (self.seed, cx, cy, ch)).encode()).digest()
        u = int.from_bytes(h[:8], 'big') / float(1 << 64)
        return (2.0 * u - 1.0) * self.err_deg

    def _move(self, x, y):
        dt = math.hypot(x - self.pos[0], y - self.pos[1]) / R.SPEED
        self.pos = (float(x), float(y))
        return dt

    def enter(self):
        if self.entered:
            return {'accepted': False, 'virtual_time_s': 0}
        self.entered = True
        return {'accepted': True, 'virtual_time_s': 0.0,
                'max_virtual_duration_s': R.MAX_VIRTUAL_S, 'max_real_duration_s':
                ↪  R.MAX_REAL_S,
                'remaining_real_duration_s': int(R.MAX_REAL_S)}

    def measure(self, x, y, ch):
        if not self.entered or self.done:
            return {'accepted': False, 'virtual_time_s': 0}
        dt = self._move(x, y)
        if ch != self.ch:
            dt += R.T_SWITCH
            self.ch = ch
        dt += R.T_MEASURE
        self.t += dt
```

```
        s = self.by_ch.get(ch)
        res = {'accepted': True}
        if s is None or s.cleared:
            res['measure_result'] = 'no_signal'
        else:
            d = math.hypot(x - s.x, y - s.y)
            if d > s.radius or not s.covers(x, y):
                res['measure_result'] = 'no_signal'
            elif d <= R.NEAR_R:
                res['measure_result'] = 'near'
            else:
                true_b = math.degrees(math.atan2(s.y - y, s.x - x))
                res['measure_result'] = 'direction'
                res['svd_deg'] = round((true_b + self._err(x, y, ch)) % 360.0, 2)
        res['virtual_time_s'] = self.t
        self.log.append(('measure', x, y, ch, res['measure_result'],
        ↪  res.get('svd_deg'), self.t))
        return res

    def clear(self, x, y, ch):
        if not self.entered or self.done:
            return {'accepted': False, 'virtual_time_s': 0}
        dt = self._move(x, y)
        s = self.by_ch.get(ch)
        hit = s is not None and not s.cleared and math.hypot(x - s.x, y - s.y) <=
        ↪  R.CLEAR_R
        dt += R.T_CLEAR if hit else R.T_LOCATE
        self.t += dt
        if hit:
            s.cleared = True
            self.n_cleared += 1
            self.t_last_clear = self.t
        res = {'accepted': True, 'virtual_time_s': self.t,
               'clear_result': 'success' if hit else 'no_target_in_range'}
        self.log.append(('clear', x, y, ch, res['clear_result'], None, self.t))
        return res

    def exit(self):
        self.done = True
        return {'accepted': True, 'virtual_time_s': self.t, 'exit_reason':
        ↪  'user_exit'}

    def stats(self):
        return {'n': self.n, 'n_dir': self.n_dir, 'cleared': self.n_cleared,
                'ratio': self.n_cleared / self.n, 'total_time': self.t,
                'avg_time': self.t / self.n_cleared if self.n_cleared else
                ↪  float('inf'),
                'tail_time': self.t - self.t_last_clear}


def self_test():
    """ 用附件 2 第 10 节的计时示例核对复刻：虚拟时刻应依次为 105、111、194、199。"""
    far = [Source(-1700.0, 0.0, 9, 1000.0)]
    a = Arena(sources=far)
    a.enter()
    got = [a.measure(300, 400, 1)['virtual_time_s'],
           a.measure(300, 400, 2)['virtual_time_s'],
           a.clear(300, 0, 3)['virtual_time_s'],
```

```
            a.measure(300, 0, 2)['virtual_time_s']]
    assert got == [105.0, 111.0, 194.0, 199.0], got
    return got


if __name__ == '__main__':
    print('附件 2 计时示例核对', self_test())
```

kernel/api.py　与模拟器的通信

```
# -*- coding: utf-8 -*-
""" 机器狗与真模拟器的通信层，照附件 2 的接口约定实现。只能在装有模拟器的本机上跑。

几条硬规定都在这里落实：每个新动作用新的 request_id，只有网络异常重试同一动作时才复用原
request_id 和原请求体；逐次等待响应，不并发；同时检查 HTTP 状态和 accepted；连不上与收到
↪ JSON
两种情况都要处理。模拟器不替机器狗记录指令序列，所以每次请求与响应都写进本地日志。
"""
import json
import time
import itertools
from urllib.request import Request, urlopen
from urllib.error import HTTPError, URLError

DEFAULT_BASE = 'http://127.0.0.1:2026'


class TestOver(Exception):
    """ 接口关闭，测试已结束。此时不要再调用 /exit 询问原因。"""


class LocalClient:
    """ 打离线复刻环境，方法签名与 HttpClient 一致。"""

    def __init__(self, arena):
        self.a = arena

    def enter(self, wait_s=0.0):
        return self.a.enter()

    def measure(self, x, y, ch):
        return self.a.measure(x, y, ch)

    def clear(self, x, y, ch):
        return self.a.clear(x, y, ch)

    def exit(self):
        return self.a.exit()

    def real_left(self):
        return None


class HttpClient:
    def __init__(self, robot_id, base=DEFAULT_BASE, log_path=None, timeout=5.0,
    ↪ retries=3):
        self.base = base.rstrip('/')
```

```
        self.robot_id = robot_id
        self.timeout = timeout
        self.retries = retries
        self.seq = itertools.count(1)
        self.run_tag = time.strftime('%H%M%S')     # 同一局里程序重启也不会撞上旧
        ↪ request_id
        self.log = open(log_path, 'a', encoding='utf-8') if log_path else None
        self.remaining_real = None
        self.t_enter = None

    def close(self):
        if self.log:
            self.log.close()
            self.log = None

    def _record(self, path, payload, resp, note=''):
        if self.log:
            self.log.write(json.dumps({'wall': time.time(), 'path': path, 'req':
            ↪ payload,
                                       'resp': resp, 'note': note},
                                       ↪ ensure_ascii=False) + '\n')
            self.log.flush()

    def _payload(self, tag):
        return {'arena_id': 'default', 'robot_id': self.robot_id,
                'request_id': '%s-%s-%d' % (self.run_tag, tag, next(self.seq))}

    def _post(self, path, payload, retries=None, quiet=False):
        body = json.dumps(payload).encode('utf-8')
        last = None
        for k in range(retries or self.retries):
            req = Request(self.base + path, data=body,
                          headers={'Content-Type': 'application/json'},
                          ↪ method='POST')
            try:
                with urlopen(req, timeout=self.timeout) as r:
                    resp = json.loads(r.read().decode('utf-8'))
                self._record(path, payload, resp)
                return resp
            except HTTPError as e:
                # 带响应体的 HTTP 错误说明请求本身有问题，重试同一请求没有意义
                try:
                    resp = json.loads(e.read().decode('utf-8'))
                except Exception:
                    resp = {'accepted': False}
                resp['http_status'] = e.code
                self._record(path, payload, resp, 'HTTPError')
                return resp
            except (URLError, OSError, ValueError) as e:
                last = e
                if not quiet:
                    self._record(path, payload, None, '%r 第%d 次' % (e, k + 1))
                time.sleep(0.2 * (k + 1))
        raise TestOver('连不上模拟器接口 %r' % last)

    def enter(self, wait_s=0.0):
        """进入目标区域。接口未开放时连接会被直接关闭、动作不执行，
        ↪ 所以可以用同一请求反复等。"""
```

```
        payload = self._payload('enter')
        deadline = time.time() + wait_s
        n_fail = 0
        while True:
            try:
                r = self._post('/enter', payload, retries=1, quiet=n_fail > 0)
                break
            except TestOver:
                n_fail += 1
                if time.time() >= deadline:
                    raise
                if n_fail == 1:
                    print('等待模拟器接口开放', flush=True)
                time.sleep(0.5)
        if r.get('accepted'):
            self.remaining_real = r.get('remaining_real_duration_s', 1200)
            self.t_enter = time.time()
        return r

    def measure(self, x, y, ch):
        p = self._payload('m')
        p['position'] = {'x': float(x), 'y': float(y)}
        p['channel'] = int(ch)
        return self._post('/measure', p)

    def clear(self, x, y, ch):
        p = self._payload('c')
        p['position'] = {'x': float(x), 'y': float(y)}
        p['channel'] = int(ch)
        return self._post('/clear', p)

    def exit(self):
        try:
            return self._post('/exit', self._payload('exit'))
        except TestOver:
            return {'accepted': False}

    def real_left(self):
        if self.t_enter is None:
            return None
        return self.remaining_real - (time.time() - self.t_enter)
```

q1_diameter.py　问题一：定位区域直径与覆盖性

```
# -*- coding: utf-8 -*-
""" 问题 1：交会定位区域直径的算法，以及以直径为直径的圆能否覆盖定位区域。

交会定位区域就是各检测点示向度扇形的交。每个扇形是两个半平面之交，所以区域是半平面交，
必为凸多边形；在凸多边形上用旋转卡壳求直径。

覆盖问题按题目原意判断：以直径两个端点的连线为直径作圆，圆心在两端点的中点，
区域的每个顶点都在圆内才算覆盖。另外附带给出一个更宽的口径，即半径为直径一半的圆
放在任何位置能否覆盖，它等价于最小覆盖圆半径不超过直径的一半。

运行：python q1_diameter.py --trials 60000 --procs 19
输出：out/q1/summary.json，out/q1/examples.json
"""
```

```
import os
import sys
import json
import math
import random
import argparse
from multiprocessing import Pool

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from kernel import rules as R
from kernel import geometry as G

OUT = os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'out', 'q1')


def locate_region(obs, eps=R.ERR_DEG):
    """ 交会定位区域。obs 为 [(x, y, 示向度), ...]，返回 (凸多边形, 是否有界)。"""
    hps = []
    for x, y, th in obs:
        hps.extend(G.wedge_halfplanes((x, y), th, eps))
    return G.intersect_halfplanes(hps)


def region_diameter(obs):
    """算法主体：半平面交得定位区域，旋转卡壳得直径。返回 (直径, 端点p, 端点q, 多边形,
    ↪  是否有界)。"""
    poly, bounded = locate_region(obs)
    if len(poly) < 2:
        return 0.0, None, None, poly, bounded
    d, p, q = G.diameter(poly)
    return d, p, q, poly, bounded


def cover_ratios(poly, d, p, q):
    """ 两个覆盖口径的比值，小于等于 1 表示能覆盖。

    ratio_fixed  圆心在直径中点：最远顶点到中点的距离除以 d/2
    ratio_any    任意放置：最小覆盖圆半径除以 d/2
    """
    c = ((p[0] + q[0]) / 2.0, (p[1] + q[1]) / 2.0)
    ratio_fixed = max(math.dist(c, v) for v in poly) / (d / 2.0)
    _, r = G.min_enclosing_circle(poly)
    return ratio_fixed, r / (d / 2.0)


def sample_case(rng, n):
    """随机构型：干扰源在目标区域内，n 个检测点在源周围 100 到 1400 米，
    ↪  示向度误差均匀取在正负 1 度内。"""
    rad = 1000.0 * math.sqrt(rng.random())
    ang = rng.uniform(0, 2 * math.pi)
    g = (rad * math.cos(ang), rad * math.sin(ang))
    obs = []
    while len(obs) < n:
        a = rng.uniform(0, 2 * math.pi)
        dist = rng.uniform(100.0, 1400.0)
        s = (g[0] + dist * math.cos(a), g[1] + dist * math.sin(a))
        if s[0] ** 2 + s[1] ** 2 > R.ARENA_R ** 2:
            continue
```

```
        th = (G.bearing_deg(s, g) + rng.uniform(-R.ERR_DEG, R.ERR_DEG)) % 360.0
        obs.append((s[0], s[1], th))
    return g, obs


def worker(job):
    seed, n, trials = job
    rng = random.Random(seed)
    acc = {'n': n, 'total': 0, 'unbounded': 0, 'fail_fixed': 0, 'fail_any': 0,
           'max_fixed': 0.0, 'max_any': 0.0, 'bad_diam': 0.0, 'outside': 0,
           'worst_fixed_case': None}
    for _ in range(trials):
        g, obs = sample_case(rng, n)
        d, p, q, poly, bounded = region_diameter(obs)
        if not bounded:
            acc['unbounded'] += 1
            continue
        if len(poly) < 3 or d < 1e-9:
            continue
        acc['total'] += 1
        acc['bad_diam'] = max(acc['bad_diam'], abs(d - G.brute_diameter(poly)[0]))
        if not G.point_in_convex(g, poly, tol=1e-6):
            acc['outside'] += 1
        rf, ra = cover_ratios(poly, d, p, q)
        if rf > 1 + 1e-9:
            acc['fail_fixed'] += 1
        if ra > 1 + 1e-9:
            acc['fail_any'] += 1
        if rf > acc['max_fixed']:
            acc['max_fixed'] = rf
            acc['worst_fixed_case'] = {'obs': obs, 'source': g, 'poly': poly, 'd':
            ↪  d,
                                       'p': p, 'q': q, 'ratio_fixed': rf,
                                       ↪  'ratio_any': ra}
        acc['max_any'] = max(acc['max_any'], ra)
    return acc


def triangle_counterexample():
    """构造反例：三个检测点在半径 900 米圆周上相隔 120 度，示向度中线同偏 delta
    ↪  而不交于一点。

    交出来的区域接近正三角形。正三角形的直径是边长，以一条边为直径的圆装不下第三个顶点，
    最小覆盖圆半径是边长除以根号 3，任意放置的口径也盖不住。扫描 delta
    ↪  取两个口径下最坏的一个。
    """
    best = None
    for k in range(-100, 101):
        delta = k * 0.01
        obs = []
        for j in range(3):
            a = math.radians(120.0 * j)
            s = (900.0 * math.cos(a), 900.0 * math.sin(a))
            obs.append((s[0], s[1], (G.bearing_deg(s, (0.0, 0.0)) + delta) % 360.0))
        d, p, q, poly, bounded = region_diameter(obs)
        if not bounded or len(poly) < 3 or d < 1e-9:
            continue
        rf, ra = cover_ratios(poly, d, p, q)
```

```
        if best is None or ra > best['ratio_any']:
            best = {'delta_deg': delta, 'obs': obs, 'poly': poly, 'd': d, 'p': p,
            ↪  'q': q,
                    'ratio_fixed': rf, 'ratio_any': ra,
                    'mec_radius': ra * d / 2.0, 'half_d': d / 2.0}
    return best


def typical_two_point():
    """ 典型两点交会：检测点 (0,0) 与 (0,-800)，源在 (1000,0)，示向度取真值。"""
    g = (1000.0, 0.0)
    obs = [(0.0, 0.0, G.bearing_deg((0.0, 0.0), g)), (0.0, -800.0,
    ↪  G.bearing_deg((0.0, -800.0), g))]
    d, p, q, poly, bounded = region_diameter(obs)
    rf, ra = cover_ratios(poly, d, p, q)
    return {'obs': obs, 'source': g, 'poly': poly, 'd': d, 'p': p, 'q': q,
            'ratio_fixed': rf, 'ratio_any': ra}


def quad_angles(poly, p, q):
    """四边形里不在直径上的两个顶点处，对直径两端所张的角。小于 90 度的顶点落在圆外。"""
    out = []
    for v in poly:
        if math.dist(v, p) < 1e-6 or math.dist(v, q) < 1e-6:
            continue
        a = (p[0] - v[0], p[1] - v[1])
        b = (q[0] - v[0], q[1] - v[1])
        cosv = (a[0] * b[0] + a[1] * b[1]) / (math.hypot(*a) * math.hypot(*b))
        out.append(math.degrees(math.acos(max(-1.0, min(1.0, cosv)))))
    return out


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--trials', type=int, default=60000,
    ↪  help='每种检测点数的随机构型数')
    ap.add_argument('--procs', type=int, default=19)
    a = ap.parse_args()
    os.makedirs(OUT, exist_ok=True)

    jobs = []
    for n in (2, 3, 4, 5):
        per = a.trials // a.procs + 1
        for k in range(a.procs):
            jobs.append((1000 * n + k, n, per))
    with Pool(a.procs) as pool:
        parts = pool.map(worker, jobs)

    summary = {'trials_per_n': a.trials, 'by_n': []}
    worst = {}
    for n in (2, 3, 4, 5):
        rows = [x for x in parts if x['n'] == n]
        tot = sum(x['total'] for x in rows)
        s = {
            'detection_points': n,
            'bounded_cases': tot,
            'unbounded_cases': sum(x['unbounded'] for x in rows),
            'fail_fixed': sum(x['fail_fixed'] for x in rows),
```

```
            'fail_fixed_pct': 100.0 * sum(x['fail_fixed'] for x in rows) / tot,
            'max_ratio_fixed': max(x['max_fixed'] for x in rows),
            'fail_any': sum(x['fail_any'] for x in rows),
            'fail_any_pct': 100.0 * sum(x['fail_any'] for x in rows) / tot,
            'max_ratio_any': max(x['max_any'] for x in rows),
            'max_diameter_error_vs_brute': max(x['bad_diam'] for x in rows),
            'source_outside_region': sum(x['outside'] for x in rows),
        }
        summary['by_n'].append(s)
        w = max(rows, key=lambda x: x['max_fixed'])['worst_fixed_case']
        worst[n] = w
        print('%d 个检测点  有界构型 %6d  圆心在中点盖不住 %5d 占 %6.2f%% 最大比值
        ↪  %.4f  '
              '任意放置盖不住 %6.2f%% 最大比值 %.4f  直径核对最大偏差 %.1e
              ↪  真值落在区域外 %d'
              % (n, tot, s['fail_fixed'], s['fail_fixed_pct'], s['max_ratio_fixed'],
                 s['fail_any_pct'], s['max_ratio_any'],
                 ↪  s['max_diameter_error_vs_brute'],
                 s['source_outside_region']))

    tri = triangle_counterexample()
    two = typical_two_point()
    w2 = worst[2]
    summary['triangle_counterexample'] = {k: tri[k] for k in
                                          ('delta_deg', 'd', 'mec_radius', 'half_d',
                                          ↪  'ratio_fixed', 'ratio_any')}
    summary['typical_two_point'] = {k: two[k] for k in ('d', 'ratio_fixed',
    ↪  'ratio_any')}
    summary['worst_two_point'] = {'d': w2['d'], 'ratio_fixed': w2['ratio_fixed'],
                                  'ratio_any': w2['ratio_any'],
                                  'angles_at_other_vertices_deg':
                                  ↪  quad_angles(w2['poly'], w2['p'], w2['q'])}
    summary['jung_bound_ratio'] = 2.0 / math.sqrt(3.0)
    print('三角形反例 偏置 %.2f 度  直径 %.3f 米  最小覆盖圆半径 %.3f 米  直径一半
    ↪  %.3f 米  '
          '圆心在中点比值 %.4f  任意放置比值 %.4f' % (tri['delta_deg'], tri['d'],
          ↪  tri['mec_radius'],
                                               tri['half_d'], tri['ratio_fixed'],
                                               ↪  tri['ratio_any']))
    print('典型两点 直径 %.3f 米  圆心在中点比值 %.6f  任意放置比值 %.6f'
          % (two['d'], two['ratio_fixed'], two['ratio_any']))
    print('两点最坏 直径 %.3f 米  圆心在中点比值 %.6f  另两顶点张角 %s'
          % (w2['d'], w2['ratio_fixed'], ['%.3f' % t for t in
          ↪  summary['worst_two_point']['angles_at_other_vertices_deg']]))

    with open(os.path.join(OUT, 'summary.json'), 'w', encoding='utf-8') as f:
        json.dump(summary, f, ensure_ascii=False, indent=1)
    with open(os.path.join(OUT, 'examples.json'), 'w', encoding='utf-8') as f:
        json.dump({'triangle': tri, 'typical_two_point': two, 'worst_by_n': worst},
                  f, ensure_ascii=False, indent=1)
    print('结果写入', os.path.normpath(OUT))


if __name__ == '__main__':
    main()
```

q1_twopoint.py　问题一：两个检测点时覆盖性判据的核对

```
# -*- coding: utf-8 -*-
""" 问题 1 两个检测点时的覆盖性核对。

两个扇形交出的四边形，四个内角由四条边界线两两的夹角决定。设两条示向度的交会角为 gamma，
误差上界为 eps，则一组对角为 gamma 附近，另一组对角为 180-gamma 附近，偏差不超过 2*eps。
直径是连接两个锐角顶点的对角线，其余两个顶点的内角就是它们对直径的张角，
所以以直径为直径的圆盖不住，只可能发生在 gamma 与 90 度相差不超过 2*eps 时。

本脚本随机生成构型，逐个核对：盖不住的构型的交会角是否全部落在 90 度正负 2*eps 以内，
并记录四边形内角与交会角的最大偏差。

  python q1_twopoint.py --trials 200000
"""
import os
import sys
import json
import math
import argparse
import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from kernel import rules as R
from kernel import geometry as G


def interior_angles(poly):
    n = len(poly)
    out = []
    for i in range(n):
        a, b, c = poly[i - 1], poly[i], poly[(i + 1) % n]
        v1 = (a[0] - b[0], a[1] - b[1])
        v2 = (c[0] - b[0], c[1] - b[1])
        cosv = (v1[0] * v2[0] + v1[1] * v2[1]) / (math.hypot(*v1) * math.hypot(*v2))
        out.append(math.degrees(math.acos(max(-1.0, min(1.0, cosv)))))
    return out


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--trials', type=int, default=200000)
    ap.add_argument('--seed', type=int, default=1)
    a = ap.parse_args()
    rng = np.random.default_rng(a.seed)
    eps = R.ERR_DEG
    n_ok = n_fail = 0
    fail_gamma = []
    max_ratio = 0.0
    max_dev = 0.0
    by_gamma = np.zeros((18, 2))           # 交会角每 5 度一档：构型数、盖不住数
    for _ in range(a.trials):
        g = rng.uniform(-R.ARENA_R, R.ARENA_R, 2)
        s = []
        for _k in range(2):
            ang = rng.uniform(0, 2 * math.pi)
            d = rng.uniform(100.0, 1400.0)
            s.append((g[0] + d * math.cos(ang), g[1] + d * math.sin(ang)))
        dets = []
```

```
        for sx, sy in s:
            th = math.degrees(math.atan2(g[1] - sy, g[0] - sx)) + rng.uniform(-eps,
            ↪  eps)
            dets.append((sx, sy, th))
        hps = []
        for sx, sy, th in dets:
            hps.extend(G.wedge_halfplanes((sx, sy), th, eps))
        poly, bounded = G.intersect_halfplanes(hps, box=1e6)
        if not bounded or len(poly) != 4:
            continue
        hull = G.convex_hull(poly)
        if len(hull) != 4:
            continue
        d, P, Q = G.diameter(hull)
        M = ((P[0] + Q[0]) / 2, (P[1] + Q[1]) / 2)
        ratio = max(math.dist(v, M) for v in hull) / (d / 2)
        gam = abs(G.ang_diff(dets[0][2], dets[1][2])) % 180.0
        gam = min(gam, 180.0 - gam)                 # 取锐角一侧，范围 0 到 90
        angs = sorted(interior_angles(hull))
        dev = max(abs(angs[0] - gam), abs(angs[1] - gam), abs(angs[2] - (180 -
        ↪  gam)), abs(angs[3] - (180 - gam)))
        max_dev = max(max_dev, dev)
        k = min(int(gam // 5), 17)
        by_gamma[k, 0] += 1
        if ratio > 1.0 + 1e-9:
            n_fail += 1
            fail_gamma.append(gam)
            max_ratio = max(max_ratio, ratio)
            by_gamma[k, 1] += 1
        else:
            n_ok += 1
    fg = np.array(fail_gamma) if fail_gamma else np.array([np.nan])
    summary = {
        'trials_quadrilateral': n_ok + n_fail, 'fail': n_fail,
        'fail_pct': 100.0 * n_fail / max(1, n_ok + n_fail),
        'fail_gamma_min_deg': float(np.nanmin(fg)), 'fail_gamma_max_deg':
        ↪  float(np.nanmax(fg)),
        'max_ratio': max_ratio, 'max_interior_angle_dev_from_gamma_deg': max_dev,
        'share_gamma_within_2eps_of_90_pct': 100.0 * float(by_gamma[17, 0]) / max(1,
        ↪  n_ok + n_fail),
        'by_gamma_5deg': [[int(5 * i), int(by_gamma[i, 0]), int(by_gamma[i, 1])] for
        ↪  i in range(18)],
    }
    out = os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'out',
    ↪  'q1new')
    os.makedirs(out, exist_ok=True)
    with open(os.path.join(out, 'twopoint.json'), 'w', encoding='utf-8') as f:
        json.dump(summary, f, ensure_ascii=False, indent=1)
    print(json.dumps(summary, ensure_ascii=False, indent=1))


if __name__ == '__main__':
    main()
```

q2_second_point.py　问题二：源距后验与定位区域直径的闭式

```
# -*- coding: utf-8 -*-
```

```
""" 问题 2：已知某全向干扰源在一个检测点的示向度，怎样选第二个检测点，以及候选区域。

坐标化简：第一个检测点 S1 为原点，测得的示向度方向为 t 轴正向，逆时针转 90 度为 h 轴，
第二个检测点记为 S2 = (t, h)。干扰源在 t 轴上距 S1 为 rho 处，rho 未知。

rho 的分布：源在目标区域内均匀分布给出面积因子 rho；源确实在 S1 被测到，要求有效接收半径
不小于 rho，有效接收半径在 1000 到 1500 米间均匀分布；源还必须在目标区域内，
所以 rho 不超过从 S1 沿示向度方向走出目标区域的距离。三者相乘就是后验密度。

定位区域直径：两条示向度各对应一条宽 2*rho*tan(eps) 与 2*L*tan(eps) 的窄条带，
以交会角 gamma 相交成近似平行四边形，其长对角线

    D = 2*eps*sqrt(rho^2 + L^2 + 2*rho*L*|cos(gamma)|) / sin(gamma)

L 为 S2 到源的距离。交会角越接近 90 度、两点离源越近，定位区域越小。

运行：python q2_second_point.py
输出：out/q2/summary.json，out/q2/grids.npz
"""
import os
import sys
import json
import math
import argparse
import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from kernel import rules as R
from kernel import geometry as G
from q1_diameter import region_diameter

OUT = os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'out', 'q2')
EPS = math.radians(R.ERR_DEG)


def exit_distance(s1, theta_deg):
    """ 从 S1 沿示向度方向走出目标区域的距离，即源到 S1 距离的几何上限。"""
    ux, uy = math.cos(math.radians(theta_deg)), math.sin(math.radians(theta_deg))
    b = s1[0] * ux + s1[1] * uy
    c = s1[0] ** 2 + s1[1] ** 2 - R.ARENA_R ** 2
    disc = b * b - c
    return -b + math.sqrt(max(disc, 0.0))


def rho_density(rho, rho_geo=R.ARENA_R):
    """rho 的后验密度，未归一化。"""
    rho = np.asarray(rho, dtype=float)
    p_recv = np.where(rho <= R.R_MIN, 1.0, np.clip((R.R_MAX - rho) / (R.R_MAX -
    ↪  R.R_MIN), 0.0, 1.0))
    ok = (rho > R.NEAR_R) & (rho <= min(R.R_MAX, rho_geo))
    return np.where(ok, rho * p_recv, 0.0)


def rho_nodes(rho_geo=R.ARENA_R, m=120):
    top = min(R.R_MAX, rho_geo)
    r = np.linspace(R.NEAR_R + 1e-6, top, m)
    w = rho_density(r, rho_geo)
    return r, w / w.sum()
```

```
def rho_quantile(q, rho_geo=R.ARENA_R):
    r = np.linspace(R.NEAR_R + 1e-6, min(R.R_MAX, rho_geo), 5000)
    cdf = np.cumsum(rho_density(r, rho_geo))
    return float(np.interp(q, cdf / cdf[-1], r))


def crossing(rho, t, h):
    """L 与交会角 gamma 的余弦、正弦。gamma 是源处指向 S1 与指向 S2 两条视线的夹角。"""
    L = np.hypot(t - rho, h)
    cosg = (-rho * (t - rho)) / np.maximum(rho * L, 1e-12)
    cosg = np.clip(cosg, -1.0, 1.0)
    sing = np.sqrt(np.maximum(1.0 - cosg ** 2, 1e-18))
    return L, cosg, sing


def closed_form_D(rho, t, h):
    L, cosg, sing = crossing(rho, t, h)
    w1 = 2.0 * rho * math.tan(EPS)
    w2 = 2.0 * L * math.tan(EPS)
    return np.sqrt(w1 ** 2 + w2 ** 2 + 2.0 * w1 * w2 * np.abs(cosg)) / sing


def exact_D(rho, t, h):
    """ 用问题 1 的算法直接求两条示向度扇形之交的直径，示向度取真值。"""
    g = (rho, 0.0)
    obs = [(0.0, 0.0, 0.0), (t, h, G.bearing_deg((t, h), g))]
    d, p, q, poly, bounded = region_diameter(obs)
    return d if bounded else float('inf')


def verify_closed_form_random(n=20000, seed=2026):
    """随机大样本核对闭式直径：rho 按后验抽样，第二点在源周围 50 到 1000 米内随机取，
    ↪ 只保留交会角不小于 5 度的构型。"""
    rng = np.random.default_rng(seed)
    rs, ws = rho_nodes()
    errs = []
    gammas = []
    while len(errs) < n:
        rho = float(rng.choice(rs, p=ws))
        ang = rng.uniform(0, 2 * math.pi)
        dist = rng.uniform(50.0, 1000.0)
        t, h = rho + dist * math.cos(ang), dist * math.sin(ang)
        L, cosg, sing = crossing(rho, t, h)
        g = math.degrees(math.acos(float(cosg)))
        if g < 5.0 or g > 175.0:
            continue
        e = exact_D(rho, t, h)
        if not math.isfinite(e) or e <= 0:
            continue
        errs.append(abs(float(closed_form_D(rho, t, h)) - e) / e)
        gammas.append(g)
    errs = np.array(errs)
    gammas = np.array(gammas)
    return {'samples': int(n), 'mean_rel_err': float(errs.mean()), 'p95_rel_err':
    ↪ float(np.quantile(errs, 0.95)),
            'max_rel_err': float(errs.max()),
```

```
            'max_rel_err_gamma_ge_30': float(errs[(gammas >= 30) & (gammas <=
            ↪  150)].max())}


def verify_closed_form():
    rows = []
    for rho in (200.0, 500.0, 800.0, 1100.0, 1400.0):
        for t, h in ((0.0, 300.0), (0.0, 1000.0), (0.5, 0.4), (0.8, 0.2), (1.0,
        ↪  0.1), (-0.3, 0.7)):
            tt, hh = (t * rho, h * rho) if abs(t) <= 1.0 and 0 < h < 1.0 else (t, h)
            e = exact_D(rho, tt, hh)
            c = float(closed_form_D(rho, tt, hh))
            rows.append({'rho': rho, 't': tt, 'h': hh, 'exact': e, 'closed': c,
            ↪  'rel_err': abs(c - e) / e})
    return rows


def perpendicular_optimum():
    """只沿 h 方向平移时, D(b) = (2eps/b) sqrt((rho^2+b^2)(4rho^2+b^2)), 最优 b =
    ↪  sqrt(2) rho, D = 6 eps rho。"""
    out = []
    for rho in (300.0, 600.0, 1000.0, 1500.0):
        b = np.linspace(1.0, 6.0 * rho, 60000)
        D = closed_form_D(rho, 0.0, b)
        k = int(np.argmin(D))
        out.append({'rho': rho, 'b_num': float(b[k]), 'b_theory': math.sqrt(2.0) *
        ↪  rho,
                    'D_num': float(D[k]), 'D_theory': 6.0 * EPS * rho,
                    'D_exact_at_theory': exact_D(rho, 0.0, math.sqrt(2.0) * rho)})
    return out


def grids(rho_geo=R.ARENA_R, step=15.0, span=2400.0, m_rho=120):
    """ 在 (t, h) 网格上算每个候选点的两项指标。

    p_hit   一次交会后定位区域直径不超过两倍清除半径、并且一定收得到信号的概率
    t_exp   把这个源定位并清除掉的期望时间：走到 S2，检测一次，从 S2 走到源；
            一次交会不够时补一次检测和一段横移，横移量按剩余距离的一半计；
            ↪  收不到信号就白跑一趟重来
    d_exp   定位区域直径的期望，直径截断在 5000 米以内再求平均
    """
    ts = np.arange(-span, span + step / 2, step)
    hs = np.arange(0.0, span + step / 2, step)          # 关于 t 轴对称，只算上半平面
    T, H = np.meshgrid(ts, hs, indexing='ij')
    rs, ws = rho_nodes(rho_geo, m_rho)
    p_hit = np.zeros_like(T)
    p_recv = np.zeros_like(T)
    t_exp = np.zeros_like(T)
    d_exp = np.zeros_like(T)
    go = np.hypot(T, H) / R.SPEED + R.T_MEASURE
    for rho, w in zip(rs, ws):
        L, cosg, sing = crossing(rho, T, H)
        D = closed_form_D(rho, T, H)
        recv = L <= R.R_MIN
        hit = recv & (D <= 2.0 * R.CLEAR_R)
        p_hit += w * hit
        p_recv += w * recv
        d_exp += w * np.minimum(D, 5000.0)
```

```
        cost = go + L / R.SPEED + R.T_CLEAR
        cost = cost + (~hit) * (0.5 * L / R.SPEED + R.T_MEASURE)
        cost = cost + (~recv) * (L / R.SPEED + R.T_MEASURE)
        t_exp += w * cost
    return ts, hs, T, H, p_hit, p_recv, t_exp, d_exp


def candidate_region(T, H, gamma_min_deg, q_lo=0.05, q_hi=0.95, rho_geo=R.ARENA_R,
↪  budget=None):
    """ 候选区域：对 rho 的可信区间 [q_lo, q_hi] 一致满足下面几条的点。

    交会角不低于 gamma_min：|h| >= |rho - t| tan(gamma_min)，并且 t < rho 一侧与 t >
    ↪  rho 一侧都算
    一定收得到信号：sqrt((t - rho)^2 + h^2) <= R_min
    离源不小于近距阈值 5 米
    可选的移动预算：sqrt(t^2 + h^2) <= budget
    """
    lo, hi = rho_quantile(q_lo, rho_geo), rho_quantile(q_hi, rho_geo)
    ok = np.ones_like(T, dtype=bool)
    tan_g = math.tan(math.radians(gamma_min_deg))
    for rho in np.linspace(lo, hi, 40):
        L = np.hypot(T - rho, H)
        ok &= np.abs(H) >= np.abs(rho - T) * tan_g
        ok &= L <= R.R_MIN
        ok &= L > R.NEAR_R
    if budget is not None:
        ok &= np.hypot(T, H) <= budget
    return ok, lo, hi


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--s1x', type=float, default=0.0, help='第一个检测点的 x 坐标，
    ↪  用于确定 rho 的几何上限')
    ap.add_argument('--s1y', type=float, default=0.0)
    ap.add_argument('--theta', type=float, default=0.0,
    ↪  help='第一个检测点测得的示向度')
    a = ap.parse_args()
    os.makedirs(OUT, exist_ok=True)
    rho_geo = exit_distance((a.s1x, a.s1y), a.theta)

    ver = verify_closed_form()
    ver_rand = verify_closed_form_random()
    perp = perpendicular_optimum()
    # 步长 7.5 米、rho 取 400 个节点，与论文配图同一套网格，表与图的最优点才对得上
    ts, hs, T, H, p_hit, p_recv, t_exp, d_exp = grids(rho_geo, step=7.5, m_rho=400)
    step = ts[1] - ts[0]

    k_hit = np.unravel_index(np.argmax(p_hit), p_hit.shape)
    k_time = np.unravel_index(np.argmin(t_exp), t_exp.shape)
    k_d = np.unravel_index(np.argmin(d_exp), d_exp.shape)

    def point_info(k):
        t, h = float(T[k]), float(H[k])
        return {'t': t, 'h': h, 'dist_from_s1': math.hypot(t, h), 'p_hit':
        ↪  float(p_hit[k]),
                'p_recv': float(p_recv[k]), 'expected_time_s': float(t_exp[k]),
                'expected_D_m': float(d_exp[k])}
```

```
cands = {}
masks = {}
arena_area = math.pi * R.ARENA_R ** 2
for gmin in (20.0, 30.0, 45.0, 60.0):
    ok, lo, hi = candidate_region(T, H, gmin, rho_geo=rho_geo)
    area = 2.0 * ok.sum() * step * step           # 网格只算了上半平面, 乘 2
    inside = ok & (T >= 0)
    cands['%g' % gmin] = {
        'area_m2': float(area), 'share_of_target_area_pct': 100.0 * area /
        ↪ arena_area,
        'rho_credible_interval': [lo, hi],
        't_range': [float(T[ok].min()), float(T[ok].max())] if ok.any() else
        ↪ None,
        'h_range': [float(H[ok].min()), float(H[ok].max())] if ok.any() else
        ↪ None,
        'best_p_hit_inside': float(p_hit[ok].max()) if ok.any() else None,
        'best_time_inside': float(t_exp[ok].min()) if ok.any() else None,
    }
    masks['cand_%g' % gmin] = ok

summary = {
    's1': [a.s1x, a.s1y], 'theta_deg': a.theta, 'rho_geometric_cap': rho_geo,
    'rho_quantiles': {'5%': rho_quantile(0.05, rho_geo), '50%':
    ↪ rho_quantile(0.5, rho_geo),
                      '95%': rho_quantile(0.95, rho_geo)},
    'closed_form_max_rel_err': max(r['rel_err'] for r in ver),
    'closed_form_check': ver,
    'closed_form_random_check': ver_rand,
    'perpendicular_optimum': perp,
    'best_for_localization': point_info(k_hit),
    'best_for_time': point_info(k_time),
    'best_for_expected_D': point_info(k_d),
    'candidate_regions': cands,
}
with open(os.path.join(OUT, 'summary.json'), 'w', encoding='utf-8') as f:
    json.dump(summary, f, ensure_ascii=False, indent=1)
np.savez_compressed(os.path.join(OUT, 'grids.npz'), ts=ts, hs=hs, p_hit=p_hit,
↪ p_recv=p_recv,
                    t_exp=t_exp, d_exp=d_exp, **masks)

print('rho 几何上限 %.0f 米  分位数 5%% %.0f  中位 %.0f  95%% %.0f'
      % (rho_geo, summary['rho_quantiles']['5%'],
      ↪ summary['rho_quantiles']['50%'],
         summary['rho_quantiles']['95%']))
print('闭式直径与精确直径最大相对偏差 %.2f%%' % (100 *
↪ summary['closed_form_max_rel_err']))
print('随机 %d 组核对  平均 %.3f%%  九成五分位 %.3f%%  最大 %.2f%%  交会角 30 到
↪ 150 度内最大 %.3f%%'
      % (ver_rand['samples'], 100 * ver_rand['mean_rel_err'], 100 *
      ↪ ver_rand['p95_rel_err'],
         100 * ver_rand['max_rel_err'], 100 *
         ↪ ver_rand['max_rel_err_gamma_ge_30']))
for r in perp:
    print('只横移 rho=%4.0f  最优横移 数值 %.1f 理论 %.1f  最小直径 数值 %.2f 理论
    ↪ %.2f 精确 %.2f'
          % (r['rho'], r['b_num'], r['b_theory'], r['D_num'], r['D_theory'],
          ↪ r['D_exact_at_theory']))
```

```
    for name, k in (('一次交会即可清除的概率最大', k_hit), ('期望完成时间最短',
    ↪  k_time), ('期望直径最小', k_d)):
        i = point_info(k)
        print('%s  (t,h)=(%.0f, %.0f)  离S1 %.0f 米  P_hit %.3f  期望时间 %.0f 秒
        ↪  期望直径 %.1f 米'
              % (name, i['t'], i['h'], i['dist_from_s1'], i['p_hit'],
              ↪  i['expected_time_s'], i['expected_D_m']))
    for g, c in cands.items():
        print('交会角下限 %s 度  候选区域面积 %.3e 平方米  占目标区域 %.2f%%  t 范围
        ↪  %s  h 范围 %s'
              % (g, c['area_m2'], c['share_of_target_area_pct'], c['t_range'],
              ↪  c['h_range']))
    print('结果写入', os.path.normpath(OUT))


if __name__ == '__main__':
    main()
```

q2_redo.py　问题二：按清除全过程期望耗时选第二检测点

```
# -*- coding: utf-8 -*-
""" 问题 2 重做：第二个检测点选在哪里，按选完之后真正把源清掉要花多少时间来评价。

设定。第一个检测点 S1 在原点，测得示向度沿 x 轴正向。真实源的位置按后验抽样：
距离 rho 的密度正比于 rho 乘以接收半径不小于 rho 的概率，方位在示向度两侧 1 度内均匀，
接收半径在 max(1000, rho) 到 1500 米之间均匀。示向度误差由离线复刻环境按检测位置给出。

对每个候选第二点 S2，逐个样本完整模拟：走到 S2 测一次，然后按固定的后续流程直到清除成功。
后续流程只有两条规则：可行域上一次清除的命中概率不低于 tau 就去命中概率最大的点清除；
否则走到该点再测一次示向度。清除落空后，落空圆内的可能位置被排除，同样按这两条规则继续。
所有候选点共用同一批样本，比较时噪声相互抵消。

输出三张图的数据：期望总耗时、测完 S2 后第一次清除就成功的概率、两条示向度交会区域的直径。
↪
期望耗时不超过最优值 (1+delta) 倍的点组成候选区域。

  python q2_redo.py --samples 200 --step 50
输出 ../out/q2new/summary.json 与 grids.npz
"""
import os
for _k in ('OMP_NUM_THREADS', 'OPENBLAS_NUM_THREADS', 'MKL_NUM_THREADS'):
    os.environ.setdefault(_k, '1')
import sys
import json
import math
import time
import argparse
import numpy as np
from multiprocessing import Pool

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from kernel import rules as R
from kernel import geometry as G
from kernel.sim import Arena, Source
from kernel.api import LocalClient
from q34_plan import PlanRobot, Params, WEDGE
```

```
OUT = os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'out', 'q2new')


def rho_samples(n, rng, rho_max=R.R_MAX):
    """ 按 f(rho) 正比于 rho * P(R >= rho) 抽样，拒绝采样。"""
    out = []
    while len(out) < n:
        r = rng.uniform(R.NEAR_R, rho_max, 4 * n)
        pr = np.where(r <= R.R_MIN, 1.0, (R.R_MAX - r) / (R.R_MAX - R.R_MIN))
        acc = rng.random(4 * n) * rho_max <= r * pr
        out.extend(r[acc].tolist())
    return np.array(out[:n])


def make_samples(n, seed):
    rng = np.random.default_rng(seed)
    rho = rho_samples(n, rng)
    phi = np.radians(rng.uniform(-R.ERR_DEG, R.ERR_DEG, n))
    radius = np.maximum(R.R_MIN, rho) + rng.random(n) * (R.R_MAX -
    ↪  np.maximum(R.R_MIN, rho))
    return [(float(rho[i] * math.cos(phi[i])), float(rho[i] * math.sin(phi[i])),
    ↪  float(radius[i]), seed * 100003 + i)
            for i in range(n)]


def region_diameter(dets):
    hps = []
    for x, y, th in dets:
        hps.extend(G.wedge_halfplanes((x, y), th, R.ERR_DEG))
    poly, bounded = G.intersect_halfplanes(hps, box=1e5)
    if not poly or not bounded:
        return float('nan')
    return G.diameter(poly)[0]


def episode(S2, sample, tau, max_steps=40):
    gx, gy, radius, sd = sample
    arena = Arena(seed=sd, sources=[Source(gx, gy, 1, radius)])
    robot = PlanRobot(LocalClient(arena), Params(seed=sd), omni=True)
    arena.enter()
    robot.measure(0.0, 0.0, 1)
    if robot.chans[1].state != 'seen':
        return None
    t0 = arena.t
    res = robot.measure(S2[0], S2[1], 1)
    c = robot.chans[1]
    got = res == 'direction'
    diam = region_diameter(c.det) if got and len(c.det) >= 2 else float('nan')
    first_hit = float('nan')
    steps = 0
    measured_at = {(round(S2[0], 1), round(S2[1], 1))}
    while c.state == 'seen' and steps < max_steps:
        steps += 1
        P, W = robot.particles(c)
        x, p = robot.best_clear(P, W)
        key = (round(float(x[0]), 1), round(float(x[1]), 1))
        near_prev = any(math.hypot(x[0] - a, x[1] - b) < 25.0 for a, b in
        ↪  measured_at)
```

```
        if p >= tau or near_prev:
            ok = robot.clear(x[0], x[1], 1)
            if first_hit != first_hit:
                first_hit = 1.0 if ok else 0.0
        else:
            robot.measure(x[0], x[1], 1)
            measured_at.add(key)
    return {'T': arena.t - t0 + 0.0, 'recv': got, 'hit1': first_hit, 'D': diam,
            'cleared': c.state == 'cleared'}


def eval_point(job):
    S2, samples, tau = job
    rows = [episode(S2, s, tau) for s in samples]
    rows = [r for r in rows if r is not None]
    T = np.array([r['T'] for r in rows])
    D = np.array([r['D'] for r in rows])
    h1 = np.array([r['hit1'] for r in rows])
    return {'S2': S2, 'ET': float(T.mean()), 'recv': float(np.mean([r['recv'] for r
    ↪  in rows])),
            'hit1': float(np.nanmean(h1)), 'ED': float(np.nanmedian(D)) if
            ↪  np.isfinite(D).any() else float('nan'),
            'cleared': float(np.mean([r['cleared'] for r in rows])), 'n': len(rows)}


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--samples', type=int, default=200)
    ap.add_argument('--step', type=float, default=50.0)
    ap.add_argument('--tau', type=float, default=0.5)
    ap.add_argument('--procs', type=int, default=19)
    ap.add_argument('--seed', type=int, default=7)
    ap.add_argument('--tmin', type=float, default=-400.0)
    ap.add_argument('--tmax', type=float, default=1700.0)
    ap.add_argument('--hmax', type=float, default=1200.0)
    ap.add_argument('--tag', default='')
    a = ap.parse_args()
    global OUT
    if a.tag:
        OUT = OUT + '_' + a.tag
    os.makedirs(OUT, exist_ok=True)
    samples = make_samples(a.samples, a.seed)
    ts = np.arange(a.tmin, a.tmax + 1e-6, a.step)
    hs = np.arange(0.0, a.hmax + 1e-6, a.step)
    pts = [(float(t), float(h)) for h in hs for t in ts]
    extra = [(705.0, 457.5), (517.5, 7.5)]          # 旧论文给出的两个点，放进来对照
    t0 = time.time()
    with Pool(a.procs) as pool:
        rows = pool.map(eval_point, [(p, samples, a.tau) for p in pts + extra],
        ↪  chunksize=2)
    grid, old = rows[:len(pts)], rows[len(pts):]
    ET = np.array([r['ET'] for r in grid]).reshape(len(hs), len(ts))
    H1 = np.array([r['hit1'] for r in grid]).reshape(len(hs), len(ts))
    ED = np.array([r['ED'] for r in grid]).reshape(len(hs), len(ts))
    RC = np.array([r['recv'] for r in grid]).reshape(len(hs), len(ts))
    np.savez(os.path.join(OUT, 'grids.npz'), t=ts, h=hs, ET=ET, H1=H1, ED=ED, RC=RC)
    k = int(np.nanargmin(ET))
    best = grid[k]
```

```
    kh = int(np.nanargmax(np.where(RC >= 0.999, H1, -1).ravel()))
    cell = a.step * a.step
    cand = {}
    for delta in (0.05, 0.10):
        m = ET <= best['ET'] * (1 + delta)
        ii, jj = np.nonzero(m)
        cand[str(delta)] = {'area_m2_half_plane': float(m.sum() * cell),
                            't_range': [float(ts[jj].min()), float(ts[jj].max())],
                            'h_range': [float(hs[ii].min()), float(hs[ii].max())]}
    summary = {'samples': a.samples, 'tau': a.tau, 'step': a.step, 'wall_s':
    ↪  time.time() - t0,
               'best_time': best, 'best_hit1': grid[kh], 'old_points': old,
               ↪  'candidate': cand,
               'straight_ahead': [r for r in grid if r['S2'][1] == 0.0 and
               ↪  r['S2'][0] in (300.0, 600.0, 900.0)]}
    with open(os.path.join(OUT, 'summary.json'), 'w', encoding='utf-8') as f:
        json.dump(summary, f, ensure_ascii=False, indent=1)
    print(json.dumps({k2: summary[k2] for k2 in ('best_time', 'best_hit1',
    ↪  'old_points', 'candidate', 'wall_s')},
                     ensure_ascii=False, indent=1))


if __name__ == '__main__':
    main()
```

q34_plan.py　问题三、四：覆盖约束、最短路与期望耗时决策的策略

```
# -*- coding: utf-8 -*-
""" 问题 3、问题 4 的新策略：覆盖扫描点、全局路线、粒子可行域与试探清除。

整体分三层。
1 覆盖层。从没测到过信号的频道，要证明整个目标区域里都没有它的源。
  问题3 全是全向源，1000 米内必被测到，取原点加半径 a 的正六边形 6 个顶点作扫描点即可。
  问题 4 有定向源，只有落在源前方半圆内的点才收得到，扫描点改用离线设计的三角点阵，
  使区域内任一位置、任一朝向的源都有扫描点落在它前方 1000 米内。
  每个这类频道用网格记录还没被排除的位置，问题 4 的网格再多一个朝向维度。
2 路线层。还需要去的扫描点和已测到信号的源一起排一条从当前位置出发的开放最短路，
  每做完一件事重排一次，只执行路线上的第一件。
3 定位清除层。测到过信号的频道用粒子表示源的可能位置：示向度扇形、接收半径上界与目标区域
  交出凸多边形，在里面均匀撒点，再用无信号与清除落空两类约束筛选加权。问题4 的粒子还带着
  是全向还是定向的概率，以及定向方向还说得通的圆弧。
  每步比较两种做法的期望耗时：直接去命中概率最大的点试清除，或先找一个点再测一次示向度。
  清除落空只花 3 秒，远比为了保证必中多走多测便宜。
"""
import os
import json
import math
import time
import numpy as np

from kernel import rules as R
from kernel import geometry as G

WEDGE = R.ERR_DEG + 0.01          # 误差上界加两位小数的舍入
CLR = R.CLEAR_R - 0.1             # 判命中用的清除半径，留一点数值余量
HERE = os.path.dirname(os.path.abspath(__file__))
```

```
class Params:
    ring_r = 1150.0       # 问题3 六边形扫描点到原点的距离，米；网格 15 米时 1150
    ↪ 仍能判定盖全，800 局上比 1200 快 0.9%
    ring_rot = 0.0        # 问题 3 六边形的起始方位，度
    grid = 15.0           # 问题 3 未发现频道的网格边长，米
    grid4 = 40.0          # 问题 4 网格边长，米
    bins = 24             # 问题 4 朝向区间个数
    q4_pts = 'kernel/q4_design_sa_c8.json'  # 退火设计 25 点，路线 17.1 公里，
    ↪ 千万个源抽样核验零漏检；旧的 32 点在 q4_design_r970.json
    p_omni = 0.5          # 问题 4 未知类型的源是全向源的先验概率
    n_part = 900          # 每个频道的粒子数
    n_eval = 500          # 决策时从粒子里重抽的个数
    n_hyp = 20            # 评估一个检测点时抽的真值假设数
    n_ccand = 150         # 清除候选点数
    fail_cost = 20.0      # 试清除落空后的兜底耗时估计，秒
    meas_pdet_min = 0.3   # 候选检测点收到信号的概率低于它就不考虑，
    ↪ 免得在定向源背面一路白测
    seen_scan_p = 0.7     # 已发现频道在扫描点顺带测向的门槛：一次命中概率低于它才测
    seen_scan_d = 1300.0  # 顺带测向只测离扫描点不超过这个距离的频道
    seen_scan_ang = 20.0  # 顺带测向要求与已有示向度的交会角不小于它，度
    seen_scan_det = 0.8   # 顺带测向要求收到信号的概率不低于它
    max_steps_src = 30
    tsp_oropt = 1         # 为 1 时路线规划在 2-opt 之后再做 Or-opt，800 局上快 0.7%
    scan_at_src = 0.0     # 大于 0 时，清除完一个源后，
    ↪ 原地扫描能排除的元素不少于最好扫描点的这个比例就地扫
    seen_at_src = 1       # 为 1 时，清除完一个源后给附近还没定准的已发现频道顺带测向，
    ↪ 两题都快约 0.9%
    q4_mode = 'samples'   # 问题4 覆盖判定：grid 为格子加朝向区间按最坏情形收紧，
    ↪ samples 为抽样判定
    q4_nsamp = 120000     # 抽样判定的样本数
    q4_samp_seed = 20260912
    q4_r_design = 990.0   # 抽样判定的距离上限，比 1000 米留 10 米余量
    q4_ang_design = 89.5  # 抽样判定的方位差上限，比 90 度留 0.5 度余量
    ghost = 0             # 为 1 时路线规划里加入尚未发现的源的虚拟节点
    ghost_n = 13.0        # 虚拟节点按总源数的这个期望值补足
    order_mode = 'tsp'    # tsp 为扫描点与源一起排最短路；scanfirst 为先排扫描点，
    ↪ 源只在顺路时插入
    ins_max = 400.0       # scanfirst 下源插入扫描路线允许多走的距离，米
    flex = 1              # 为 1 时扫描点位置在局中动态调整：挪到顺路处、相邻合并、
    ↪ 当前位置能顶替就地扫
    flex_r = 700.0        # 挪点搜索半径，米
    flex_step = 100.0     # 挪点候选网格间距，米
    flex_merge = 1        # 为 1 时相邻两个扫描点能被一点盖住且省时就合并
    flex_pass = 2         # 挪点遍数
    flex_horizon = 99     # 只对最短路顺序里最靠前的这几个扫描点做挪动与合并；
    ↪ 全做比只做 8 个快 1%，单局程序约 70 秒，真模拟器够用
    flex_r_design = 950.0  # 挪点与就地扫描判覆盖用的距离上限，比设计余量再紧一点
    flex_ang_design = 88.5 # 同上，方位差上限，度
    ring_n = 6            # 问题 3 环上扫描点个数
    defer_single = 0      # 为 1 时只测到一条示向度、
    ↪ 附近又还有扫描点能顺带补测的源先不排进路线
    defer_d = 1300.0      # 上一条里「附近」的距离，米
    max_actions = 4000
    real_budget = 1000.0
    real_reserve = 25.0
    seed = 0
```

```
    def __init__(self, **kw):
        for k, v in kw.items():
            if not hasattr(Params, k):
                raise KeyError(k)
            setattr(self, k, v)

    def as_dict(self):
        return {k: getattr(self, k) for k in dir(Params)
                if not k.startswith('_') and not callable(getattr(Params, k))}


def ang_wrap(a):
    return (a + 180.0) % 360.0 - 180.0


def arc_cut(c, w, c2, w2):
    """两段圆弧 (中心, 半宽) 求交, 逐元素。半宽都不超过 90 度时交集只有一段。
    ↪ 空集半宽记为负。"""
    d = ang_wrap(c2 - c)
    lo = np.maximum(-w, d - w2)
    hi = np.minimum(w, d + w2)
    return c + 0.5 * (lo + hi), np.where(lo <= hi, 0.5 * (hi - lo), -1.0)


def cover_mask(px, py, X, Y, h, bins):
    """ 检测点 (px, py) 无信号时可以排除的 (格子, 朝向区间)。

    格子里任何位置、区间里任何朝向的定向源, 都必须保证能在这里收到信号才可排除: 距离加上
    格子半对角线仍不超过 1000 米, 方位与朝向之差加上区间半宽和格子张角仍不超过 90 度。
    """
    half = 180.0 / bins
    centers = (np.arange(bins) + 0.5) * (360.0 / bins)
    d = np.hypot(px - X, py - Y)
    inr = d <= R.R_MIN - h
    out = np.zeros((len(X), bins), dtype=bool)
    if not inr.any():
        return out
    idx = np.flatnonzero(inr)
    ang = np.degrees(np.arctan2(py - Y[idx], px - X[idx]))
    slop = np.degrees(np.arcsin(np.clip(h / np.maximum(d[idx], h), 0.0, 1.0)))
    lim = R.HALF_COVER_DEG - half - slop
    diff = np.abs(ang_wrap(ang[:, None] - centers[None, :]))
    out[idx] = diff <= lim[:, None]
    return out


_SAMPLE_CACHE = {}


def q4_samples(n, seed, boundary_share=0.2):
    """问题4 抽样判定用的源样本 (x, y, 定向方向)。区域内均匀抽,
    ↪ 另在边界圆上加抽大致朝外的源,
    因为它们只能在区域外被测到, 是覆盖最难的一类。同样的 n 与 seed 总是得到同一批样本。
    ↪ """
    key = (n, seed, boundary_share)
    if key not in _SAMPLE_CACHE:
        rng = np.random.default_rng(seed)
        nb = int(n * boundary_share)
```

```
        na = n - nb
        r = R.ARENA_R * np.sqrt(rng.random(na))
        a = rng.uniform(0.0, 2.0 * math.pi, na)
        ab = rng.uniform(0.0, 2.0 * math.pi, nb)
        X = np.concatenate([r * np.cos(a), R.ARENA_R * np.cos(ab)])
        Y = np.concatenate([r * np.sin(a), R.ARENA_R * np.sin(ab)])
        psi = np.concatenate([rng.uniform(0.0, 360.0, na), np.degrees(ab) +
        ↪ rng.uniform(-90.0, 90.0, nb)])
        _SAMPLE_CACHE[key] = (X, Y, psi)
    return _SAMPLE_CACHE[key]


def sample_cover(px, py, X, Y, psi, r, ang):
    """检测点 (px, py) 能保证发现的样本：距离不超过 r 且方位与定向方向相差不超过 ang。
    ↪ """
    dx, dy = px - X, py - Y
    out = dx * dx + dy * dy <= r * r
    idx = np.flatnonzero(out)
    if idx.size:
        b = np.degrees(np.arctan2(dy[idx], dx[idx]))
        out[idx] = np.abs(ang_wrap(b - psi[idx])) <= ang
    return out


class Chan:
    def __init__(self, ch, shape):
        self.ch = ch
        self.state = 'unknown'      # unknown 从未测到，seen 测到过，cleared 已清除，
        ↪ empty 已证明为空
        self.det = []               # 测到示向度的 (x, y, 示向度)
        self.nosig = []             # 无信号的检测点
        self.miss = []              # 清除落空的位置
        self.alive = np.ones(shape, dtype=bool)
        self.P = None               # 粒子位置
        self.W = None               # 粒子权重
        self.H = None               # 问题3：每个粒子的 (接收半径下限，上限)；问题4：
        ↪ (全向概率，半径下限，圆弧中心，圆弧半宽，全向假设下的半径上限)
        self.dirty = True


def _path_len(D, path):
    return sum(D[path[i], path[i + 1]] for i in range(len(path) - 1))


def _two_opt(D, path):
    m = len(path)
    improved = True
    while improved:
        improved = False
        for i in range(0, m - 2):
            for j in range(i + 2, m):
                a, b, c = path[i], path[i + 1], path[j]
                if j + 1 < m:
                    d = path[j + 1]
                    delta = D[a, c] + D[b, d] - D[a, b] - D[c, d]
                else:
                    delta = D[a, c] - D[a, b]
                if delta < -1e-6:
```

```
                    path[i + 1:j + 1] = path[i + 1:j + 1][::-1]
                    improved = True
    return path


def _or_opt(D, path):
    """把长 1 到 3 的一段挪到路径别处，正反两个方向都试，有改进就接受。起点固定不动。"""
    improved = True
    while improved:
        improved = False
        m = len(path)
        for L in (1, 2, 3):
            for i in range(1, m - L + 1):
                seg = path[i:i + L]
                prev, nxt = path[i - 1], (path[i + L] if i + L < m else None)
                rem_gain = D[prev, seg[0]] + (D[seg[-1], nxt] if nxt is not None
                ↪  else 0.0) \
                    - (D[prev, nxt] if nxt is not None else 0.0)
                rest = path[:i] + path[i + L:]
                best = None
                for k in range(0, len(rest)):
                    a = rest[k]
                    b = rest[k + 1] if k + 1 < len(rest) else None
                    for s in (seg, seg[::-1]):
                        add = D[a, s[0]] + (D[s[-1], b] if b is not None else 0.0) -
                        ↪  (D[a, b] if b is not None else 0.0)
                        if add - rem_gain < -1e-6 and (best is None or add -
                        ↪  rem_gain < best[0]):
                            best = (add - rem_gain, k, s)
                if best is not None:
                    _, k, s = best
                    path = rest[:k + 1] + list(s) + rest[k + 1:]
                    improved = True
                    break
            if improved:
                break
    return path


def open_tsp(start, pts, oropt=False):
    """从 start 出发经过全部 pts 的开放路径，最近邻起步再做 2-opt，可再加 Or-opt，返回
    ↪  pts 的访问顺序。"""
    n = len(pts)
    if n <= 1:
        return list(range(n))
    P = np.array([start] + list(pts), dtype=float)
    D = np.hypot(P[:, None, 0] - P[None, :, 0], P[:, None, 1] - P[None, :, 1])
    path = [0]
    left = set(range(1, n + 1))
    while left:
        nxt = min(left, key=lambda j: D[path[-1], j])
        path.append(nxt)
        left.remove(nxt)
    path = _two_opt(D, path)
    if oropt:
        for _ in range(3):
            L0 = _path_len(D, path)
            path = _two_opt(D, _or_opt(D, path))
```

```
            if _path_len(D, path) > L0 - 1e-6:
                break
    return [k - 1 for k in path[1:]]


class PlanRobot:
    def __init__(self, client, params=None, omni=True, verbose=False):
        self.cl = client
        self.p = params or Params()
        self.omni = omni
        self.verbose = verbose
        self.rng = np.random.default_rng(self.p.seed)
        g = self.p.grid if omni else self.p.grid4
        n = int(math.ceil((R.ARENA_R + g) / g))
        ax = np.arange(-n, n + 1) * g
        X, Y = np.meshgrid(ax, ax, indexing='ij')
        X, Y = X.ravel(), Y.ravel()
        self.h = g / math.sqrt(2.0)
        keep = np.hypot(X, Y) <= R.ARENA_R + self.h
        self.gx, self.gy = X[keep], Y[keep]
        self.samples_mode = (not omni) and self.p.q4_mode == 'samples'
        if self.samples_mode:
            self.sx, self.sy, self.spsi = q4_samples(int(self.p.q4_nsamp),
            ↪  int(self.p.q4_samp_seed))
            shape = len(self.sx)
        else:
            shape = len(self.gx) if omni else (len(self.gx), self.p.bins)
        self.chans = {c: Chan(c, shape) for c in range(R.CH_MIN, R.CH_MAX + 1)}
        if omni:
            a, rot = self.p.ring_r, math.radians(self.p.ring_rot)
            nr = int(self.p.ring_n)
            self.scan_pts = [(a * math.cos(rot + 2.0 * k * math.pi / nr), a *
            ↪  math.sin(rot + 2.0 * k * math.pi / nr))
                             for k in range(nr)]
        else:
            with open(os.path.join(HERE, self.p.q4_pts), encoding='utf-8') as f:
                design = json.load(f)
            self.scan_pts = [tuple(p) for p in design['points']]
            if self.samples_mode:
                # 判定余量与样本数都以设计文件为准，保证运行时与设计口径一致
                self.p.q4_r_design = float(design.get('r_design',
                ↪  self.p.q4_r_design))
                self.p.q4_ang_design = float(design.get('ang_design',
                ↪  self.p.q4_ang_design))
                if int(design.get('nsamp', self.p.q4_nsamp)) != int(self.p.q4_nsamp)
                ↪  or int(design.get('samp_seed', self.p.q4_samp_seed)) !=
                ↪  int(self.p.q4_samp_seed):
                    self.p.q4_nsamp = int(design['nsamp'])
                    self.p.q4_samp_seed = int(design.get('samp_seed',
                    ↪  self.p.q4_samp_seed))
                    self.sx, self.sy, self.spsi = q4_samples(self.p.q4_nsamp,
                    ↪  self.p.q4_samp_seed)
                    shape = len(self.sx)
                    self.chans = {c: Chan(c, shape) for c in range(R.CH_MIN,
                    ↪  R.CH_MAX + 1)}
            if self.samples_mode and design.get('extra_samples'):
                # 设计时补漏加入的困难样本，运行时必须同样纳入判定，两边口径才一致
                ex = np.asarray(design['extra_samples'], dtype=float)
```

```
            self.sx = np.concatenate([self.sx, ex[:, 0]])
            self.sy = np.concatenate([self.sy, ex[:, 1]])
            self.spsi = np.concatenate([self.spsi, ex[:, 2]])
            shape = len(self.sx)
            self.chans = {c: Chan(c, shape) for c in range(R.CH_MIN, R.CH_MAX +
            ↪ 1)}
        self.scan_masks = [self._cover(x, y) for x, y in self.scan_pts]
        self.scan_done = set()
        self.pos = (0.0, 0.0)
        self.cur_ch = R.CH_MIN
        self.vt = 0.0
        self.n_clear = 0
        self.actions = 0
        self.t0 = time.time()
        self.stop_reason = ''
        self.unresolved = []
        self.n_measure = 0
        self.n_miss = 0

    # ---------- 基础动作 ----------
    def say(self, *a):
        if self.verbose:
            print('[%7.0f s]' % self.vt, *a, flush=True)

    def out_of_budget(self):
        if self.actions >= self.p.max_actions:
            self.stop_reason = '动作数到上限'
            return True
        if time.time() - self.t0 > self.p.real_budget:
            self.stop_reason = '现实时间到本地预算'
            return True
        left = self.cl.real_left()
        if left is not None and left < self.p.real_reserve:
            self.stop_reason = '接近程序运行时间上限'
            return True
        return False

    def _cover(self, x, y):
        """ 在 (x, y) 无信号时可以排除的网格元素。"""
        if self.omni:
            rc = R.R_MIN - self.h
            return (self.gx - x) ** 2 + (self.gy - y) ** 2 <= rc * rc
        if self.samples_mode:
            return sample_cover(x, y, self.sx, self.sy, self.spsi,
            ↪ self.p.q4_r_design, self.p.q4_ang_design)
        return cover_mask(x, y, self.gx, self.gy, self.h, self.p.bins)

    def _cover_many(self, C, idx):
        """ 候选点阵 C 里每个点对元素 idx 的覆盖，返回 (候选数, 元素数) 的布尔阵。"""
        C = np.asarray(C, dtype=float)
        if self.omni:
            rc = R.R_MIN - self.h
            X, Y = self.gx[idx], self.gy[idx]
            return (C[:, None, 0] - X[None, :]) ** 2 + (C[:, None, 1] - Y[None, :])
            ↪ ** 2 <= rc * rc
        X, Y, psi = self.sx[idx], self.sy[idx], self.spsi[idx]
        dx, dy = C[:, None, 0] - X[None, :], C[:, None, 1] - Y[None, :]
        r = self.p.flex_r_design
```

```
        out = dx * dx + dy * dy <= r * r
        b = np.degrees(np.arctan2(dy, dx))
        out &= np.abs(ang_wrap(b - psi[None, :])) <= self.p.flex_ang_design
        return out

    def _flex_ok(self):
        return bool(self.p.flex) and (self.omni or self.samples_mode)

    def _cand_grid(self, ctr, rad):
        step = self.p.flex_step
        ax = np.arange(-rad, rad + 0.1, step)
        gx, gy = np.meshgrid(ax, ax, indexing='ij')
        C = np.stack([gx.ravel() + ctr[0], gy.ravel() + ctr[1]], axis=1)
        keep = (gx.ravel() ** 2 + gy.ravel() ** 2 <= rad * rad) & (np.hypot(C[:, 0],
        ↪  C[:, 1]) <= R.ARENA_R + 1000.0)
        return C[keep]

    def measure(self, x, y, ch):
        x, y = float(x), float(y)
        r = self.cl.measure(x, y, ch)
        self.actions += 1
        if not r.get('accepted'):
            self.stop_reason = '检测指令被拒绝'
            raise RuntimeError('measure rejected %r' % r)
        self.pos = (x, y)
        self.vt = r['virtual_time_s']
        self.cur_ch = ch
        self.n_measure += 1
        c = self.chans[ch]
        res = r['measure_result']
        if res == 'direction':
            c.det.append((x, y, float(r['svd_deg'])))
            if c.state == 'unknown':
                c.state = 'seen'
            c.dirty = True
        elif res == 'near':
            if c.state == 'unknown':
                c.state = 'seen'
            self.clear(x, y, ch)
        else:
            c.nosig.append((x, y))
            if c.state == 'unknown':
                c.alive &= ~self._cover(x, y)
            else:
                c.dirty = True
        return res

    def clear(self, x, y, ch):
        x, y = float(x), float(y)
        r = self.cl.clear(x, y, ch)
        self.actions += 1
        if not r.get('accepted'):
            self.stop_reason = '清除指令被拒绝'
            raise RuntimeError('clear rejected %r' % r)
        self.pos = (x, y)
        self.vt = r['virtual_time_s']
        c = self.chans[ch]
        ok = r['clear_result'] == 'success'
```

```
        if ok:
            c.state = 'cleared'
            self.n_clear += 1
        else:
            c.miss.append((x, y))
            c.dirty = True
            self.n_miss += 1
        return ok

    # ---------- 粒子可行域 ----------
    def _sample_poly(self, poly, n):
        P = np.asarray(poly, dtype=float)
        a, b, c = P[0], P[1:-1], P[2:]
        area = 0.5 * np.abs((b[:, 0] - a[0]) * (c[:, 1] - a[1]) - (b[:, 1] - a[1]) *
        ↪  (c[:, 0] - a[0]))
        tot = area.sum()
        if tot <= 1e-9:
            return np.repeat(P.mean(0)[None, :], n, axis=0)
        k = self.rng.choice(len(area), size=n, p=area / tot)
        u, v = self.rng.random(n), self.rng.random(n)
        f = u + v > 1.0
        u[f], v[f] = 1.0 - u[f], 1.0 - v[f]
        return a + (b[k] - a) * u[:, None] + (c[k] - a) * v[:, None]

    def _poly(self, c, wedge, use_bisect):
        hps = []
        for x, y, th in c.det:
            hps.extend(G.wedge_halfplanes((x, y), th, wedge))
        if use_bisect and self.omni:
            # 在 p 测到而在 q 没测到，说明源离 p 比离 q 近，即落在两点中垂线靠 p 的一侧
            for qx, qy in c.nosig:
                for x, y, _ in c.det:
                    hps.append((2.0 * (qx - x), 2.0 * (qy - y), x * x + y * y - qx *
                    ↪  qx - qy * qy))
        for x, y, _ in c.det:
            hps.extend(G.disk_halfplanes((x, y), R.R_MAX + 0.5, m=24))
        hps.extend(G.disk_halfplanes((0.0, 0.0), R.ARENA_R + 0.5, m=48))
        poly, _ = G.intersect_halfplanes(hps, box=4.0 * R.ARENA_R)
        return poly

    def _weights(self, c, P, use_nosig=True):
        """粒子权重。问题3 只看接收半径的似然；问题4 还要对全向与定向两种类型、
        ↪  定向方向求边缘。"""
        n = len(P)
        dmax = np.zeros(n)
        for x, y, _ in c.det:
            dmax = np.maximum(dmax, np.hypot(P[:, 0] - x, P[:, 1] - y))
        lo = np.maximum(R.R_MIN, dmax)
        span = R.R_MAX - R.R_MIN
        if self.omni:
            hi = np.full(n, R.R_MAX)
            if use_nosig:
                for qx, qy in c.nosig:
                    hi = np.minimum(hi, np.hypot(P[:, 0] - qx, P[:, 1] - qy))
            W = np.clip((hi - lo) / span, 0.0, 1.0)
            H = np.stack([lo, hi], axis=1)
        else:
            pR = np.clip((R.R_MAX - lo) / span, 0.0, 1.0)
```

```
            Rs = lo + self.rng.random(n) * np.maximum(R.R_MAX - lo, 0.0)
            ok_o = np.ones(n, dtype=bool)
            cc = np.zeros(n)
            ww = np.full(n, 180.0)
            first = True
            for x, y, _ in c.det:
                b = np.degrees(np.arctan2(y - P[:, 1], x - P[:, 0]))
                if first:
                    cc, ww = b, np.full(n, 90.0)
                    first = False
                else:
                    cc, ww = arc_cut(cc, ww, b, 90.0)
            hi = np.full(n, R.R_MAX)
            if use_nosig:
                for qx, qy in c.nosig:
                    d = np.hypot(P[:, 0] - qx, P[:, 1] - qy)
                    hi = np.minimum(hi, d)      # 全向假设下,
                    ↪ 任何无信号点都把接收半径压到该距离以内
                    inr = d <= Rs
                    ok_o &= ~inr
                    b = np.degrees(np.arctan2(qy - P[:, 1], qx - P[:, 0])) + 180.0
                    c2, w2 = arc_cut(cc, ww, b, 90.0)
                    cc = np.where(inr, c2, cc)
                    ww = np.where(inr, w2, ww)
            wd = np.clip(ww, 0.0, None) * 2.0 / 360.0
            wo = ok_o.astype(float)
            mix = self.p.p_omni * wo + (1.0 - self.p.p_omni) * wd
            W = pR * mix
            pom = np.where(mix > 0, self.p.p_omni * wo / np.maximum(mix, 1e-12),
            ↪ 0.0)
            H = np.stack([pom, lo, cc, ww, hi], axis=1)
        for mx, my in c.miss:
            W[np.hypot(P[:, 0] - mx, P[:, 1] - my) <= R.CLEAR_R] = 0.0
        return W, H

    def particles(self, c):
        if not c.dirty and c.P is not None:
            return c.P, c.W
        for wedge, strict in ((WEDGE, True), (WEDGE + 0.5, False), (WEDGE + 2.0,
        ↪ False)):
            poly = self._poly(c, wedge, strict)
            if len(poly) < 3:
                continue
            Ps, Ws, Hs = [], [], []
            got = 0
            for _ in range(6):
                P = self._sample_poly(poly, self.p.n_part)
                W, H = self._weights(c, P, use_nosig=strict)
                k = W > 0
                Ps.append(P[k])
                Ws.append(W[k])
                if H is not None:
                    Hs.append(H[k])
                got += int(k.sum())
                if got >= self.p.n_part // 2:
                    break
            if got >= 20:
                c.P, c.W = np.vstack(Ps), np.concatenate(Ws)
```

```
            c.H = np.vstack(Hs) if Hs else None
            c.dirty = False
            if not strict:
                self.say('频道%2d 可行域为空，放宽到 %.1f 度' % (c.ch, wedge))
            return c.P, c.W
    # 所有约束都放宽后仍为空，只能退回最后一条示向度的扇形
    x, y, th = c.det[-1]
    t = math.radians(th)
    m = self.p.n_part
    d = self.rng.uniform(R.NEAR_R, R.R_MAX, m)
    e = np.radians(self.rng.uniform(-3.0, 3.0, m))
    c.P = np.stack([x + d * np.cos(t + e), y + d * np.sin(t + e)], axis=1)
    c.W = np.ones(m)
    if not self.omni:
        b = np.degrees(np.arctan2(y - c.P[:, 1], x - c.P[:, 0]))
        c.H = np.stack([np.full(m, 0.5), np.full(m, R.R_MIN), b, np.full(m,
        ↪  90.0), np.full(m, R.R_MAX)], axis=1)
    c.dirty = False
    return c.P, c.W

# ---------- 定位清除决策 ----------
@staticmethod
def _within(C, P):
    return (C[:, None, 0] - P[None, :, 0]) ** 2 + (C[:, None, 1] - P[None, :,
    ↪  1]) ** 2 <= CLR * CLR

def _resample_idx(self, W, n):
    return self.rng.choice(len(W), size=n, p=W / W.sum())

def best_clear(self, P, W):
    tot = W.sum()
    k = min(self.p.n_ccand, len(P))
    C = P[self._resample_idx(W, k)]
    C = np.vstack([C, (P * W[:, None]).sum(0) / tot])
    mass = self._within(C, P) @ W
    bx, bm = C[int(mass.argmax())], float(mass.max())
    for t in np.argsort(-mass)[:3]:
        x = C[t]
        for _ in range(4):
            m = self._within(x[None, :], P)[0]
            if not m.any():
                break
            x = (P[m] * W[m, None]).sum(0) / W[m].sum()
        mm = float((self._within(x[None, :], P)[0] * W).sum())
        if mm > bm:
            bx, bm = x, mm
    return bx, bm / tot

def p_detect(self, y, P, H):
    """ 每个粒子在 y 处收到信号的概率。"""
    d = np.hypot(P[:, 0] - y[0], P[:, 1] - y[1])
    if H is None:
        return np.clip((R.R_MAX - d) / (R.R_MAX - R.R_MIN), 0.0, 1.0)
    if self.omni:
        lo, hi = H[:, 0], H[:, 1]
        return np.where(d <= lo, 1.0, np.clip((hi - d) / np.maximum(hi - lo,
        ↪  1e-6), 0.0, 1.0))
    pom, lo, cc, ww, hi = H[:, 0], H[:, 1], H[:, 2], H[:, 3], H[:, 4]
```

```
        pr = np.clip((R.R_MAX - d) / np.maximum(R.R_MAX - lo, 1e-6), 0.0, 1.0)
        pr = np.where(d <= lo, 1.0, pr)
        pro = np.where(d <= lo, 1.0, np.clip((hi - d) / np.maximum(hi - lo, 1e-6),
        ↪  0.0, 1.0))
        b = np.degrees(np.arctan2(y[1] - P[:, 1], y[0] - P[:, 0]))
        dd = ang_wrap(b - cc)
        ov = np.clip(np.minimum(ww, dd + 90.0) - np.maximum(-ww, dd - 90.0), 0.0,
        ↪  None)
        pdir = np.where(ww > 1e-6, ov / np.maximum(2.0 * ww, 1e-6), (np.abs(dd) <=
        ↪  90.0).astype(float))
        return pro * pom + pr * (1.0 - pom) * np.clip(pdir, 0.0, 1.0)

    def q_measure(self, y, ch, Pe, He, C, A, base_cost):
        """在 y 处测一次示向度之后，到清除成功为止的期望耗时。Pe 是等权粒子，He
        ↪  是其类型信息。

        收不到信号时按没得到位置信息算，代价取 base_cost，即从 y
        ↪  出发按现有可行域直接试清除。
        """
        K = self.p.n_hyp
        j = self.rng.integers(0, len(Pe), size=K)
        Gj = Pe[j]
        dx, dy = Gj[:, 0] - y[0], Gj[:, 1] - y[1]
        dj = np.hypot(dx, dy)
        th = np.degrees(np.arctan2(dy, dx)) + self.rng.uniform(-R.ERR_DEG,
        ↪  R.ERR_DEG, K)
        ang = np.degrees(np.arctan2(Pe[:, 1] - y[1], Pe[:, 0] - y[0]))
        keep = (np.abs(ang_wrap(ang[None, :] - th[:, None])) <=
        ↪  WEDGE).astype(np.float32)
        tot = keep.sum(1)
        mass = keep @ A.T
        b = mass.argmax(1)
        pk = mass[np.arange(K), b] / np.maximum(tot, 1.0)
        xk = C[b]
        cost = np.hypot(xk[:, 0] - y[0], xk[:, 1] - y[1]) / R.SPEED + R.T_CLEAR +
        ↪  (1.0 - pk) * self.p.fail_cost
        cost = np.where(dj <= R.NEAR_R, R.T_CLEAR, cost)
        pdet = self.p_detect(y, Gj, He[j] if He is not None else None)
        cost = pdet * cost + (1.0 - pdet) * base_cost
        sw = R.T_SWITCH if ch != self.cur_ch else 0.0
        return R.T_MEASURE + sw + float(cost.mean())

    def meas_candidates(self, c, Pe, pos):
        ctr = Pe.mean(0)
        v = pos - ctr
        D = float(np.hypot(v[0], v[1]))
        phi0 = math.atan2(v[1], v[0]) if D > 1.0 else 0.0
        out = [pos.copy()]
        for d in (40.0, 100.0, 200.0, 400.0, 700.0, 1000.0):
            if d > D + 120.0:
                continue
            for off in (0.0, 30.0, -30.0, 60.0, -60.0, 90.0, -90.0):
                a = phi0 + math.radians(off)
                out.append(ctr + d * np.array([math.cos(a), math.sin(a)]))
        if not self.omni:
            # 定向源只在测到过信号的一侧可测，把已有检测点朝中心收拢的点也放进来
            for x, y, _ in c.det[-3:]:
                u = np.array([x, y]) - ctr
```

```
                L = float(np.hypot(*u))
                if L > 1.0:
                    for d in (60.0, 150.0, 300.0, 600.0):
                        if d < L:
                            out.append(ctr + u / L * d)
        prev = [(x, y) for x, y, _ in c.det] + list(c.nosig)
        res = []
        for y in out:
            if any((y[0] - px) ** 2 + (y[1] - py) ** 2 < 25.0 ** 2 for px, py in
            ↪  prev):
                continue
            res.append(y)
        return res

    def decide(self, c):
        P, W = self.particles(c)
        pos = np.array(self.pos)
        x, p = self.best_clear(P, W)
        if p >= 0.995:
            return 'clear', x, np.hypot(*(x - pos)) / R.SPEED + R.T_CLEAR
        idx = self._resample_idx(W, self.p.n_eval)
        Pe = P[idx]
        He = c.H[idx] if c.H is not None else None
        C = np.vstack([P[self._resample_idx(W, self.p.n_ccand)], x[None, :]])
        A = self._within(C, Pe).astype(np.float32)
        fc = self.p.fail_cost
        # 试清除: 落空后粒子里去掉清除圆内的, 再按在原地测一次估计
        mk = ~self._within(x[None, :], Pe)[0]
        if mk.sum() >= 5:
            qm = self.q_measure(x, c.ch, Pe[mk], He[mk] if He is not None else None,
            ↪  C, A[:, mk],
                                R.T_CLEAR + fc)
        else:
            qm = R.T_CLEAR
        best = ('clear', x, np.hypot(*(x - pos)) / R.SPEED + p * R.T_CLEAR + (1 - p)
        ↪  * (R.T_LOCATE + qm))
        for y in self.meas_candidates(c, Pe, pos):
            if float(self.p_detect(y, Pe, He).mean()) < self.p.meas_pdet_min:
                continue
            base = np.hypot(*(x - y)) / R.SPEED + R.T_CLEAR + (1 - p) * fc
            cm = np.hypot(*(y - pos)) / R.SPEED + self.q_measure(y, c.ch, Pe, He, C,
            ↪  A, base)
            if cm < best[2]:
                best = ('measure', y, cm)
        return best

    def work_source(self, c):
        for _ in range(self.p.max_steps_src):
            if c.state != 'seen' or self.out_of_budget():
                return
            kind, pt, cost = self.decide(c)
            if kind == 'clear':
                ok = self.clear(pt[0], pt[1], c.ch)
                self.say('试清除 频道%2d 于 (%.0f, %.0f) %s' % (c.ch, pt[0], pt[1],
                ↪  '成功' if ok else '落空'))
            else:
                res = self.measure(pt[0], pt[1], c.ch)
```

```
            self.say('测向 频道%2d 于 (%.0f, %.0f) 得 %s' % (c.ch, pt[0], pt[1],
            ↪  res))
    # 步数用完：按粒子逐点清除，每次落空都会去掉一块，直到清掉
    n = 0
    while c.state == 'seen' and n < 300 and not self.out_of_budget():
        P, W = self.particles(c)
        x, _ = self.best_clear(P, W)
        self.clear(x[0], x[1], c.ch)
        n += 1
    if c.state == 'seen':
        self.unresolved.append(c.ch)
        c.state = 'empty'

# ---------- 扫描 ----------
def scan_channels(self, pt, with_unknown=True):
    """在扫描点上要测的频道：能被这里排除位置的未发现频道，
    ↪  加上值得顺带测向的已发现频道。"""
    x, y = pt
    chans = []
    if with_unknown:
        mask = self._cover(x, y)
        chans = [c.ch for c in self.chans.values() if c.state == 'unknown' and
        ↪  (c.alive & mask).any()]
    for c in self.chans.values():
        if c.state != 'seen':
            continue
        P, W = self.particles(c)
        tot = W.sum()
        ctr = (P * W[:, None]).sum(0) / tot
        d = np.hypot(P[:, 0] - x, P[:, 1] - y)
        if (W * (d <= self.p.seen_scan_d)).sum() < 0.95 * tot:
            continue
        if any(math.hypot(x - px, y - py) < 25.0 for px, py, _ in c.det):
            continue
        if not self.omni and float((self.p_detect(np.array([x, y]), P, c.H) *
        ↪  W).sum() / tot) < self.p.seen_scan_det:
            continue
        _, pc = self.best_clear(P, W)
        if pc >= self.p.seen_scan_p:
            continue
        b = math.degrees(math.atan2(ctr[1] - y, ctr[0] - x))
        ok = True
        for px, py, _ in c.det:
            b0 = math.degrees(math.atan2(ctr[1] - py, ctr[0] - px))
            dd = abs(G.ang_diff(b, b0)) % 180.0
            if min(dd, 180.0 - dd) < self.p.seen_scan_ang:
                ok = False
                break
        if ok:
            chans.append(c.ch)
    chans.sort(key=lambda ch: (ch != self.cur_ch, ch))
    return chans

def do_scan(self, pt):
    chans = self.scan_channels(pt)
    self.say('扫描 于 (%.0f, %.0f) 测 %d 个频道' % (pt[0], pt[1], len(chans)))
    for ch in chans:
        if self.out_of_budget():
```

```
            return
        if self.chans[ch].state in ('cleared', 'empty'):
            continue
        self.measure(pt[0], pt[1], ch)

def extra_scan_point(self, unknown):
    """ 设计好的扫描点都用完后仍有未排除的元素，就挑单位时间排除最多元素的点。"""
    alive = np.zeros(self.chans[R.CH_MIN].alive.shape, dtype=np.int32)
    for c in unknown:
        alive += c.alive
    if not alive.any():
        return None
    best, bs = None, 0.0
    lim = R.ARENA_R + (200.0 if self.omni else 1000.0)
    step = 150.0 if self.omni else 250.0
    ax = np.arange(-lim, lim + 0.1, step)
    for px in ax:
        for py in ax:
            if px * px + py * py > lim * lim:
                continue
            gain = alive[self._cover(px, py)].sum()
            if gain <= 0:
                continue
            cost = math.hypot(px - self.pos[0], py - self.pos[1]) / R.SPEED +
            ↪  6.0 * len(unknown)
            if gain / cost > bs:
                best, bs = (float(px), float(py)), gain / cost
    return best

# ---------- 主循环 ----------
def tasks(self):
    seen = [c for c in self.chans.values() if c.state == 'seen']
    unknown = [c for c in self.chans.values() if c.state == 'unknown']
    if unknown and self.n_clear + len(seen) >= R.N_MAX:
        for c in unknown:
            c.state = 'empty'
        self.say('已清除加已发现到 16 个，其余频道判空')
        unknown = []
    for c in unknown:
        if not c.alive.any():
            c.state = 'empty'
    unknown = [c for c in unknown if c.state == 'unknown']
    nodes = []
    flex = self._flex_ok() and bool(unknown)
    if flex:
        nodes = self.flex_scan_nodes(unknown)
        if nodes is None:
            return self.tasks()
    else:
        for k, sp in enumerate(self.scan_pts):
            if k in self.scan_done:
                continue
            if any((c.alive & self.scan_masks[k]).any() for c in unknown):
                nodes.append(('scan', sp, [k], None))
            else:
                self.scan_done.add(k)
    if unknown and not any(n[0] == 'scan' for n in nodes):
        ep = self.extra_scan_point(unknown)
```

```
            if ep is not None:
                nodes.append(('scan', ep, [], None))
            else:
                for c in unknown:
                    c.state = 'empty'
        scan_xy = [n[1] for n in nodes if n[0] == 'scan']
        for c in seen:
            P, W = self.particles(c)
            ctr = (P * W[:, None]).sum(0) / W.sum()
            if self.p.defer_single and len(c.det) < 2 and scan_xy:
                # 只有一条示向度时位置沿射线拉得很长，先等附近扫描点顺带补一条再排进路线
                if any(math.hypot(sx - ctr[0], sy - ctr[1]) <= self.p.defer_d for
                ↪  sx, sy in scan_xy):
                    _, pc = self.best_clear(P, W)
                    if pc < 0.5:
                        continue
            nodes.append(('src', (float(ctr[0]), float(ctr[1])), c, None))
        if self.p.ghost and unknown:
            nodes.extend(self.ghost_nodes(nodes, unknown, len(seen)))
        if flex and any(n[0] == 'scan' and n[3] is not None for n in nodes):
            nodes = self.relocate(nodes, len(unknown))
        return nodes

    def flex_scan_nodes(self, unknown):
        """ 动态扫描：只有本点独占的未排除元素才必须由它来盖。
        先剪掉没有独占元素的冗余点，再看当前位置能否盖住某点的独占元素，
        ↪  能就在这里扫并放弃那点。
        返回扫描节点列表，节点带独占元素在 need 索引里的位置；就地扫过返回 None
        ↪  让上层重算。"""
        need = np.zeros(unknown[0].alive.shape, dtype=bool)
        for c in unknown:
            need |= c.alive
        idx = np.flatnonzero(need)
        self._need_idx = idx
        rem = [k for k in range(len(self.scan_pts)) if k not in self.scan_done]
        cols = {k: self.scan_masks[k][idx] for k in rem}
        cnt = np.zeros(len(idx), dtype=np.int32)
        for k in rem:
            cnt += cols[k]
        # 剪枝：离当前位置远的先试
        for k in sorted(rem, key=lambda k: -math.hypot(self.scan_pts[k][0] -
        ↪  self.pos[0], self.scan_pts[k][1] - self.pos[1])):
            if not (cols[k] & (cnt == 1)).any():
                cnt -= cols[k]
                rem.remove(k)
                self.scan_done.add(k)
        if not rem:
            return []
        here = self._cover_many([self.pos], idx)[0]
        hit = [k for k in rem if not (cols[k] & (cnt == 1) & ~here).any()]
        if hit:
            self.say('当前位置可顶替扫描点 %s' % hit)
            self.do_scan(self.pos)
            for k in hit:
                self.scan_done.add(k)
            return None
        self._flex_cols, self._flex_cnt = cols, cnt
```

```
        return [('scan', self.scan_pts[k], [k], np.flatnonzero(cols[k] & (cnt ==
        ↪  1))) for k in rem]

    def relocate(self, nodes, n_unknown):
        """按当前最短路顺序，把每个扫描点挪到仍能盖住它独占元素、且前后两段路最短的位置；
        相邻两个扫描点若能被一点盖住两点合起来独占的元素且合并后总代价下降，
        ↪  就合并成一个。"""
        scan_m = R.T_MEASURE * n_unknown * R.SPEED     #
        ↪  少走一个扫描点省下的检测耗时折算成米
        idx = self._need_idx
        cnt = self._flex_cnt.copy()
        ncol = {}
        for n in nodes:
            if n[0] == 'scan' and n[3] is not None:
                ncol[id(n)] = self._flex_cols[n[2][0]]
        nodes = list(nodes)
        for _ in range(int(self.p.flex_pass)):
            pts = [n[1] for n in nodes]
            order = open_tsp(self.pos, pts, oropt=bool(self.p.tsp_oropt))
            seq = [nodes[i] for i in order]
            chain = [np.array(self.pos, dtype=float)] + [np.array(n[1], dtype=float)
            ↪  for n in seq]
            changed = False
            j = 1
            done_scan = 0
            while j < len(chain):
                n = seq[j - 1]
                if n[0] != 'scan' or n[3] is None:
                    j += 1
                    continue
                done_scan += 1
                if done_scan > int(self.p.flex_horizon):
                    break
                prev = chain[j - 1]
                nxt = chain[j + 1] if j + 1 < len(chain) else None
                p = chain[j]
                col = ncol[id(n)]

                def seg(c, a=prev, b=nxt):
                    d = np.hypot(c[:, 0] - a[0], c[:, 1] - a[1])
                    if b is not None:
                        d = d + np.hypot(c[:, 0] - b[0], c[:, 1] - b[1])
                    return d
                merged = False
                if self.p.flex_merge and nxt is not None and seq[j][0] == 'scan' and
                ↪  seq[j][3] is not None:
                    n2 = seq[j]
                    col2 = ncol[id(n2)]
                    nxt2 = chain[j + 2] if j + 2 < len(chain) else None
                    res_u = np.flatnonzero((col | col2) & (cnt - col - col2 == 0))
                    mid = 0.5 * (p + nxt)
                    C = np.vstack([self._cand_grid(mid, self.p.flex_r), p[None, :],
                    ↪  nxt[None, :]])
                    ok = self._cover_many(C, idx[res_u]).all(1) if res_u.size else
                    ↪  np.ones(len(C), dtype=bool)
                    if ok.any():
                        Ck = C[ok]
                        d = seg(Ck, prev, nxt2)
```

```
                    old = float(np.hypot(*(p - prev)) + np.hypot(*(nxt - p))
                                + (np.hypot(*(nxt2 - nxt)) if nxt2 is not None
                                ↪  else 0.0))
                    i = int(d.argmin())
                    if float(d[i]) - old < scan_m:
                        c = Ck[i]
                        newcol = self._cover(c[0], c[1])[idx]
                        cnt = cnt - col - col2 + newcol
                        m = ('scan', (float(c[0]), float(c[1])), list(n[2]) +
                        ↪  list(n2[2]), res_u)
                        ncol[id(m)] = newcol
                        seq[j - 1] = m
                        del seq[j]
                        chain[j] = c
                        del chain[j + 1]
                        self.say('合并扫描点 %s 与 %s 到 (%.0f, %.0f)' % (n[2],
                        ↪  n2[2], c[0], c[1]))
                        changed = merged = True
                        continue
            if not merged:
                res = np.flatnonzero(col & (cnt == 1))
                base = float(seg(p[None, :])[0])
                C = np.vstack([self._cand_grid(p, self.p.flex_r), prev[None, :]]
                ↪  + ([nxt[None, :]] if nxt is not None else []))
                ok = self._cover_many(C, idx[res]).all(1) if res.size else
                ↪  np.ones(len(C), dtype=bool)
                if ok.any():
                    Ck = C[ok]
                    d = seg(Ck)
                    i = int(d.argmin())
                    if float(d[i]) < base - 1.0:
                        c = Ck[i]
                        newcol = self._cover(c[0], c[1])[idx]
                        cnt = cnt - col + newcol
                        m = ('scan', (float(c[0]), float(c[1])), n[2], res)
                        ncol[id(m)] = newcol
                        seq[j - 1] = m
                        chain[j] = c
                        changed = True
            j += 1
        nodes = seq
        if not changed:
            break
    return nodes

def ghost_nodes(self, nodes, unknown, n_seen):
    """尚未发现的源的虚拟节点：期望还剩的个数按各个未扫描点附近未排除的面积分摊，
    ↪  位置在其中随机取。
    虚拟节点只用来排路线，不会被执行，
    ↪  作用是让路线提前给扫描后才会发现的源留出顺路的走法。"""
    scan_nodes = [n for n in nodes if n[0] == 'scan' and n[2]]
    expect = self.p.ghost_n - self.n_clear - n_seen
    if not scan_nodes or expect <= 0:
        return []
    PX, PY = (self.sx, self.sy) if self.samples_mode else (self.gx, self.gy)
    alive = np.zeros(len(PX), dtype=bool)
    for c in unknown:
        alive |= c.alive if c.alive.ndim == 1 else c.alive.any(1)
```

```
        masks, wts = [], []
        for n in scan_nodes:
            m = self.scan_masks[n[2][0]]
            m = (m if m.ndim == 1 else m.any(1)) & alive
            masks.append(m)
            wts.append(int(m.sum()))
        tot = sum(wts)
        out = []
        if tot <= 0:
            return out
        for m, w in zip(masks, wts):
            k = int(round(expect * w / tot))
            idx = np.flatnonzero(m)
            if k <= 0 or idx.size == 0:
                continue
            for i in self.rng.choice(idx, size=min(k, idx.size), replace=False):
                out.append(('ghost', (float(PX[i]), float(PY[i])), None, None))
        return out

    def next_node(self, nodes):
        """ 挑下一件要做的事。tsp 模式取整条最短路的第一件；scanfirst 模式先排扫描点，
        再把插进去多走不超过 ins_max 的源逐个插到最省的位置，
        ↪  远的源留到扫描点走完再统一排。"""
        pts = [n[1] for n in nodes]
        oropt = bool(self.p.tsp_oropt)
        real = [i for i, n in enumerate(nodes) if n[0] != 'ghost']
        scans = [i for i in real if nodes[i][0] == 'scan']
        if self.p.order_mode != 'scanfirst' or not scans:
            order = open_tsp(self.pos, pts, oropt=oropt)
            order = [k for k in order if nodes[k][0] != 'ghost']
            return nodes[order[0]]
        srcs = [i for i in real if nodes[i][0] == 'src']
        seq = [scans[k] for k in open_tsp(self.pos, [pts[i] for i in scans],
        ↪  oropt=oropt)]
        remaining = list(srcs)
        while remaining:
            chain = [self.pos] + [pts[j] for j in seq]
            best = None
            for i in remaining:
                q = pts[i]
                for k in range(len(chain)):
                    a = chain[k]
                    if k + 1 < len(chain):
                        b = chain[k + 1]
                        add = math.dist(a, q) + math.dist(q, b) - math.dist(a, b)
                    else:
                        add = math.dist(a, q)
                    if best is None or add < best[0]:
                        best = (add, i, k)
            if best[0] > self.p.ins_max:
                break
            _, i, k = best
            seq.insert(k, i)
            remaining.remove(i)
        return nodes[seq[0]]

    def maybe_scan_here(self):
```

```
    """清除完一个源后，若原地扫描未发现频道能排除的元素不少于最好扫描点的一定比例，
    ↪ 就地扫。"""
    unknown = [c for c in self.chans.values() if c.state == 'unknown']
    if not unknown:
        return
    here = self._cover(*self.pos)
    g_here = sum(int((c.alive & here).sum()) for c in unknown)
    if g_here == 0:
        return
    g_best = 0
    for k in range(len(self.scan_pts)):
        if k not in self.scan_done:
            g_best = max(g_best, sum(int((c.alive & self.scan_masks[k]).sum())
            ↪ for c in unknown))
    if g_best == 0 or g_here < self.p.scan_at_src * g_best:
        return
    chans = sorted((c.ch for c in unknown if (c.alive & here).any()), key=lambda
    ↪ ch: (ch != self.cur_ch, ch))
    self.say('就地扫描 于 (%.0f, %.0f) 测 %d 个频道' % (self.pos[0], self.pos[1],
    ↪ len(chans)))
    for ch in chans:
        if self.out_of_budget():
            return
        self.measure(self.pos[0], self.pos[1], ch)

def play(self):
    self.do_scan((0.0, 0.0))
    mem_retry = 0
    while not self.out_of_budget():
        try:
            self._play_step()
            if self.stop_reason == '所有频道已清除或确认为空':
                break
            mem_retry = 0
        except MemoryError:
            # 本机内存紧时 numpy 偶发分配失败，回收后重试这一步，
            ↪ 已完成的动作都记在状态里不会重复
            import gc
            gc.collect()
            mem_retry += 1
            if mem_retry > 20:
                raise
            time.sleep(1.0)

def _play_step(self):
    if True:
        nodes = self.tasks()
        if not nodes:
            self.stop_reason = '所有频道已清除或确认为空'
            return
        kind, pt, obj = self.next_node(nodes)[:3]
        if kind == 'scan':
            for k in obj:
                self.scan_done.add(k)
            self.do_scan(pt)
        else:
            self.work_source(obj)
            if self.p.scan_at_src > 0:
```

```
                    self.maybe_scan_here()
                if self.p.seen_at_src:
                    for ch in self.scan_channels(self.pos, with_unknown=False):
                        if self.out_of_budget():
                            break
                        if self.chans[ch].state == 'seen':
                            self.measure(self.pos[0], self.pos[1], ch)

    def run(self):
        r = self.cl.enter()
        if not r.get('accepted'):
            return None
        try:
            self.play()
        finally:
            self.cl.exit()
        return {'cleared': self.n_clear, 'virtual_time': self.vt, 'actions':
        ↪  self.actions,
                'stop_reason': self.stop_reason}
```

q4_cover.py　问题四：三角点阵覆盖设计

```
# -*- coding: utf-8 -*-
""" 问题 4 的覆盖扫描点设计。

定向源在 G 处、定向方向 psi、有效接收半径 R 时，只有落在以 G 为圆心、半径 R、朝 psi
↪  的半圆内的
检测点才收得到。要保证任何位置、任何朝向的源都能被发现，扫描点集合 Q 必须满足：
↪  对区域内每个 G
和每个 psi，Q 里至少有一点离 G 不超过 1000 米、且方位与 psi 相差不超过 90 度。

做法是离散成元素 (格子，朝向区间)，候选点取三角点阵，用按路程加权的贪心集合覆盖选点，
再删掉冗余点，最后排开放最短路并报告总路程。格子与朝向区间都有宽度，
↪  判覆盖时按最坏情形收紧：
距离减去格子半对角线，角度减去朝向区间半宽和格子张角。

  python q4_cover.py --cell 100 --bins 12 --cand 200 --ext 1000
输出 kernel/q4_scan_pts.json
"""
import os
import sys
import json
import math
import argparse
import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from kernel import rules as R
from q34_plan import open_tsp

HERE = os.path.dirname(os.path.abspath(__file__))


def elements(cell, bins):
    n = int(math.ceil(R.ARENA_R / cell)) + 1
    ax = np.arange(-n, n + 1) * cell
    X, Y = np.meshgrid(ax, ax, indexing='ij')
```

```
    X, Y = X.ravel(), Y.ravel()
    h = cell / math.sqrt(2.0)
    keep = np.hypot(X, Y) <= R.ARENA_R + h
    return X[keep], Y[keep], h


def tri_lattice(spacing, rmax, rot=0.0):
    pts = []
    n = int(rmax / spacing) + 2
    for j in range(-2 * n, 2 * n + 1):
        for i in range(-2 * n, 2 * n + 1):
            x = spacing * (i + 0.5 * (j % 2))
            y = spacing * math.sqrt(3.0) / 2.0 * j
            if x * x + y * y <= rmax * rmax:
                c, s = math.cos(rot), math.sin(rot)
                pts.append((c * x - s * y, s * x + c * y))
    return np.array(pts)


def cover_matrix(cand, X, Y, h, bins):
    """cand 个候选点各自覆盖哪些 (格子, 朝向区间)。返回形状 (候选数, 格子数, 区间数)
    ↪ 的布尔阵。"""
    half = 180.0 / bins
    centers = (np.arange(bins) + 0.5) * (360.0 / bins)
    out = np.zeros((len(cand), len(X), bins), dtype=bool)
    for k, (px, py) in enumerate(cand):
        d = np.hypot(px - X, py - Y)
        inr = d <= R.R_MIN - h
        if not inr.any():
            continue
        ang = np.degrees(np.arctan2(py - Y, px - X))
        slop = np.degrees(np.arcsin(np.clip(h / np.maximum(d, h), 0.0, 1.0)))
        lim = R.HALF_COVER_DEG - half - slop
        diff = np.abs((ang[:, None] - centers[None, :] + 180.0) % 360.0 - 180.0)
        out[k] = inr[:, None] & (diff <= lim[:, None])
    return out


def greedy_cover(cand, M, start=(0.0, 0.0), dist_w=0.0):
    need = np.ones(M.shape[1:], dtype=bool)
    chosen = []
    pos = np.array(start)
    flat = M.reshape(len(cand), -1)
    needf = need.ravel()
    while needf.any():
        gain = flat[:, needf].sum(1).astype(float)
        if gain.max() <= 0:
            break
        if dist_w > 0:
            d = np.hypot(cand[:, 0] - pos[0], cand[:, 1] - pos[1])
            score = gain / (1.0 + dist_w * d)
        else:
            score = gain
        k = int(score.argmax())
        chosen.append(k)
        needf &= ~flat[k]
        pos = cand[k]
    return chosen, int(needf.sum())
```

```
def prune(chosen, M):
    """ 逆序尝试删点，删了仍全覆盖就删。"""
    flat = M.reshape(M.shape[0], -1)
    keep = list(chosen)
    for k in sorted(chosen, key=lambda k: flat[k].sum()):
        rest = [j for j in keep if j != k]
        cov = np.zeros(flat.shape[1], dtype=bool)
        for j in rest:
            cov |= flat[j]
        if cov.all():
            keep = rest
    return keep


def tour_len(pts, start=(0.0, 0.0)):
    order = open_tsp(start, [tuple(p) for p in pts])
    cur = np.array(start)
    L = 0.0
    for k in order:
        L += float(np.hypot(*(pts[k] - cur)))
        cur = pts[k]
    return L, order


def lattice_points(s, rot_deg, ox, oy, rmax):
    """ 间距 s 的三角点阵，先平移 (ox, oy) 再旋转，保留离原点不超过 rmax 的点。"""
    rot = math.radians(rot_deg)
    c, si = math.cos(rot), math.sin(rot)
    n = int(rmax / s) + 3
    pts = []
    for j in range(-n, n + 1):
        for i in range(-n, n + 1):
            x = s * (i + 0.5 * (j % 2)) + ox
            y = s * math.sqrt(3.0) / 2.0 * j + oy
            X, Y = c * x - si * y, si * x + c * y
            if X * X + Y * Y <= rmax * rmax:
                pts.append((X, Y))
    return np.array(pts)


def lattice_search(a):
    """在点阵间距 s 下搜旋转与平移，原点强制入选，删冗余后取路线最短且全覆盖的一组。"""
    X, Y, h = elements(a.cell, a.bins)
    best = None
    s = a.lattice
    for rot in np.arange(0.0, 60.0, a.rot_step):
        for fi in range(a.off_n):
            for fj in range(a.off_n):
                ox, oy = s * fi / a.off_n, s * math.sqrt(3.0) / 2.0 * fj / a.off_n
                cand = np.vstack([[0.0, 0.0], lattice_points(s, rot, ox, oy,
                ↪  R.ARENA_R + s)])
                M = cover_matrix(cand, X, Y, h, a.bins)
                flat = M.reshape(len(cand), -1)
                if not flat.any(0).all():
                    continue
                # 原点必去，不参与删点；其余按独有贡献从少到多试删
```

```
                keep = list(range(len(cand)))
                cnt = flat.sum(0)
                for k in sorted(range(1, len(cand)), key=lambda k: flat[k].sum()):
                    if (cnt - flat[k] > 0).all():
                        cnt = cnt - flat[k]
                        keep.remove(k)
                pts = cand[keep]
                L, order = tour_len(pts[1:])
                if best is None or (len(keep), L) < (best[0], best[1]):
                    best = (len(keep), L, rot, ox, oy, pts[1:][order])
    return best, len(X)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--cell', type=float, default=100.0)
    ap.add_argument('--bins', type=int, default=12)
    ap.add_argument('--cand', type=float, default=200.0)
    ap.add_argument('--ext', type=float, default=1000.0)
    ap.add_argument('--dist_w', type=float, default=0.0)
    ap.add_argument('--lattice', type=float, default=0.0, help='大于 0
    ↪  时改用三角点阵搜索，值为间距')
    ap.add_argument('--rot_step', type=float, default=15.0)
    ap.add_argument('--off_n', type=int, default=4)
    ap.add_argument('--out', default='')
    a = ap.parse_args()
    if a.lattice > 0:
        best, ncell = lattice_search(a)
        if best is None:
            print('间距 %.0f 在所有旋转平移下都盖不全' % a.lattice)
            return
        n, L, rot, ox, oy, pts = best
        print('点阵间距 %.0f  元素 %d 格 x %d 区间  原点加 %d 点  旋转 %.0f 平移
        ↪  (%.0f, %.0f)  路线 %.0f 米'
              % (a.lattice, ncell, a.bins, n - 1, rot, ox, oy, L))
        if a.out:
            with open(os.path.join(HERE, a.out), 'w', encoding='utf-8') as f:
                json.dump({'args': vars(a), 'points': pts.tolist(), 'tour_m': L}, f,
                ↪  ensure_ascii=False, indent=1)
        return
    X, Y, h = elements(a.cell, a.bins)
    cand = tri_lattice(a.cand, R.ARENA_R + a.ext)
    cand = np.vstack([[0.0, 0.0], cand])
    M = cover_matrix(cand, X, Y, h, a.bins)
    chosen, left = greedy_cover(cand, M, dist_w=a.dist_w)
    kept = prune(chosen, M)
    pts = cand[kept]
    L, order = tour_len(pts)
    pts = pts[order]
    print('元素 %d 格 x %d 区间  候选 %d  贪心 %d 点 未覆盖 %d  删冗余后 %d 点  路线
    ↪  %.0f 米'
          % (len(X), a.bins, len(cand), len(chosen), left, len(kept), L))
    for p in pts:
        print('  (%.0f, %.0f)  r=%.0f' % (p[0], p[1], math.hypot(*p)))
    if a.out:
        with open(os.path.join(HERE, a.out), 'w', encoding='utf-8') as f:
            json.dump({'args': vars(a), 'points': pts.tolist(), 'tour_m': L}, f,
            ↪  ensure_ascii=False, indent=1)
```

```
if __name__ == '__main__':
    main()
```

q4_verify.py　问题四：覆盖扫描点的抽样核验

```
# -*- coding: utf-8 -*-
""" 问题 4 覆盖扫描点的抽样核验，以及不做网格保守收紧时点阵能省多少点。

核验：在目标区域内均匀抽源的位置，定向方向在 0 到 360 度均匀抽，接收半径取最坏的 1000 米，
↪
另在边界圆上加抽朝外的源，因为它们只能在区域外被测到。一个源被发现，
↪  当且仅当某个扫描点离它
不超过接收半径，且方位与定向方向相差不超过 90 度。统计漏掉的个数。

另外按接收半径在 1000 到 1500 米均匀分布，统计放宽点阵间距时的漏检概率，供灵敏度分析用。

  python q4_verify.py --pts kernel/q4_lattice_950.json
  python q4_verify.py --lattice 1000 1100 1200
"""
import os
import sys
import json
import math
import argparse
import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from kernel import rules as R
from q34_plan import open_tsp
from q4_cover import lattice_points, tour_len

HERE = os.path.dirname(os.path.abspath(__file__))


def draw(n, rng, boundary_share=0.2):
    nb = int(n * boundary_share)
    na = n - nb
    r = R.ARENA_R * np.sqrt(rng.random(na))
    a = rng.uniform(0, 2 * math.pi, na)
    X = np.concatenate([r * np.cos(a), R.ARENA_R * np.cos(ab := rng.uniform(0, 2 *
    ↪  math.pi, nb))])
    Y = np.concatenate([r * np.sin(a), R.ARENA_R * np.sin(ab)])
    psi = np.concatenate([rng.uniform(0, 360, na), np.degrees(ab) + rng.uniform(-90,
    ↪  90, nb)])
    return X, Y, psi


def detect_matrix(Q, X, Y, psi, radius):
    """ 每个源能被哪些扫描点发现，形状 (源数, 点数)。"""
    dx = Q[None, :, 0] - X[:, None]
    dy = Q[None, :, 1] - Y[:, None]
    d = np.hypot(dx, dy)
    ang = np.degrees(np.arctan2(dy, dx))
    diff = np.abs((ang - psi[:, None] + 180.0) % 360.0 - 180.0)
    rr = radius[:, None] if np.ndim(radius) else radius
```

```
    return (d <= rr) & ((diff <= 90.0) | (d < 1e-9))


def miss_stats(Q, n, seed, chunk=200000):
    rng = np.random.default_rng(seed)
    miss_worst = miss_rand = tot = 0
    for _ in range(int(math.ceil(n / chunk))):
        X, Y, psi = draw(chunk, rng)
        miss_worst += int((~detect_matrix(Q, X, Y, psi, R.R_MIN).any(1)).sum())
        rad = rng.uniform(R.R_MIN, R.R_MAX, chunk)
        miss_rand += int((~detect_matrix(Q, X, Y, psi, rad).any(1)).sum())
        tot += chunk
    return miss_worst, miss_rand, tot


def exact_lattice(s, n_samp, seed, rot_step=15.0, off_n=4):
    """ 按抽样元素直接删冗余点，不做格子与朝向区间的保守收紧。原点强制入选。"""
    rng = np.random.default_rng(seed)
    X, Y, psi = draw(n_samp, rng)
    best = None
    for rot in np.arange(0.0, 60.0, rot_step):
        for fi in range(off_n):
            for fj in range(off_n):
                ox, oy = s * fi / off_n, s * math.sqrt(3.0) / 2.0 * fj / off_n
                cand = np.vstack([[0.0, 0.0], lattice_points(s, rot, ox, oy,
                ↪  R.ARENA_R + s)])
                M = detect_matrix(cand, X, Y, psi, R.R_MIN)
                if not M.any(1).all():
                    continue
                cnt = M.sum(1)
                keep = list(range(len(cand)))
                for k in sorted(range(1, len(cand)), key=lambda k: M[:, k].sum()):
                    if (cnt - M[:, k] > 0).all():
                        cnt = cnt - M[:, k]
                        keep.remove(k)
                pts = cand[keep]
                L, order = tour_len(pts[1:])
                if best is None or (len(keep), L) < (best[0], best[1]):
                    best = (len(keep), L, rot, ox, oy, pts[1:][order])
    return best


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--pts', default='')
    ap.add_argument('--lattice', type=float, nargs='*', default=[])
    ap.add_argument('--n', type=int, default=2000000)
    ap.add_argument('--samp', type=int, default=150000)
    ap.add_argument('--seed', type=int, default=11)
    a = ap.parse_args()
    out = {}
    if a.pts:
        with open(os.path.join(HERE, a.pts), encoding='utf-8') as f:
            pts = np.array(json.load(f)['points'])
        Q = np.vstack([[0.0, 0.0], pts])
        mw, mr, tot = miss_stats(Q, a.n, a.seed)
        out[a.pts] = {'points_incl_origin': len(Q), 'samples': tot, 'miss_R1000':
        ↪  mw, 'miss_Runiform': mr}
```

```
        print(a.pts, out[a.pts], flush=True)
    for s in a.lattice:
        b = exact_lattice(s, a.samp, a.seed)
        if b is None:
            print('间距 %.0f 抽样意义下也盖不全' % s, flush=True)
            continue
        n, L, rot, ox, oy, pts = b
        Q = np.vstack([[0.0, 0.0], pts])
        mw, mr, tot = miss_stats(Q, a.n, a.seed + 1)
        out['lattice_%.0f' % s] = {'points_incl_origin': n, 'tour_m': L, 'rot': rot,
        ↪  'off': [ox, oy],
                                   'check_samples': tot, 'miss_R1000': mw,
                                   ↪  'miss_Runiform': mr}
        print('间距 %.0f  原点加 %d 点  路线 %.0f 米  复核 %d 个源 最坏半径漏 %d 个
        ↪  半径随机漏 %d 个'
              % (s, n - 1, L, tot, mw, mr), flush=True)
        with open(os.path.join(HERE, 'kernel', 'q4_exact_%.0f.json' % s), 'w',
        ↪  encoding='utf-8') as f:
            json.dump({'points': pts.tolist(), 'tour_m': L, 'rot': rot, 'off': [ox,
            ↪  oy]}, f, indent=1)
    with open(os.path.join(HERE, '..', 'out', 'q4_verify.json'), 'w',
    ↪  encoding='utf-8') as f:
        json.dump(out, f, ensure_ascii=False, indent=1)


if __name__ == '__main__':
    main()
```

q4_design2.py　问题四：抽样集合覆盖设计

```
# -*- coding: utf-8 -*-
""" 问题 4 覆盖扫描点的抽样设计。与 q34_plan 的 samples 判定用同一批样本、同样的余量，
设计出的点在运行时一定判为全覆盖，不会再触发补点。

两种选点办法都试，按点数与路线长度取优：
1 三角点阵：搜间距、旋转与平移，原点强制入选，逐个试删冗余点。
2 贪心集合覆盖：候选取 150 米间距的细点阵，每次选能多盖住样本最多的点，盖满后再删冗余点。
最后用另一批 200 万个源、接收半径取 1000 米整做独立核验。

  python q4_design2.py --lattice 980 1000 1030 --greedy
输出 kernel/q4_samples_design.json
"""
import os
import sys
import json
import math
import argparse
import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from kernel import rules as R
from q34_plan import Params, q4_samples, sample_cover, HERE
from q4_cover import lattice_points, tour_len
from q4_verify import miss_stats


def cover_cols(cands, X, Y, psi, r, ang):
```

```
    return np.stack([sample_cover(cx, cy, X, Y, psi, r, ang) for cx, cy in cands],
    ↪  axis=1)


def prune(M, forced=(0,)):
    """ 按贡献从少到多试删，删后每个样本仍至少被一点盖住就删。forced 里的列不删。"""
    cnt = M.sum(1)
    keep = list(range(M.shape[1]))
    for k in sorted((k for k in keep if k not in forced), key=lambda k: M[:,
    ↪  k].sum()):
        col = M[:, k]
        if (cnt[col] > 1).all():
            cnt = cnt - col
            keep.remove(k)
    return keep


def lattice_design(s, X, Y, psi, r, ang, rot_step, off_n):
    best = None
    for rot in np.arange(0.0, 60.0, rot_step):
        for fi in range(off_n):
            for fj in range(off_n):
                ox, oy = s * fi / off_n, s * math.sqrt(3.0) / 2.0 * fj / off_n
                cand = np.vstack([[0.0, 0.0], lattice_points(s, rot, ox, oy,
                ↪  R.ARENA_R + s)])
                M = cover_cols(cand, X, Y, psi, r, ang)
                if not M.any(1).all():
                    continue
                keep = prune(M)
                pts = cand[keep][1:]
                L, order = tour_len(pts)
                if best is None or (len(pts), L) < (best[0], best[1]):
                    best = (len(pts), L, pts[order], 'lattice s=%.0f rot=%.0f
                    ↪  off=(%.0f,%.0f)' % (s, rot, ox, oy))
    return best


def greedy_design(X, Y, psi, r, ang, spacing, rmax):
    cand = np.vstack([[0.0, 0.0], lattice_points(spacing, 0.0, 0.0, 0.0, rmax)])
    M = cover_cols(cand, X, Y, psi, r, ang)
    if not M.any(1).all():
        return None
    covered = M[:, 0].copy()
    chosen = [0]
    while not covered.all():
        gain = M[~covered].sum(0)
        k = int(gain.argmax())
        chosen.append(k)
        covered |= M[:, k]
    sub = M[:, chosen]
    keep = prune(sub)
    pts = cand[[chosen[i] for i in keep]][1:]
    L, order = tour_len(pts)
    return (len(pts), L, pts[order], 'greedy spacing=%.0f' % spacing)


def missed(Q, n, seed, chunk=200000):
```

```
    """独立核验，接收半径取 1000 米整、方位差上限 90 度，返回漏掉的源 (x, y, 定向方向)。
    ↪  """
    from q4_verify import draw, detect_matrix
    rng = np.random.default_rng(seed)
    out = []
    for _ in range(int(math.ceil(n / chunk))):
        X, Y, psi = draw(chunk, rng)
        m = ~detect_matrix(Q, X, Y, psi, R.R_MIN).any(1)
        out.extend(zip(X[m], Y[m], psi[m]))
    return out


def repair_greedy(a, p):
    """贪心设计加补漏迭代：核验漏掉的源连同附近的扰动副本并入设计样本，重新贪心，
    ↪  直到核验零漏检。"""
    X0, Y0, psi0 = q4_samples(int(p.q4_nsamp), int(p.q4_samp_seed))
    r, ang = p.q4_r_design, p.q4_ang_design
    rng = np.random.default_rng(99)
    extra = np.zeros((0, 3))
    best = None
    for it in range(a.iters):
        X = np.concatenate([X0, extra[:, 0]])
        Y = np.concatenate([Y0, extra[:, 1]])
        psi = np.concatenate([psi0, extra[:, 2]])
        b = greedy_design(X, Y, psi, r, ang, a.spacing, R.ARENA_R + 1000.0)
        Q = np.vstack([[0.0, 0.0], b[2]])
        miss = missed(Q, a.check, seed=1000 + it)
        print('补漏第 %d 轮  补充样本 %d  原点加 %d 点  路线 %.0f 米  核验 %d 个源漏
        ↪  %d 个'
              % (it, len(extra), b[0], b[1], a.check, len(miss)), flush=True)
        best = (b, extra.copy(), len(miss))
        if not miss:
            break
        m = np.array(miss)
        jit = np.repeat(m, 20, axis=0)
        jit[:, 0] += rng.normal(0, 15.0, len(jit))
        jit[:, 1] += rng.normal(0, 15.0, len(jit))
        jit[:, 2] += rng.normal(0, 3.0, len(jit))
        rr = np.hypot(jit[:, 0], jit[:, 1])
        jit[rr > R.ARENA_R, :2] *= (R.ARENA_R / rr[rr > R.ARENA_R])[:, None]
        extra = np.vstack([extra, m, jit])
    b, extra, nmiss = best
    Q = np.vstack([[0.0, 0.0], b[2]])
    final = missed(Q, a.final_check, seed=424242)
    print('最终独立核验 %d 个源漏 %d 个' % (a.final_check, len(final)), flush=True)
    with open(os.path.join(HERE, a.out), 'w', encoding='utf-8') as f:
        json.dump({'points': b[2].tolist(), 'tour_m': b[1], 'design': b[3] + '
        ↪  repaired',
                   'nsamp': p.q4_nsamp, 'samp_seed': p.q4_samp_seed, 'r_design': r,
                   ↪  'ang_design': ang,
                   'extra_samples': extra.tolist(),
                   'check': {'samples': a.final_check, 'miss_R1000': len(final)}},
                  f, ensure_ascii=False, indent=1)


def main():
    ap = argparse.ArgumentParser()
```

```
    ap.add_argument('--lattice', type=float, nargs='*', default=[980.0, 1000.0,
    ↪  1030.0])
    ap.add_argument('--greedy', action='store_true')
    ap.add_argument('--repair', action='store_true', help='只做贪心加补漏迭代')
    ap.add_argument('--spacing', type=float, default=150.0)
    ap.add_argument('--iters', type=int, default=8)
    ap.add_argument('--check', type=int, default=2000000)
    ap.add_argument('--final_check', type=int, default=10000000)
    ap.add_argument('--rot_step', type=float, default=10.0)
    ap.add_argument('--off_n', type=int, default=5)
    ap.add_argument('--out', default='kernel/q4_samples_design.json')
    a = ap.parse_args()
    p = Params()
    if a.repair:
        repair_greedy(a, p)
        return
    X, Y, psi = q4_samples(int(p.q4_nsamp), int(p.q4_samp_seed))
    r, ang = p.q4_r_design, p.q4_ang_design
    designs = []
    for s in a.lattice:
        b = lattice_design(s, X, Y, psi, r, ang, a.rot_step, a.off_n)
        if b is None:
            print('点阵间距 %.0f 盖不全' % s, flush=True)
            continue
        print('%s  原点加 %d 点  路线 %.0f 米' % (b[3], b[0], b[1]), flush=True)
        designs.append(b)
    if a.greedy:
        for sp in (150.0, 250.0):
            b = greedy_design(X, Y, psi, r, ang, sp, R.ARENA_R + 1000.0)
            if b is not None:
                print('%s  原点加 %d 点  路线 %.0f 米' % (b[3], b[0], b[1]),
                ↪  flush=True)
                designs.append(b)
    # 路线长度加上每点扫描大约 8 个频道的检测耗时折算成米，作为选优口径
    best = min(designs, key=lambda d: d[1] + d[0] * 8 * 6.0 * R.SPEED)
    Q = np.vstack([[0.0, 0.0], best[2]])
    mw, mr, tot = miss_stats(Q, 2000000, seed=777)
    print('选用 %s  原点加 %d 点  路线 %.0f 米  独立核验 %d 个源 半径 1000 漏 %d 个
    ↪  半径随机漏 %d 个'
          % (best[3], best[0], best[1], tot, mw, mr), flush=True)
    with open(os.path.join(HERE, a.out), 'w', encoding='utf-8') as f:
        json.dump({'points': best[2].tolist(), 'tour_m': best[1], 'design': best[3],
                   'nsamp': p.q4_nsamp, 'samp_seed': p.q4_samp_seed, 'r_design': r,
                   ↪  'ang_design': ang,
                   'check': {'samples': tot, 'miss_R1000': mw, 'miss_Runiform': mr},
                   'all': [{'design': d[3], 'n': d[0], 'tour_m': d[1]} for d in
                   ↪  designs]},
                  f, ensure_ascii=False, indent=1)


if __name__ == '__main__':
    main()
```

q4_design3.py　问题四：多起点贪心、挪点与补漏迭代设计

```
# -*- coding: utf-8 -*-
""" 问题 4 覆盖扫描点的路线导向搜索与补漏。
```

```
总代价 = 开放最短路线长度 + 点数乘每点约 8 个频道的检测耗时折算的距离。
1 多起点贪心：候选细点阵的间距与平移、同分时的随机打破、偏向离已选点近的紧凑项，
↪ 都作为变体。
2 局部挪点：对代价最小的几组，逐点尝试挪到 200 米内的其他位置，
↪ 覆盖不破且总代价下降就接受。
3 补漏：已选的点保持不动，用独立样本核验，漏掉的源并入设计样本，
↪ 只追加能盖住它们的最少的点，
  换一批独立样本再核验，直到零漏检，最后用一千万个源做终检。
设计余量可由 --r 与 --ang 指定，写进结果文件，运行时照文件判定。

  python q4_design3.py --procs 4 --r 990 --ang 89.5
输出 kernel/q4_design_best.json
"""
import os
for _k in ('OMP_NUM_THREADS', 'OPENBLAS_NUM_THREADS', 'MKL_NUM_THREADS'):
    os.environ.setdefault(_k, '1')
import sys
import json
import math
import argparse
import numpy as np
from multiprocessing import Pool

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from kernel import rules as R
from q34_plan import Params, q4_samples, sample_cover, open_tsp, HERE
from q4_cover import lattice_points
from q4_design2 import cover_cols, prune, missed

SCAN_M = 8 * 6.0 * R.SPEED          # 每个扫描点的检测耗时折算成米
G = {}


def tour_m(pts):
    if len(pts) == 0:
        return 0.0
    order = open_tsp((0.0, 0.0), [tuple(p) for p in pts], oropt=True)
    cur = np.zeros(2)
    L = 0.0
    for k in order:
        L += float(np.hypot(*(pts[k] - cur)))
        cur = pts[k]
    return L


def cost(pts):
    return tour_m(pts) + len(pts) * SCAN_M


def init_worker(X, Y, psi, r, ang):
    G.update(X=X, Y=Y, psi=psi, r=r, ang=ang)


def greedy_variant(job):
    spacing, shift, lam, seed = job
    rng = np.random.default_rng(seed)
    X, Y, psi, r, ang = G['X'], G['Y'], G['psi'], G['r'], G['ang']
```

```
    cand = np.vstack([[0.0, 0.0], lattice_points(spacing, 0.0, shift[0], shift[1],
    ↪  R.ARENA_R + 1000.0)])
    M = cover_cols(cand, X, Y, psi, r, ang)
    if not M.any(1).all():
        return None
    covered = M[:, 0].copy()
    chosen = [0]
    while not covered.all():
        gain = M[~covered].sum(0).astype(float)
        if lam > 0:
            dmin = np.min(np.hypot(cand[:, None, 0] - cand[None, chosen, 0],
                                   cand[:, None, 1] - cand[None, chosen, 1]),
                                   ↪  axis=1)
            gain = gain * np.clip(1.0 - lam * dmin / 1000.0, 0.05, None)
        gain = gain + rng.random(len(gain)) * 0.5
        k = int(gain.argmax())
        chosen.append(k)
        covered |= M[:, k]
    keep = prune(M[:, chosen])
    pts = cand[[chosen[i] for i in keep]][1:]
    return {'job': job, 'pts': pts, 'cost': cost(pts), 'n': len(pts), 'tour':
    ↪  tour_m(pts)}


def local_move(pts, X, Y, psi, r, ang, radius=200.0, step=50.0, rounds=3):
    """ 逐点尝试挪到附近位置，覆盖不破且总代价下降就接受。"""
    pts = pts.copy()
    Mcur = cover_cols(np.vstack([[0.0, 0.0], pts]), X, Y, psi, r, ang)
    cnt = Mcur.sum(1)
    best = cost(pts)
    offs = [(dx, dy) for dx in np.arange(-radius, radius + 1, step) for dy in
    ↪  np.arange(-radius, radius + 1, step)
            if 0 < dx * dx + dy * dy <= radius * radius]
    for _ in range(rounds):
        improved = False
        for i in range(len(pts)):
            need = (cnt - Mcur[:, i + 1]) == 0
            for dx, dy in offs:
                q = pts[i] + np.array([dx, dy])
                col = sample_cover(q[0], q[1], X, Y, psi, r, ang)
                if need.any() and not col[need].all():
                    continue
                trial = pts.copy()
                trial[i] = q
                c = cost(trial)
                if c < best - 1.0:
                    cnt = cnt - Mcur[:, i + 1] + col
                    Mcur[:, i + 1] = col
                    pts, best = trial, c
                    need = (cnt - col) == 0
                    improved = True
        if not improved:
            break
    return pts, best


def add_points_for(pts, miss, r, ang, spacing=100.0):
```

```
    """保持已有点不动，对漏掉的源贪心追加最少的点。候选取细点阵，只需对漏检样本求覆盖。
    ↪  """
    m = np.asarray(miss, dtype=float)
    cand = lattice_points(spacing, 0.0, 0.0, 0.0, R.ARENA_R + 1000.0)
    M = cover_cols(cand, m[:, 0], m[:, 1], m[:, 2], r, ang)
    left = np.ones(len(m), dtype=bool)
    added = []
    while left.any():
        gain = M[left].sum(0)
        if gain.max() <= 0:
            break
        # 同样盖得住时挑离已有路线最近的，少绕路
        dmin = np.min(np.hypot(cand[:, None, 0] - pts[None, :, 0], cand[:, None, 1]
        ↪  - pts[None, :, 1]), axis=1)
        k = int(np.lexsort((dmin, -gain))[0])
        added.append(cand[k])
        left &= ~M[:, k]
    return np.vstack([pts, np.array(added)]) if added else pts


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--procs', type=int, default=4)
    ap.add_argument('--variants', type=int, default=48)
    ap.add_argument('--top', type=int, default=3)
    ap.add_argument('--rounds', type=int, default=3)
    ap.add_argument('--r', type=float, default=None)
    ap.add_argument('--ang', type=float, default=None)
    ap.add_argument('--check', type=int, default=2000000)
    ap.add_argument('--final_check', type=int, default=10000000)
    ap.add_argument('--out', default='kernel/q4_design_best.json')
    ap.add_argument('--scan_ch', type=float, default=8.0,
    ↪  help='每个扫描点按检测多少个频道折算代价')
    a = ap.parse_args()
    global SCAN_M
    SCAN_M = a.scan_ch * 6.0 * R.SPEED
    p = Params()
    X, Y, psi = q4_samples(int(p.q4_nsamp), int(p.q4_samp_seed))
    r = a.r if a.r is not None else p.q4_r_design
    ang = a.ang if a.ang is not None else p.q4_ang_design
    rng = np.random.default_rng(2026)
    jobs = []
    for v in range(a.variants):
        spacing = (100.0, 150.0, 200.0)[v % 3]
        shift = (float(rng.uniform(0, spacing)), float(rng.uniform(0, spacing)))
        lam = (0.0, 0.3, 0.6, 1.0)[(v // 3) % 4]
        jobs.append((spacing, shift, lam, int(rng.integers(1 << 30))))
    with Pool(a.procs, initializer=init_worker, initargs=(X, Y, psi, r, ang)) as
    ↪  pool:
        res = [x for x in pool.map(greedy_variant, jobs) if x is not None]
    res.sort(key=lambda x: x['cost'])
    print('余量 r=%.0f ang=%.1f  贪心变体 %d 个，代价最小五个：' % (r, ang, len(res)),
    ↪  flush=True)
    for x in res[:5]:
        print('   间距 %.0f 紧凑 %.1f  原点加 %d 点  路线 %.0f 米' % (x['job'][0],
        ↪  x['job'][2], x['n'], x['tour']), flush=True)
    best = None
    for x in res[:a.top]:
```

```
        pts, c = local_move(x['pts'], X, Y, psi, r, ang, rounds=a.rounds)
        print('   挪点后 原点加 %d 点  路线 %.0f 米' % (len(pts), tour_m(pts)),
        ↪  flush=True)
        if best is None or c < best[1]:
            best = (pts, c)
    pts = best[0]
    extra = []
    for it in range(10):
        n_check = a.check if it < 9 else a.final_check
        miss = missed(np.vstack([[0.0, 0.0], pts]), n_check, seed=7000 + it)
        print('补漏第 %d 轮  原点加 %d 点  路线 %.0f 米  核验 %d 个源漏 %d 个'
              % (it, len(pts), tour_m(pts), n_check, len(miss)), flush=True)
        if not miss:
            if n_check == a.final_check:
                break
            # 两百万零漏检后直接做千万终检
            final = missed(np.vstack([[0.0, 0.0], pts]), a.final_check, seed=909090)
            print('   终检 %d 个源漏 %d 个' % (a.final_check, len(final)),
            ↪  flush=True)
            if not final:
                break
            miss = final
        extra.extend(miss)
        pts = add_points_for(pts, miss, r, ang)
    order = open_tsp((0.0, 0.0), [tuple(q) for q in pts], oropt=True)
    pts = pts[order]
    L = tour_m(pts)
    print('选定 原点加 %d 点  路线 %.0f 米' % (len(pts), L), flush=True)
    with open(os.path.join(HERE, a.out), 'w', encoding='utf-8') as f:
        json.dump({'points': pts.tolist(), 'tour_m': L, 'design': 'greedy variants +
        ↪  local move + add-point repair',
                   'nsamp': p.q4_nsamp, 'samp_seed': p.q4_samp_seed, 'r_design': r,
                   ↪  'ang_design': ang,
                   'extra_samples': [list(map(float, e)) for e in extra]},
                  f, ensure_ascii=False, indent=1)


if __name__ == '__main__':
    main()
```

q4_design_sa.py　问题四：贪心起步的模拟退火设计与补漏核验

```
# -*- coding: utf-8 -*-
""" 问题 4 覆盖扫描点的模拟退火设计。

从已有设计出发，随机挪点、删点、换点，覆盖不破时按总代价的 Metropolis 准则接受。
总代价 = 开放最短路线长度 + 点数乘每点检测耗时折算的距离。
多条链并行，各链种子与每点折算距离不同，最后取代价最小的一条，用千万个源终检并补漏。

  python q4_design_sa.py --procs 16 --iters 20000 --out kernel/q4_design_sa.json
"""
import os
for _k in ('OMP_NUM_THREADS', 'OPENBLAS_NUM_THREADS', 'MKL_NUM_THREADS'):
    os.environ.setdefault(_k, '1')
import sys
import json
import math
```

```
import time
import argparse
import numpy as np
from multiprocessing import Pool

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from kernel import rules as R
from q34_plan import Params, q4_samples, sample_cover, open_tsp, HERE
from q4_design2 import missed
from q4_design3 import add_points_for

G = {}


def tour_m(pts):
    if len(pts) == 0:
        return 0.0
    order = open_tsp((0.0, 0.0), [tuple(p) for p in pts], oropt=True)
    cur = np.zeros(2)
    L = 0.0
    for k in order:
        L += float(np.hypot(*(pts[k] - cur)))
        cur = pts[k]
    return L


def init_worker(X, Y, psi, r, ang):
    G.update(X=X, Y=Y, psi=psi, r=r, ang=ang)


def cov(q):
    return sample_cover(q[0], q[1], G['X'], G['Y'], G['psi'], G['r'], G['ang'])


def chain(job):
    seed, iters, scan_m, pts0, T0 = job
    rng = np.random.default_rng(seed)
    pts = [np.array(p, dtype=float) for p in pts0]
    cols = [cov(p) for p in pts]
    cnt = cov(np.zeros(2)).astype(np.int32)
    for c in cols:
        cnt += c
    if not (cnt > 0).all():
        # 样本比起点设计时多，先对没盖住的样本补点再退火
        X, Y, psi = G['X'], G['Y'], G['psi']
        m = np.flatnonzero(cnt == 0)
        P2 = add_points_for(np.array(pts), np.stack([X[m], Y[m], psi[m]], axis=1),
        ↪  G['r'], G['ang'])
        for q in P2[len(pts):]:
            q = np.array(q, dtype=float)
            pts.append(q)
            cols.append(cov(q))
            cnt += cols[-1]
    assert (cnt > 0).all()
    L = tour_m(np.array(pts))
    cost = L + scan_m * len(pts)
    best = (cost, [p.copy() for p in pts], L)
    lim = R.ARENA_R + 1000.0
```

```
    t0 = time.time()
    for it in range(iters):
        T = T0 * (1.0 - it / iters) + 1.0
        u = rng.random()
        n = len(pts)
        if u < 0.15 and n > 1:
            # 删点
            i = int(rng.integers(n))
            if ((cnt - cols[i]) > 0).all():
                trial = pts[:i] + pts[i + 1:]
                L2 = tour_m(np.array(trial))
                c2 = L2 + scan_m * (n - 1)
                if c2 < cost or rng.random() < math.exp((cost - c2) / T):
                    cnt -= cols[i]
                    pts, cols, L, cost = trial, cols[:i] + cols[i + 1:], L2, c2
        elif u < 0.25:
            # 换点: 删一个, 在另一个点附近或随机处加一个
            i = int(rng.integers(n))
            if rng.random() < 0.5:
                j = int(rng.integers(n))
                q = pts[j] + rng.normal(0.0, 400.0, 2)
            else:
                a = rng.uniform(0.0, 2.0 * math.pi)
                q = lim * math.sqrt(rng.random()) * np.array([math.cos(a),
                ↪  math.sin(a)])
            if np.hypot(*q) > lim:
                continue
            cq = cov(q)
            if not ((cnt - cols[i] + cq) > 0).all():
                continue
            trial = pts[:i] + pts[i + 1:] + [q]
            L2 = tour_m(np.array(trial))
            c2 = L2 + scan_m * n
            if c2 < cost or rng.random() < math.exp((cost - c2) / T):
                cnt = cnt - cols[i] + cq
                pts, cols, L, cost = trial, cols[:i] + cols[i + 1:] + [cq], L2, c2
        else:
            # 挪点
            i = int(rng.integers(n))
            sig = (30.0, 80.0, 200.0)[int(rng.integers(3))]
            q = pts[i] + rng.normal(0.0, sig, 2)
            if np.hypot(*q) > lim:
                continue
            cq = cov(q)
            if not ((cnt - cols[i] + cq) > 0).all():
                continue
            trial = list(pts)
            trial[i] = q
            L2 = tour_m(np.array(trial))
            c2 = L2 + scan_m * n
            if c2 < cost or rng.random() < math.exp((cost - c2) / T):
                cnt = cnt - cols[i] + cq
                cols[i] = cq
                pts, L, cost = trial, L2, c2
        if cost < best[0] - 1e-6:
            best = (cost, [p.copy() for p in pts], L)
    return {'seed': seed, 'scan_m': scan_m, 'cost': best[0], 'n': len(best[1]),
    ↪  'tour': best[2],
```

```
            'pts': [p.tolist() for p in best[1]], 'wall': time.time() - t0}


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--procs', type=int, default=16)
    ap.add_argument('--chains', type=int, default=16)
    ap.add_argument('--iters', type=int, default=20000)
    ap.add_argument('--T0', type=float, default=300.0)
    ap.add_argument('--scan_ch', default='8',
    ↪  help='每个扫描点按检测多少个频道折算代价，逗号分隔多组轮流分给各链')
    ap.add_argument('--init', default='kernel/q4_design_r970.json')
    ap.add_argument('--out', default='kernel/q4_design_sa.json')
    ap.add_argument('--final_check', type=int, default=10000000)
    ap.add_argument('--nsamp', type=int, default=None, help='设计样本数，不给就按
    ↪  Params.q4_nsamp')
    a = ap.parse_args()
    p = Params()
    if a.nsamp:
        p.q4_nsamp = a.nsamp
    with open(os.path.join(HERE, a.init), encoding='utf-8') as f:
        d0 = json.load(f)
    r, ang = float(d0.get('r_design', p.q4_r_design)), float(d0.get('ang_design',
    ↪  p.q4_ang_design))
    X, Y, psi = q4_samples(int(p.q4_nsamp), int(p.q4_samp_seed))
    if d0.get('extra_samples'):
        ex = np.asarray(d0['extra_samples'], dtype=float)
        X, Y, psi = np.concatenate([X, ex[:, 0]]), np.concatenate([Y, ex[:, 1]]),
        ↪  np.concatenate([psi, ex[:, 2]])
    chs = [float(x) for x in a.scan_ch.split(',')]
    jobs = [(1000 + k, a.iters, chs[k % len(chs)] * 6.0 * R.SPEED, d0['points'],
    ↪  a.T0) for k in range(a.chains)]
    print('起点 %d 点 路线 %.0f 米  余量 r=%.0f ang=%.1f  %d 条链各 %d 步' %
    ↪  (len(d0['points']), tour_m(np.array(d0['points'])), r, ang, a.chains,
    ↪  a.iters), flush=True)
    with Pool(a.procs, initializer=init_worker, initargs=(X, Y, psi, r, ang)) as
    ↪  pool:
        res = pool.map(chain, jobs, chunksize=1)
    res.sort(key=lambda x: x['cost'])
    for x in res:
        print('  链 %d 折算频道 %.0f  %d 点 路线 %.0f 米 代价 %.0f  用时 %.0f 秒' %
        ↪  (x['seed'], x['scan_m'] / 30.0, x['n'], x['tour'], x['cost'],
        ↪  x['wall']), flush=True)
    init_worker(X, Y, psi, r, ang)
    # 各链的最优都存下来，按折算频道分组各留最好的一个
    by = {}
    for x in res:
        by.setdefault(x['scan_m'], x)
    outs = {}
    for scan_m, x in sorted(by.items()):
        pts = np.array(x['pts'])
        extra = list(d0.get('extra_samples', []))
        for it in range(8):
            n_check = 2000000 if it < 7 else a.final_check
            miss = missed(np.vstack([[0.0, 0.0], pts]), n_check, seed=8100 + it)
            print('折算 %.0f 补漏第 %d 轮 %d 点 路线 %.0f 米 核验 %d 漏 %d' % (scan_m
            ↪  / 30.0, it, len(pts), tour_m(pts), n_check, len(miss)), flush=True)
            if not miss:
```

```
                if n_check == a.final_check:
                    break
                final = missed(np.vstack([[0.0, 0.0], pts]), a.final_check,
                ↪  seed=919191)
                print('   终检 %d 漏 %d' % (a.final_check, len(final)), flush=True)
                if not final:
                    break
                miss = final
            extra.extend([list(map(float, e)) for e in miss])
            pts = add_points_for(pts, miss, r, ang)
        order = open_tsp((0.0, 0.0), [tuple(q) for q in pts], oropt=True)
        pts = pts[order]
        L = tour_m(pts)
        name = a.out.replace('.json', '_c%d.json' % round(scan_m / 30.0))
        outs[name] = {'points': pts.tolist(), 'tour_m': L, 'design': 'simulated
        ↪  annealing from %s' % a.init,
                      'nsamp': p.q4_nsamp, 'samp_seed': p.q4_samp_seed, 'r_design':
                      ↪  r, 'ang_design': ang,
                      'scan_ch': scan_m / 30.0, 'extra_samples': extra}
        print('选定 折算 %.0f: 原点加 %d 点  路线 %.0f 米 -> %s' % (scan_m / 30.0,
        ↪  len(pts), L, name), flush=True)
        with open(os.path.join(HERE, name), 'w', encoding='utf-8') as f:
            json.dump(outs[name], f, ensure_ascii=False, indent=1)


if __name__ == '__main__':
    main()
```

q34_batch2.py　问题三、四：离线批量演练、灵敏度与策略对比

```
# -*- coding: utf-8 -*-
""" 新策略 q34_plan 的离线批量演练，只在服务器上跑。

  python q34_batch2.py --problem 3 --cases 200 --tag base
  python q34_batch2.py --problem 3 --cases 200 --ab ring_r=1250,1350,1450
  python q34_batch2.py --problem 3 --one 30001 -v        单局带过程输出，查问题用
"""
import os
# 每局一个进程，numpy 再开多线程会让 19 个进程互相抢核，必须在导入 numpy 前限定单线程
for _k in ('OMP_NUM_THREADS', 'OPENBLAS_NUM_THREADS', 'MKL_NUM_THREADS'):
    os.environ.setdefault(_k, '1')
import sys
import json
import time
import argparse
import statistics as st
from multiprocessing import Pool

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from kernel.sim import Arena
from kernel.api import LocalClient
from q34_plan import PlanRobot, Params
import math
import numpy as np
from kernel import rules as R
```

```
def leak_check(robot, problem, n=60000, seed=1):
    """从未测到信号就判空的频道，用它实际测过的位置按真规则核验：
    ↪ 随机源里有多少个从这些位置都收不到。
    问题3 按接收半径 1000 米整；问题4 还按定向源前方 90 度。返回 (漏检数, 核验总数)。"""
    from q4_verify import draw, detect_matrix
    rng = np.random.default_rng(seed)
    miss = tot = 0
    for c in robot.chans.values():
        if c.state != 'empty' or c.det or c.alive.any():
            continue      # 测到过信号的、或凑满 16 个后直接判空的频道不在核验之列
        Q = np.array(c.nosig, dtype=float)
        if len(Q) == 0:
            continue
        X, Y, psi = draw(n, rng)
        if problem == 3:
            d = np.hypot(Q[None, :, 0] - X[:, None], Q[None, :, 1] - Y[:, None])
            m = ~(d <= R.R_MIN).any(1)
        else:
            m = ~detect_matrix(Q, X, Y, psi, R.R_MIN).any(1)
        miss += int(m.sum())
        tot += n
    return miss, tot

OUT_ROOT = os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'out')


ENV = {}          # 真实环境与题设不同时的设置，由命令行传入，灵敏度分析用


def play_one(job):
    seed, problem, kw, verbose = job[:4]
    env = job[4] if len(job) > 4 else {}
    n_dir = 0 if problem == 3 else env.get('n_dir', -1)
    r_range = (env['r_lo'], env['r_hi']) if 'r_lo' in env else None
    arena = Arena(seed=seed, n=env.get('n'), n_dir=n_dir, r_range=r_range,
    ↪ err_deg=env.get('err_deg'))
    k = dict(kw)
    k.setdefault('seed', seed)
    robot = PlanRobot(LocalClient(arena), Params(**k), omni=(problem == 3),
    ↪ verbose=verbose)
    t0 = time.time()
    err = ''
    try:
        robot.run()
    except Exception as e:
        import traceback
        err = traceback.format_exc() if verbose else repr(e)
    s = arena.stats()
    leak, leak_tot = leak_check(robot, problem)
    missed = [{'ch': x.ch, 'dir': x.face is not None, 'r': (x.x ** 2 + x.y ** 2) **
    ↪ 0.5}
              for x in arena.sources if not x.cleared]
    return {'seed': seed, 'n': s['n'], 'n_dir': s['n_dir'], 'cleared': s['cleared'],
            'ratio': s['ratio'], 'total_time': s['total_time'], 'avg_time':
            ↪ s['avg_time'],
            'tail_time': s['tail_time'], 'actions': robot.actions, 'measures':
            ↪ robot.n_measure,
```

```
            'misses': robot.n_miss, 'leak': leak, 'leak_tot': leak_tot, 'real_s':
            ↪  time.time() - t0, 'stop_reason': robot.stop_reason,
            'missed': missed, 'err': err}


def summarize(rows):
    ok = [r for r in rows if r['cleared']]
    avg = sorted(r['avg_time'] for r in ok)
    return {
        'cases': len(rows),
        'mean_ratio': st.mean(r['ratio'] for r in rows),
        'full_clear_pct': 100.0 * sum(r['ratio'] >= 1.0 for r in rows) / len(rows),
        'mean_avg_time': st.mean(avg),
        'pooled_avg_time': sum(r['total_time'] for r in ok) / sum(r['cleared'] for r
        ↪  in ok),
        'median_avg_time': avg[len(avg) // 2],
        'p90_avg_time': avg[int(0.9 * (len(avg) - 1))],
        'mean_total_time': st.mean(r['total_time'] for r in ok),
        'mean_tail_share_pct': 100.0 * st.mean(r['tail_time'] / r['total_time'] for
        ↪  r in ok),
        'mean_measures': st.mean(r['measures'] for r in rows),
        'mean_misses': st.mean(r['misses'] for r in rows),
        'max_real_s': max(r['real_s'] for r in rows),
        'mean_real_s': st.mean(r['real_s'] for r in rows),
        'errors': sum(1 for r in rows if r['err']),
        'leak': sum(r.get('leak', 0) for r in rows),
        'leak_tot': sum(r.get('leak_tot', 0) for r in rows),
        'missed_dir': sum(sum(m['dir'] for m in r['missed']) for r in rows),
        'missed_omni': sum(sum(not m['dir'] for m in r['missed']) for r in rows),
    }


def parse_sets(items):
    kw = {}
    for it in items or []:
        k, v = it.split('=', 1)
        base = getattr(Params, k)
        kw[k] = type(base)(float(v)) if isinstance(base, (int, float)) else v
    return kw


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--problem', type=int, choices=(3, 4), required=True)
    ap.add_argument('--cases', type=int, default=200)
    ap.add_argument('--start', type=int, default=30000)
    ap.add_argument('--procs', type=int, default=19)
    ap.add_argument('--set', action='append')
    ap.add_argument('--ab', default='')
    ap.add_argument('--tag', default='plan')
    ap.add_argument('--one', type=int, default=None)
    ap.add_argument('-v', action='store_true')
    ap.add_argument('--n', type=int, default=None, help='干扰源个数，不给就在 10 到
    ↪  16 之间随机')
    ap.add_argument('--n_dir', type=int, default=None, help='问题4 定向源个数，
    ↪  不给就随机')
    ap.add_argument('--err_deg', type=float, default=None, help='真实示向度误差上界')
    ap.add_argument('--r_lo', type=float, default=None)
```

```
    ap.add_argument('--r_hi', type=float, default=None)
    a = ap.parse_args()

    env = {}
    if a.n is not None:
        env['n'] = a.n
    if a.n_dir is not None:
        env['n_dir'] = a.n_dir
    if a.err_deg is not None:
        env['err_deg'] = a.err_deg
    if a.r_lo is not None:
        env['r_lo'], env['r_hi'] = a.r_lo, a.r_hi
    kw = parse_sets(a.set)
    if a.one is not None:
        r = play_one((a.one, a.problem, kw, a.v, env))
        print(json.dumps(r, ensure_ascii=False, indent=1))
        return

    out_dir = os.path.join(OUT_ROOT, 'q%d' % a.problem)
    os.makedirs(out_dir, exist_ok=True)
    seeds = list(range(a.start, a.start + a.cases))
    variants = [(a.tag, kw)]
    if a.ab:
        name, vals = a.ab.split('=', 1)
        variants = []
        for v in vals.split(','):
            k2 = dict(kw)
            k2.update(parse_sets(['%s=%s' % (name, v)]))
            variants.append(('%s_%s%s' % (a.tag, name, v), k2))
    report = {}
    for tag, k in variants:
        t0 = time.time()
        with Pool(a.procs) as pool:
            rows = pool.map(play_one, [(s, a.problem, k, False, env) for s in
            ↪  seeds], chunksize=1)
        s = summarize(rows)
        s['wall_s'] = time.time() - t0
        s['params'] = Params(**k).as_dict()
        report[tag] = s
        with open(os.path.join(out_dir, 'plan_%s.jsonl' % tag), 'w',
        ↪  encoding='utf-8') as f:
            for r in rows:
                f.write(json.dumps(r, ensure_ascii=False) + '\n')
        print('[%s] %d 局  比例 %.4f  全清 %.1f%%  平均时间 均值 %.1f 合计口径 %.1f
        ↪  中位 %.1f 九成 %.1f  '
              '收尾 %.1f%%  检测 %.0f 次 落空 %.1f 次  单局程序 均 %.1f 最长 %.1f 秒
              ↪  漏 定向 %d 全向 %d  报错 %d  空频道核验漏 %d/%d  用时 %.0f 秒'
             % (tag, s['cases'], s['mean_ratio'], s['full_clear_pct'],
              ↪  s['mean_avg_time'], s['pooled_avg_time'],
                 s['median_avg_time'], s['p90_avg_time'], s['mean_tail_share_pct'],
                 ↪  s['mean_measures'],
                 s['mean_misses'], s['mean_real_s'], s['max_real_s'],
                 ↪  s['missed_dir'], s['missed_omni'],
                 s['errors'], s['leak'], s['leak_tot'], s['wall_s']), flush=True)
        errs = [r for r in rows if r['err']]
        if errs:
            print('  首个报错 seed %d: %s' % (errs[0]['seed'], errs[0]['err']),
            ↪  flush=True)
```

```
    with open(os.path.join(out_dir, 'plan_report_%s.json' % a.tag), 'w',
    ↪  encoding='utf-8') as f:
        json.dump(report, f, ensure_ascii=False, indent=1)


if __name__ == '__main__':
    main()
```

q34_oracle.py　问题三、四：已知全部源位置时的路线对照

```
# -*- coding: utf-8 -*-
""" 事后对照：假如一开局就知道全部源的位置，走完覆盖扫描点和全部源的最短开放路线有多长。

与新策略逐局的实际行走距离比较，差距说明信息不足带来的回头路还有多大的压缩空间。
实际行走时间按 总时间 减 检测次数乘 6 秒 减 清除成功 5 秒 减 落空 3 秒 估计，
↪  切换频道按每次检测都切计，偏保守。

  python q34_oracle.py --problem 3 --rows ../out/q3/plan_new3.jsonl
"""
import os
import sys
import json
import argparse
import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from kernel import rules as R
from kernel.sim import Arena
from q34_plan import open_tsp, Params, HERE


def tour_length(start, pts):
    order = open_tsp(start, pts, oropt=True)
    cur = np.array(start, dtype=float)
    L = 0.0
    for k in order:
        p = np.array(pts[k])
        L += float(np.hypot(*(p - cur)))
        cur = p
    return L


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--problem', type=int, choices=(3, 4), required=True)
    ap.add_argument('--rows', required=True)
    ap.add_argument('--limit', type=int, default=200)
    a = ap.parse_args()
    p = Params()
    if a.problem == 3:
        scan = [(p.ring_r * np.cos(k * np.pi / 3), p.ring_r * np.sin(k * np.pi / 3))
        ↪  for k in range(6)]
    else:
        with open(os.path.join(HERE, p.q4_pts), encoding='utf-8') as f:
            scan = [tuple(x) for x in json.load(f)['points']]
    rows = [json.loads(l) for l in open(a.rows, encoding='utf-8')][:a.limit]
    real, orac, orac_src = [], [], []
```

```
    for r in rows:
        ar = Arena(seed=r['seed'], n_dir=0 if a.problem == 3 else -1)
        src = [(s.x, s.y) for s in ar.sources]
        travel_s = r['total_time'] - 6.0 * r['measures'] - R.T_CLEAR * r['cleared']
        ↪  - R.T_LOCATE * r['misses']
        real.append(travel_s * R.SPEED)
        orac.append(tour_length((0.0, 0.0), scan + src))
        orac_src.append(tour_length((0.0, 0.0), src))
    real, orac, orac_src = map(np.array, (real, orac, orac_src))
    print('问题%d  %d 局' % (a.problem, len(rows)))
    print('实际行走 均值 %.0f 米' % real.mean())
    print('全知时走完扫描点与全部源 均值 %.0f 米，实际比它多 %.1f%%' % (orac.mean(),
    ↪  100 * (real.mean() / orac.mean() - 1)))
    print('全知时只走全部源 均值 %.0f 米' % orac_src.mean())
    print('实际平均检测 %.0f 次 约 %.0f 秒，平均总时间 %.0f 秒' %
    ↪  (np.mean([r['measures'] for r in rows]),
          6 * np.mean([r['measures'] for r in rows]), np.mean([r['total_time'] for r
          ↪  in rows])))


if __name__ == '__main__':
    main()
```

q34_belief.py　对比方法：逐步贪心的可行域

```
# -*- coding: utf-8 -*-
""" 问题 3、问题 4 的可行域：每个频道一份，记录该频道干扰源还可能在哪里。

两种表示。

测到过示向度的频道用精确凸多边形。每条示向度是张角 2 度的扇形，再加上有效接收半径上界圆和
目标区域圆，都是凸集，交出来就是问题1 的交会定位区域。误差有 1 度的硬上界，
↪  真实位置必在其中。

从没测到过示向度的频道用网格。无信号能排除的是一个圆盘，不是凸集，网格表示最直接：
全向源在有效接收半径下界 1000 米内必被测到，所以某点无信号就排掉它周围 1000 米的格点。

问题4 里源可能是定向的，无信号不再能直接排掉近处。这时对每个格点再记一段圆弧，
↪  表示该位置上
还说得通的定向方向的反方向 psi。在 P 处无信号且 P 确定在接收范围内，说明 P
↪  不在覆盖范围内，
即 psi 与该位置指向 P 的方位角相差不超过 90 度；在 P 处测到信号则相反。两类观测都是宽
↪  180 度的
圆弧，交出来仍是一段圆弧，所以每个格点只存圆弧中心与半宽。圆弧为空的格点可以排除。
"""
import math
import numpy as np

from kernel import rules as R
from kernel import geometry as G

EPS_ROUND = 0.02          # 示向度保留两位小数，留一点余量


def make_grid(res):
    n = int(math.ceil(R.ARENA_R / res))
    ax = np.arange(-n, n + 1) * res
```

```
    X, Y = np.meshgrid(ax, ax, indexing='ij')
    X, Y = X.ravel(), Y.ravel()
    keep = X * X + Y * Y <= R.ARENA_R ** 2
    return X[keep], Y[keep]


class ChannelBelief:
    def __init__(self, ch, gx, gy, omni=True):
        self.ch = ch
        self.omni = omni
        self.X, self.Y = gx, gy
        self.mask = np.ones(gx.shape, dtype=bool)
        self.cone_c = None             # 定向源的圆弧中心，问题 4 才用，用到时才分配
        self.cone_w = None             # 圆弧半宽，180 表示还没有约束
        self.wedges = []               # 测到示向度的 (x, y, 示向度)
        self.nosig = []                # 无信号的检测点
        self.miss = []                 # 清除落空的位置
        self.near_at = None            # 返回信号过强的检测点
        self.seen = False
        self.cleared = False
        self.poly = None
        self._geo = None

    # ---------- 观测 ----------
    def add_direction(self, x, y, svd):
        self.wedges.append((x, y, svd))
        self.seen = True
        if not self.omni:
            self._cut_cone((x, y), detected=True)
        self._rebuild_poly()

    def add_near(self, x, y):
        self.seen = True
        self.near_at = (x, y)
        self.poly = [(x + R.NEAR_R * math.cos(a), y + R.NEAR_R * math.sin(a))
                     for a in np.linspace(0.0, 2.0 * math.pi, 24, endpoint=False)]
        self._geo = None

    def add_no_signal(self, x, y):
        self.nosig.append((x, y))
        if self.seen:
            return
        d2 = (self.X - x) ** 2 + (self.Y - y) ** 2
        if self.omni:
            self.mask &= d2 > R.R_MIN ** 2
        else:
            self._cut_cone((x, y), detected=False)

    def add_miss(self, x, y):
        self.miss.append((x, y))
        if not self.seen:
            self.mask &= (self.X - x) ** 2 + (self.Y - y) ** 2 > R.CLEAR_R ** 2
        self._geo = None

    # ---------- 定向源的圆弧 ----------
    def _ensure_cone(self):
        if self.cone_c is None:
            self.cone_c = np.zeros(self.X.shape, dtype=np.float32)
```

```
        self.cone_w = np.full(self.X.shape, 180.0, dtype=np.float32)

    def _cut_cone(self, p, detected):
        """ 用一个检测点的观测切每个格点的圆弧。只有确定在接收范围内的格点才受约束。"""
        self._ensure_cone()
        d2 = (self.X - p[0]) ** 2 + (self.Y - p[1]) ** 2
        if detected:
            sel = self.mask & (d2 <= R.R_MAX ** 2)
            # 测到了信号：检测点在覆盖范围内，psi 与指向检测点方向的反方向相差不超过 90
            ↪ 度
            ang = np.degrees(np.arctan2(self.Y - p[1], self.X - p[0]))
        else:
            sel = self.mask & (d2 <= R.R_MIN ** 2)
            ang = np.degrees(np.arctan2(p[1] - self.Y, p[0] - self.X))
        if not sel.any():
            return
        c, w = self.cone_c, self.cone_w
        full = sel & (w >= 180.0)
        part = sel & (w < 180.0)
        c[full] = ang[full]
        w[full] = 90.0
        if part.any():
            d = (ang[part] - c[part] + 180.0) % 360.0 - 180.0
            lo = np.maximum(-w[part], d - 90.0)
            hi = np.minimum(w[part], d + 90.0)
            nw = 0.5 * (hi - lo)
            nw[lo > hi] = -1.0
            c[part] = c[part] + 0.5 * (lo + hi)
            w[part] = nw
        self.mask &= w >= 0.0

    def cone_score(self, px, py, idx):
        """假设在 (px, py) 测到无信号，给这一刀打分：
        ↪ 直接排掉的格点数与圆弧总共收窄的度数。

        一个格点至少要被切两刀才可能空，只数排掉几个的话第一刀永远是零分，
        ↪ 所以要带上收窄量。
        """
        X, Y = self.X[idx], self.Y[idx]
        inr = (X - px) ** 2 + (Y - py) ** 2 <= R.R_MIN ** 2
        if not inr.any():
            return 0, 0.0
        ang = np.degrees(np.arctan2(py - Y, px - X))
        c, w = self.cone_c[idx], self.cone_w[idx]
        full = w >= 180.0
        d = (ang - c + 180.0) % 360.0 - 180.0
        lo = np.maximum(-w, d - 90.0)
        hi = np.minimum(w, d + 90.0)
        new_w = np.where(full, 90.0, 0.5 * (hi - lo))
        dead = inr & ~full & (lo > hi)
        live = inr & ~dead
        return int(dead.sum()), float(np.clip(w[live] - new_w[live], 0.0,
        ↪ None).sum())

    # ---------- 多边形 ----------
    def _rebuild_poly(self):
        hps = list(G.disk_halfplanes((0.0, 0.0), R.ARENA_R))
        for x, y, th in self.wedges:
```

```
            hps.extend(G.wedge_halfplanes((x, y), th, R.ERR_DEG + EPS_ROUND))
            hps.extend(G.disk_halfplanes((x, y), R.R_MAX))
        poly, _ = G.intersect_halfplanes(hps, box=4.0 * R.ARENA_R)
        self.poly = poly
        self._geo = None

    # ---------- 查询 ----------
    def empty(self):
        return (not self.poly) if self.seen else (not self.mask.any())

    def alive_count(self):
        return len(self.poly or []) if self.seen else int(self.mask.sum())

    def centroid(self):
        if self.seen:
            return G.polygon_centroid(self.poly) if self.poly else None
        if not self.mask.any():
            return None
        return float(self.X[self.mask].mean()), float(self.Y[self.mask].mean())

    def enclosing(self):
        """可行域的最小覆盖圆 (圆心, 半径)。半径不超过清除半径时, 在圆心下清除指令必中。
        ↪ """
        if self._geo is None:
            if not self.seen or not self.poly:
                self._geo = (None, float('inf'))
            else:
                self._geo = G.min_enclosing_circle(self.poly)
        return self._geo

    def alive_index(self, cap=3000):
        idx = np.flatnonzero(self.mask)
        if idx.size > cap:
            idx = idx[:: idx.size // cap + 1]
        return idx
```

q34_robot.py　对比方法：逐步贪心的决策

```
# -*- coding: utf-8 -*-
"""问题3、问题4 的机器狗搜索定位清除策略。同一份代码打离线复刻和真模拟器, 区别只在
↪ client。

每一步在四件事里挑一件:
1 有频道的可行域已收到清除半径以内, 就去最小覆盖圆圆心清除;
2 有频道测到过示向度但还不够准, 就在去它那里的路上找一个点再测一次, 前进一段并横移一小段;
↪
3 只剩从没测到过信号的频道, 就挑一个单位时间能排掉最多可能位置的点, 把这些频道扫一遍;
4 可行域全空, 或者已清除加上已确认存在的源到了 16 个, 就退出。

走路是时间的大头, 检测只要 5 秒, 相当于走 25 米, 所以宁可多测几次也不绕远路。
"""
import math
import time
import numpy as np

from kernel import rules as R
from q34_belief import ChannelBelief, make_grid
```

```
class Params:
    res = 10.0            # 网格边长，米
    clear_margin = 0.5    # 覆盖圆半径不超过清除半径减去这个余量才下清除指令
    approach = 0.30       # 精化时朝可行域中心前进的比例
    offset = 0.05         # 精化时横移的比例，撑开交会角
    min_step = 200.0      # 精化步长下限，米
    oppo_k = 0            # 每次停下顺带检测的其他频道数
    survey_grid = 600.0   # 候选扫描点间距，米
    survey_ext = 0.0      # 候选扫描点可伸出目标区域的距离，米，
    ↪  背朝外的定向源只能在外面测到
    dir_side = 0          # 为 1 时，定向源精化只在测到过信号的一侧选点
    use_cap16 = 1         # 为 1 时，已清除加已确认存在的源达到上限 16 就不再扫空频道
    stuck_k = 2           # 定向源精化连续这么多次无信号，就判定站在了它的背面
    stuck_max = 10        # 连续无信号到这个次数就放弃该频道，宁可丢一个源也不拖垮整局
    sweep_clear_r = 60.0  # 定向源可行域覆盖圆半径不超过它时，改用点阵清除，不再依赖测向
    max_actions = 3000
    real_budget = 1000.0
    real_reserve = 25.0

    def __init__(self, **kw):
        for k, v in kw.items():
            setattr(self, k, v)

    def as_dict(self):
        keys = [k for k in dir(Params) if not k.startswith('_') and not
        ↪  callable(getattr(Params, k))]
        return {k: getattr(self, k) for k in keys}


class Robot:
    def __init__(self, client, params=None, omni=True, verbose=False):
        self.cl = client
        self.p = params or Params()
        self.omni = omni
        self.verbose = verbose
        gx, gy = make_grid(self.p.res)
        self.gx, self.gy = gx, gy
        self.bel = {ch: ChannelBelief(ch, gx, gy, omni) for ch in range(R.CH_MIN,
        ↪  R.CH_MAX + 1)}
        self.pos = (0.0, 0.0)
        self.vt = 0.0
        self.n_clear = 0
        self.actions = 0
        self.unresolved = []
        self.swept = set()
        self.stuck = {ch: 0 for ch in self.bel}     # 每个频道精化时连续无信号的次数
        self.stop_reason = ''
        self.t0 = time.time()
        self.cand = self._candidates()

    # ---------- 基础 ----------
    def say(self, *a):
        if self.verbose:
            print('[%8.0f s]' % self.vt, *a, flush=True)

    def _candidates(self):
```

```
        g = self.p.survey_grid
        lim = R.ARENA_R + self.p.survey_ext
        n = int(lim / g)
        pts = [(i * g, j * g) for i in range(-n, n + 1) for j in range(-n, n + 1)
               if (i * g) ** 2 + (j * g) ** 2 <= lim * lim]
        return np.array(pts, dtype=float)

    def dist(self, p):
        return math.hypot(p[0] - self.pos[0], p[1] - self.pos[1])

    def out_of_budget(self):
        if self.actions >= self.p.max_actions:
            self.stop_reason = '动作数到上限'
            return True
        if time.time() - self.t0 > self.p.real_budget:
            self.stop_reason = '现实时间到本地预算'
            return True
        left = self.cl.real_left()
        if left is not None and left < self.p.real_reserve:
            self.stop_reason = '接近程序运行时间上限'
            return True
        return False

    def measure(self, x, y, ch):
        r = self.cl.measure(x, y, ch)
        self.actions += 1
        if not r.get('accepted'):
            return None
        self.pos = (x, y)
        self.vt = r['virtual_time_s']
        b = self.bel[ch]
        res = r['measure_result']
        if res == 'direction':
            self.stuck[ch] = 0
            b.add_direction(x, y, r['svd_deg'])
        elif res == 'near':
            b.add_near(x, y)
            # 距离不超过 5 米，就地清除必中
            self.clear(x, y, ch)
        else:
            if b.seen:
                self.stuck[ch] += 1
            b.add_no_signal(x, y)
        return res

    def clear(self, x, y, ch):
        r = self.cl.clear(x, y, ch)
        self.actions += 1
        if not r.get('accepted'):
            return None
        self.pos = (x, y)
        self.vt = r['virtual_time_s']
        ok = r['clear_result'] == 'success'
        b = self.bel[ch]
        if ok:
            b.cleared = True
            self.n_clear += 1
        else:
```

```
            b.add_miss(x, y)
        return ok

    def live(self):
        return [c for c, b in self.bel.items() if not b.cleared and not b.empty()]

    # ---------- 决策 ----------
    def refine_point(self, ch):
        """ 给测到过信号的频道选下一个检测点：朝可行域中心前进一段，再横移一小段。"""
        b = self.bel[ch]
        c = b.centroid()
        if c is None:
            return None
        dx, dy = c[0] - self.pos[0], c[1] - self.pos[1]
        D = math.hypot(dx, dy)
        if D < 1.0:
            # 已站在中心上，只能横移出去换个角度
            return self.pos[0] + 60.0, self.pos[1]
        ux, uy = dx / D, dy / D
        if not self.omni and self.p.dir_side and b.wedges:
            # 定向源覆盖范围是过源的半平面，测到过信号的点都在这一侧;
            ↪ 当前位置在背面时先绕到这一侧
            mx = sum(w[0] for w in b.wedges) / len(b.wedges) - c[0]
            my = sum(w[1] for w in b.wedges) / len(b.wedges) - c[1]
            mn = math.hypot(mx, my)
            if mn > 1e-6 and (self.pos[0] - c[0]) * mx + (self.pos[1] - c[1]) * my <
            ↪ 0:
                near = max(self.p.min_step, min(0.3 * D, 400.0))
                return c[0] + mx / mn * near, c[1] + my / mn * near
        step = min(max(self.p.min_step, self.p.approach * D), D)
        off = self.p.offset * D
        best = None
        for s in (1.0, -1.0):
            px = self.pos[0] + ux * step - s * uy * off
            py = self.pos[1] + uy * step + s * ux * off
            score = -math.hypot(px, py)          # 两侧对称时偏向区域中心一侧,
            ↪ 后续更好走
            if best is None or score > best[0]:
                best = (score, px, py)
        return best[1], best[2]

    def side_point(self, ch):
        """ 定向源的背面脱困点：锚在离当前位置最近的一个测到过信号的检测点上。

        那个检测点当初收到了信号，说明它既在接收半径内、又在覆盖范围内，
        ↪ 且这两点都不随时间变。
        从它沿连线朝可行域中心走一段，方位与它看源的方位几乎相同、距离更近，
        ↪ 仍落在覆盖范围内。
        连续失败时逐次退回到更靠近检测点的位置，并左右交替错开一点，免得与检测点共线。
        """
        b = self.bel[ch]
        c = b.centroid()
        d0 = min(((w[0], w[1]) for w in b.wedges), key=self.dist)
        k = max(0, self.stuck[ch] - self.p.stuck_k)
        frac = max(0.12, 0.55 * 0.7 ** k)
        dx, dy = c[0] - d0[0], c[1] - d0[1]
        L = math.hypot(dx, dy) or 1.0
        side = 0.08 * L * (1 if k % 2 == 0 else -1)
```

```
        return (d0[0] + frac * dx - dy / L * side, d0[1] + frac * dy + dx / L *
        ↪  side)

    def lattice_clear(self, ch):
        """ 点阵清除：在可行域上铺间距 34 米的六边形点阵，逐点下清除指令直到成功。

        六边形点阵间距 s 时，任一点到最近格点不超过 s 除以根号 3，取 34 米即不超过 19.6
        ↪  米，
        小于清除半径 20 米。真实位置必在可行域内，所以按序扫完一定有一次命中。
        清除与定向源朝向无关，站在背面也能清掉。
        """
        from kernel import geometry as G
        b = self.bel[ch]
        ctr, r = b.enclosing()
        s = 34.0
        rows = int(math.ceil((r + R.CLEAR_R) / (s * math.sqrt(3) / 2.0))) + 1
        cols = int(math.ceil((r + R.CLEAR_R) / s)) + 1
        pts = []
        for j in range(-rows, rows + 1):
            for i in range(-cols, cols + 1):
                p = (ctr[0] + s * (i + 0.5 * (j % 2)), ctr[1] + s * math.sqrt(3) /
                ↪  2.0 * j)
                if self._dist_to_poly(p, b.poly) > R.CLEAR_R:
                    continue
                if any(math.dist(p, m) < 1.0 for m in b.miss):
                    continue
                pts.append(p)
        self.say('点阵清除 频道%2d 共 %d 点 覆盖圆半径 %.1f 米' % (ch, len(pts), r))
        while pts and not self.out_of_budget():
            p = min(pts, key=self.dist)
            pts.remove(p)
            if self.clear(p[0], p[1], ch):
                return True
        return False

    @staticmethod
    def _dist_to_poly(p, poly):
        from kernel import geometry as G
        if G.point_in_convex(p, poly, tol=1e-6):
            return 0.0
        best = float('inf')
        for i in range(len(poly)):
            a, q = poly[i], poly[(i + 1) % len(poly)]
            ax, ay = q[0] - a[0], q[1] - a[1]
            L2 = ax * ax + ay * ay
            t = 0.0 if L2 < 1e-12 else max(0.0, min(1.0, ((p[0] - a[0]) * ax + (p[1]
            ↪  - a[1]) * ay) / L2))
            best = min(best, math.dist(p, (a[0] + t * ax, a[1] + t * ay)))
        return best

    def survey_point(self, chans):
        if not self.omni:
            return self._survey_dir(chans)
        acc = np.zeros(self.gx.shape, dtype=bool)
        for c in chans:
            acc |= self.bel[c].mask
        X, Y = self.gx[acc], self.gy[acc]
        if X.size == 0:
```

```
            return None
        if X.size > 20000:
            X, Y = X[:: X.size // 20000 + 1], Y[:: X.size // 20000 + 1]
        gain = np.zeros(len(self.cand))
        blk = max(1, int(3_000_000 / X.size))
        for i in range(0, len(self.cand), blk):
            cx = self.cand[i:i + blk, 0][:, None]
            cy = self.cand[i:i + blk, 1][:, None]
            gain[i:i + blk] = (((X[None, :] - cx) ** 2 + (Y[None, :] - cy) ** 2) <=
            ↪  R.R_MIN ** 2).sum(1)
        cost = np.hypot(self.cand[:, 0] - self.pos[0], self.cand[:, 1] -
        ↪  self.pos[1]) / R.SPEED \
            + len(chans) * (R.T_MEASURE + R.T_SWITCH)
        k = int(np.argmax(gain / cost))
        return None if gain[k] <= 0 else (float(self.cand[k, 0]), float(self.cand[k,
        ↪  1]))

    def _survey_dir(self, chans):
        idx = {c: self.bel[c].alive_index() for c in chans}
        for c in chans:
            self.bel[c]._ensure_cone()
        best, bs = None, 0.0
        for px, py in self.cand:
            if (round(px), round(py)) in self.swept:
                continue
            dead, shrink = 0, 0.0
            for c in chans:
                d0, s0 = self.bel[c].cone_score(px, py, idx[c])
                dead += d0
                shrink += s0
            if dead == 0 and shrink <= 0.0:
                continue
            cost = math.hypot(px - self.pos[0], py - self.pos[1]) / R.SPEED \
                + len(chans) * (R.T_MEASURE + R.T_SWITCH)
            s = (dead + 0.01 * shrink) / cost
            if s > bs:
                best, bs = (float(px), float(py)), s
        return best

    def opportunistic(self, x, y, skip):
        cands = [c for c in self.live() if c != skip]
        cands.sort(key=lambda c: (self.bel[c].seen, -self.bel[c].alive_count()))
        for c in cands[: self.p.oppo_k]:
            if self.out_of_budget():
                return
            self.measure(x, y, c)

    def step(self):
        live = self.live()
        if not live:
            self.stop_reason = '所有频道已清除或确认为空'
            return False
        seen_live = [c for c in live if self.bel[c].seen]
        unseen = [c for c in live if not self.bel[c].seen]
        if self.p.use_cap16 and unseen and self.n_clear + len(seen_live) >= R.N_MAX:
            # 测到过信号的频道一定有源，已清除加上这些已经到上限，没测到过的频道必然为空
            for c in unseen:
                self.bel[c].mask[:] = False
```

```
        self.say('已确认存在的源达到 16 个，其余 %d 个频道判空' % len(unseen))
        return True

    ready, pending = [], []
    for c in seen_live:
        b = self.bel[c]
        ctr, r = b.enclosing()
        last_miss = b.miss and ctr is not None and min(math.dist(ctr, m) for m
        ↪  in b.miss) <= R.CLEAR_R
        if ctr is not None and r <= R.CLEAR_R - self.p.clear_margin and not
        ↪  last_miss:
            ready.append((c, ctr, r))
        else:
            pending.append(c)
    if ready:
        c, ctr, r = min(ready, key=lambda t: self.dist(t[1]))
        self.say('清除 频道%2d 于 (%.0f, %.0f) 覆盖圆半径 %.1f 米' % (c, ctr[0],
        ↪  ctr[1], r))
        if self.clear(ctr[0], ctr[1], c):
            self.opportunistic(ctr[0], ctr[1], c)
        return True
    if pending:
        c = min(pending, key=lambda c: self.dist(self.bel[c].centroid() or
        ↪  self.pos))
        if not self.omni and self.bel[c].wedges:
            if self.stuck[c] >= self.p.stuck_max:
                self.unresolved.append(c)
                self.bel[c].poly = []
                self.say('放弃 频道%2d 连续 %d 次测不到' % (c, self.stuck[c]))
                return True
            _, r = self.bel[c].enclosing()
            if r <= self.p.sweep_clear_r and (self.stuck[c] >= 1 or
            ↪  self.bel[c].miss):
                self.lattice_clear(c)
                return True
            if self.stuck[c] >= self.p.stuck_k:
                mp = self.side_point(c)
                res = self.measure(mp[0], mp[1], c)
                self.say('脱困 频道%2d 测于 (%.0f, %.0f) 得 %s' % (c, mp[0],
                ↪  mp[1], res))
                return True
        mp = self.refine_point(c)
        res = self.measure(mp[0], mp[1], c)
        self.say('精化 频道%2d 测于 (%.0f, %.0f) 得 %s' % (c, mp[0], mp[1], res))
        self.opportunistic(mp[0], mp[1], c)
        return True
    sp = self.survey_point(unseen)
    if sp is None:
        for c in unseen:
            if not self.omni:
                self.unresolved.append(c)
            self.bel[c].mask[:] = False
        return True
    self.swept.add((round(sp[0]), round(sp[1])))
    self.say('扫描 于 (%.0f, %.0f) 查 %d 个频道' % (sp[0], sp[1], len(unseen)))
    for c in unseen:
        if self.out_of_budget():
            break
```

```
            self.measure(sp[0], sp[1], c)
        return True

    def play(self):
        for ch in range(R.CH_MIN, R.CH_MAX + 1):
            if self.out_of_budget():
                return
            self.measure(0.0, 0.0, ch)
        while not self.out_of_budget():
            if not self.step():
                break

    def run(self):
        r = self.cl.enter()
        if not r.get('accepted'):
            return None
        self.play()
        self.cl.exit()
        return {'cleared': self.n_clear, 'virtual_time': self.vt, 'actions':
        ↪ self.actions,
                'real_time': time.time() - self.t0, 'stop_reason': self.stop_reason,
                'unresolved': list(self.unresolved)}
```

q34_batch.py　对比方法：逐步贪心的批量演练

```
# -*- coding: utf-8 -*-
""" 问题 3、问题 4 的离线批量演练，只在服务器上跑。

  python q34_batch.py --problem 3 --cases 400 --tag base
  python q34_batch.py --problem 4 --cases 200 --set survey_ext=600 --set dir_side=1
  ↪ --tag ext600
  python q34_batch.py --problem 3 --cases 400 --ab use_cap16=0,1
  ↪ 对一个开关做配对对比

同一组案例种子、同样的参数跑完，输出清除比例、平均定位清除时间、收尾时间占比等统计，
逐局明细写进 out/q3 或 out/q4 下的 jsonl，供论文作表作图。
"""
import os
import sys
import json
import time
import argparse
import statistics as st
from multiprocessing import Pool

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from kernel.sim import Arena
from kernel.api import LocalClient
from q34_robot import Robot, Params

OUT_ROOT = os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'out')


def play_one(job):
    seed, problem, kw = job
    arena = Arena(seed=seed, n_dir=0 if problem == 3 else -1)
    robot = Robot(LocalClient(arena), Params(**kw), omni=(problem == 3))
```

```
    t0 = time.time()
    err = ''
    try:
        robot.run()
    except Exception as e:           # 单局出错不拖垮整批，记下来
        err = repr(e)
    s = arena.stats()
    missed = [{'ch': x.ch, 'dir': x.face is not None, 'r': (x.x ** 2 + x.y ** 2) **
    ↪  0.5}
              for x in arena.sources if not x.cleared]
    return {'seed': seed, 'n': s['n'], 'n_dir': s['n_dir'], 'cleared': s['cleared'],
            'ratio': s['ratio'], 'total_time': s['total_time'], 'avg_time':
            ↪  s['avg_time'],
            'tail_time': s['tail_time'], 'actions': robot.actions, 'real_s':
            ↪  time.time() - t0,
            'stop_reason': robot.stop_reason, 'unresolved': robot.unresolved,
            'missed': missed, 'err': err}


def summarize(rows):
    ok = [r for r in rows if r['cleared']]
    avg = sorted(r['avg_time'] for r in ok)
    return {
        'cases': len(rows),
        'mean_ratio': st.mean(r['ratio'] for r in rows),
        'full_clear_pct': 100.0 * sum(r['ratio'] >= 1.0 for r in rows) / len(rows),
        'mean_avg_time': st.mean(avg),
        'median_avg_time': avg[len(avg) // 2],
        'p90_avg_time': avg[int(0.9 * (len(avg) - 1))],
        'mean_total_time': st.mean(r['total_time'] for r in ok),
        'mean_tail_share_pct': 100.0 * st.mean(r['tail_time'] / r['total_time'] for
        ↪  r in ok),
        'max_real_s': max(r['real_s'] for r in rows),
        'max_actions': max(r['actions'] for r in rows),
        'errors': sum(1 for r in rows if r['err']),
        'missed_dir': sum(sum(m['dir'] for m in r['missed']) for r in rows),
        'missed_omni': sum(sum(not m['dir'] for m in r['missed']) for r in rows),
    }


def run_batch(problem, seeds, kw, procs):
    with Pool(procs) as pool:
        return pool.map(play_one, [(s, problem, kw) for s in seeds], chunksize=1)


def parse_sets(items):
    kw = {}
    for it in items or []:
        k, v = it.split('=', 1)
        kw[k] = type(getattr(Params, k))(float(v)) if isinstance(getattr(Params, k),
        ↪  (int, float)) else v
    return kw


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--problem', type=int, choices=(3, 4), required=True)
    ap.add_argument('--cases', type=int, default=200)
```

```
    ap.add_argument('--start', type=int, default=9000)
    ap.add_argument('--procs', type=int, default=19)
    ap.add_argument('--set', action='append', help='覆盖参数，如 survey_ext=600')
    ap.add_argument('--params', default='', help='参数 json 文件')
    ap.add_argument('--ab', default='', help='配对对比一个参数的几个取值，如
    ↪  use_cap16=0,1')
    ap.add_argument('--tag', default='run')
    a = ap.parse_args()

    out_dir = os.path.join(OUT_ROOT, 'q%d' % a.problem)
    os.makedirs(out_dir, exist_ok=True)
    kw = {}
    if a.params:
        with open(a.params, encoding='utf-8') as f:
            kw.update(json.load(f).get('params', {}))
    kw.update(parse_sets(a.set))
    seeds = list(range(a.start, a.start + a.cases))

    variants = [(a.tag, kw)]
    if a.ab:
        name, vals = a.ab.split('=', 1)
        variants = []
        for v in vals.split(','):
            k2 = dict(kw)
            k2.update(parse_sets(['%s=%s' % (name, v)]))
            variants.append(('%s_%s%s' % (a.tag, name, v), k2))

    report = {}
    for tag, k in variants:
        t0 = time.time()
        rows = run_batch(a.problem, seeds, k, a.procs)
        s = summarize(rows)
        s['wall_s'] = time.time() - t0
        s['params'] = Params(**k).as_dict()
        report[tag] = s
        with open(os.path.join(out_dir, 'batch_%s.jsonl' % tag), 'w',
        ↪  encoding='utf-8') as f:
            for r in rows:
                f.write(json.dumps(r, ensure_ascii=False) + '\n')
        print('[%s] %d 局  清除比例 %.4f  全清 %.1f%%  平均定位清除时间 均值 %.1f 中位
        ↪  %.1f 九成 %.1f 秒  '
              '收尾占比 %.1f%%  单局程序最长 %.1f 秒  动作最多 %d  漏清 定向 %d 全向
              ↪  %d  报错 %d  用时 %.0f 秒'
             % (tag, s['cases'], s['mean_ratio'], s['full_clear_pct'],
              ↪  s['mean_avg_time'],
                s['median_avg_time'], s['p90_avg_time'], s['mean_tail_share_pct'],
                ↪  s['max_real_s'],
                s['max_actions'], s['missed_dir'], s['missed_omni'], s['errors'],
                ↪  s['wall_s']), flush=True)
    with open(os.path.join(out_dir, 'report_%s.json' % a.tag), 'w',
    ↪  encoding='utf-8') as f:
        json.dump(report, f, ensure_ascii=False, indent=1)


if __name__ == '__main__':
    main()
```

live2.py　问题三、四：连模拟器运行一局

```
# -*- coding: utf-8 -*-
"""用新策略 q34_plan 连真模拟器跑一局，问题3 与问题4 共用。只能在装有模拟器的本机上运行。
↪

  python live2.py 3                 问题3，演练与正式都用它，由模拟器界面上点的入口决定
  python live2.py 4
  python live2.py 3 --wait 1800     最多等接口开放 1800 秒

程序可以先启动，它用同一个请求每半秒重试进入，接口一开放就进入。
每局写两份记录到 live_logs：逐条请求与响应的 jsonl，以及一局汇总的 json，表1
↪  从汇总里取数。
测试案例编码在模拟器界面上，正式测试结束后从模拟器日志列表导出加密日志，不要改文件名。
"""
import os
import sys
import json
import time
import argparse

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
from kernel.api import HttpClient, TestOver
from q34_plan import PlanRobot, Params


def team_id(arg):
    if arg:
        return arg.strip()
    for p in (os.path.join(HERE, 'team.txt'), os.path.join(HERE, '..',
    ↪  '正式测试原始记录', 'team.txt')):
        if os.path.exists(p):
            with open(p, encoding='utf-8') as f:
                t = f.read().strip()
            if t:
                return t
    return ''


def main():
    ap = argparse.ArgumentParser(description='用新策略连真模拟器跑问题3 或问题4
    ↪  的一局')
    ap.add_argument('problem', type=int, choices=(3, 4))
    ap.add_argument('--team', default='')
    ap.add_argument('--wait', type=int, default=1800)
    ap.add_argument('--base', default='http://127.0.0.1:2026')
    ap.add_argument('--seed', type=int, default=0)
    ap.add_argument('--tag', default='')
    a = ap.parse_args()

    team = team_id(a.team)
    if not team:
        print('没有参赛队号，用 --team 传入或写进 team.txt')
        return 1
    log_dir = os.path.join(HERE, 'live_logs')
    os.makedirs(log_dir, exist_ok=True)
    stamp = time.strftime('%m%d_%H%M%S')
```

```
    log = os.path.join(log_dir, 'new_p%d_%s%s.jsonl' % (a.problem, stamp, ('_' +
    ↪ a.tag) if a.tag else ''))
    kw = dict(real_budget=1e9, max_actions=5000, seed=a.seed)   #
    ↪ 真机以接口给出的剩余现实时间为准
    cl = HttpClient(team, base=a.base, log_path=log)
    robot = PlanRobot(cl, Params(**kw), omni=(a.problem == 3), verbose=True)
    print('问题%d  队号 %s  接口 %s' % (a.problem, team, a.base), flush=True)

    try:
        r = cl.enter(wait_s=a.wait)
    except TestOver as e:
        print('等了 %d 秒接口一直没开放: %s' % (a.wait, e))
        cl.close()
        return 1
    if not r.get('accepted'):
        print('进入被拒绝', r)
        cl.close()
        return 1
    t0 = time.time()
    print('已进入, 本局可用现实时间 %s 秒' % r.get('remaining_real_duration_s'),
    ↪ flush=True)
    end = '主动退出'
    try:
        robot.play()
    except TestOver as e:
        end = '提前结束 %s' % e
    except KeyboardInterrupt:
        end = '手工中断'
    except Exception as e:
        end = '程序异常 %r' % e
    ex = cl.exit()
    run_s = time.time() - t0
    cl.close()

    avg = robot.vt / robot.n_clear if robot.n_clear else None
    summary = {'题号': a.problem, '队号': team, '结束方式': end, '清除干扰源个数':
    ↪ robot.n_clear,
               '虚拟世界总时间_秒': robot.vt, '平均定位清除时间_秒': avg,
               ↪ '程序运行时间_秒': run_s,
               '动作数': robot.actions, '检测次数': robot.n_measure, '清除落空次数':
               ↪ robot.n_miss,
               '停止原因': robot.stop_reason, '放弃的频道': robot.unresolved,
               '退出响应': ex, '参数': Params(**kw).as_dict(), '指令日志':
               ↪ os.path.basename(log)}
    with open(log.replace('.jsonl', '.summary.json'), 'w', encoding='utf-8') as f:
        json.dump(summary, f, ensure_ascii=False, indent=1, default=str)
    print()
    print('结束方式        %s' % end)
    print('清除干扰源个数  %d' % robot.n_clear)
    print('虚拟世界总时间  %.1f 秒' % robot.vt)
    if avg is not None:
        print('平均定位清除时间 %.1f 秒' % avg)
    print('程序运行时间    %.1f 秒' % run_s)
    print('记录 %s' % log)
    return 0


if __name__ == '__main__':
```

```
    sys.exit(main())
```

live_loop.py　问题三、四：连模拟器连续演练

```
# -*- coding: utf-8 -*-
""" 连真模拟器连续打多局演练，每局都用新策略 q34_plan。只能在装有模拟器的本机上运行。

  python live_loop.py 3 --rounds 20      问题 3，打 20 局
  python live_loop.py 4 --rounds 20

程序启动后一直等接口开放。每在模拟器界面上点一次开始演练测试，它就进入并打完这一局，
然后回到等待状态，等下一局开放。只用于演练，正式测试请用 live2.py 一局一局打。
每局的指令日志与汇总写到 live_logs，文件名带 practice 与局号。
"""
import os
import sys
import json
import time
import argparse

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
from kernel.api import HttpClient, TestOver
from q34_plan import PlanRobot, Params
from live2 import team_id


def one_round(problem, team, base, wait, k, seed):
    log_dir = os.path.join(HERE, 'live_logs')
    os.makedirs(log_dir, exist_ok=True)
    stamp = time.strftime('%m%d_%H%M%S')
    log = os.path.join(log_dir, 'practice_p%d_%s_r%02d.jsonl' % (problem, stamp, k))
    kw = dict(real_budget=1e9, max_actions=5000, seed=seed)
    cl = HttpClient(team, base=base, log_path=log)
    robot = PlanRobot(cl, Params(**kw), omni=(problem == 3), verbose=False)
    try:
        r = cl.enter(wait_s=wait)
    except TestOver:
        cl.close()
        os.remove(log) if os.path.exists(log) and os.path.getsize(log) == 0 else
        ↪  None
        return None
    if not r.get('accepted'):
        print('第 %d 局进入被拒绝 %r' % (k, r), flush=True)
        cl.close()
        time.sleep(2.0)
        return None
    t0 = time.time()
    end = '主动退出'
    try:
        robot.play()
    except TestOver as e:
        end = '提前结束 %s' % e
    except Exception as e:
        end = '程序异常 %r' % e
    ex = cl.exit()
    run_s = time.time() - t0
```

```
    cl.close()
    avg = robot.vt / robot.n_clear if robot.n_clear else None
    summary = {'题号': problem, '局号': k, '结束方式': end, '清除干扰源个数':
    ↪  robot.n_clear,
               '虚拟世界总时间_秒': robot.vt, '平均定位清除时间_秒': avg,
               ↪  '程序运行时间_秒': run_s,
               '动作数': robot.actions, '检测次数': robot.n_measure, '清除落空次数':
               ↪  robot.n_miss,
               '停止原因': robot.stop_reason, '放弃的频道': robot.unresolved,
               ↪  '退出响应': ex,
               '指令日志': os.path.basename(log)}
    with open(log.replace('.jsonl', '.summary.json'), 'w', encoding='utf-8') as f:
        json.dump(summary, f, ensure_ascii=False, indent=1, default=str)
    return summary


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('problem', type=int, choices=(3, 4))
    ap.add_argument('--rounds', type=int, default=20)
    ap.add_argument('--team', default='')
    ap.add_argument('--base', default='http://127.0.0.1:2026')
    ap.add_argument('--wait', type=int, default=3600)
    a = ap.parse_args()
    team = team_id(a.team)
    if not team:
        print('没有参赛队号')
        return 1
    print('问题%d 连续演练 %d 局，队号 %s。在模拟器上点开始演练测试即可，
    ↪  每局打完自动等下一局。'
          % (a.problem, a.rounds, team), flush=True)
    done = 0
    while done < a.rounds:
        s = one_round(a.problem, team, a.base, a.wait, done + 1, seed=done + 1)
        if s is None:
            continue
        done += 1
        print('第 %2d 局  清除 %2d 个  平均 %6.1f 秒  程序 %.1f 秒  %s'
              % (done, s['清除干扰源个数'], s['平均定位清除时间_秒'] or float('nan'),
              ↪  s['程序运行时间_秒'],
                 s['结束方式']), flush=True)
        time.sleep(1.0)
    return 0


if __name__ == '__main__':
    sys.exit(main())
```

practice_stats.py　问题三、四：从模拟器本地库统计演练结果

```
# -*- coding: utf-8 -*-
""" 从模拟器本地统计库只读汇总演练测试，算题目要求的两个统计值。

  python practice_stats.py --problem 3 --since "2026-09-12 02:00"

演练记录里有每局真实干扰源个数、清除个数与虚拟世界总时间，据此
  被清除干扰源个数的比例 = 各局清除个数之和 / 各局干扰源总数之和
```

```
  平均定位清除时间 = 各局虚拟世界总时间之和 / 各局清除个数之和
同时列出逐局明细，按局也算一遍比例与平均时间的均值。
"""
import os
import sqlite3
import argparse
import datetime as dt

DB = r'D:\BaiduNetdiskDownload\CUMCM2026B\Jammers-simulator\JammersSimulatorData\pr
↪  actice-statistics-queue.sqlite3'


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--problem', type=int, choices=(3, 4), required=True)
    ap.add_argument('--since', default='2026-09-12 00:00',
    ↪  help='只统计这个时刻之后创建的演练，北京时间')
    ap.add_argument('--db', default=DB)
    a = ap.parse_args()
    t0 = dt.datetime.strptime(a.since, '%Y-%m-%d
    ↪  %H:%M').replace(tzinfo=dt.timezone(dt.timedelta(hours=8)))
    ms0 = int(t0.timestamp() * 1000)
    con = sqlite3.connect('file:%s?mode=ro' % os.path.abspath(a.db), uri=True)
    rows = con.execute(
        'select case_code, created_at_ms, end_reason, jammer_count,
        ↪  cleared_jammer_count, virtual_time_us, '
        'program_run_duration_ms, measure_accepted_count, clear_failure_count from
        ↪  practice_statistics_tasks '
        'where problem_no=? and created_at_ms>=? and entered=1 order by
        ↪  created_at_ms', (a.problem, ms0)).fetchall()
    if not rows:
        print('没有符合条件的演练记录')
        return
    print('%-22s %-17s %-10s %4s %4s %9s %8s %6s' % ('案例编码', '时刻', '结束',
    ↪  '源数', '清除', '平均秒', '程序秒', '检测'))
    tot_n = tot_c = 0
    tot_t = 0.0
    per_ratio, per_avg = [], []
    for code, cms, end, n, c, vus, pms, nm, nf in rows:
        t = vus / 1e6
        ts = dt.datetime.fromtimestamp(cms / 1000,
        ↪  dt.timezone(dt.timedelta(hours=8))).strftime('%m-%d %H:%M:%S')
        avg = t / c if c else float('nan')
        print('%-22s %-17s %-10s %4d %4d %9.1f %8.1f %6d' % (code, ts, end, n, c,
        ↪  avg, (pms or 0) / 1000, nm))
        tot_n += n
        tot_c += c
        tot_t += t
        per_ratio.append(c / n if n else 0.0)
        if c:
            per_avg.append(avg)
    print()
    print('局数 %d  干扰源总数 %d  清除 %d' % (len(rows), tot_n, tot_c))
    print('被清除干扰源个数的比例 %.4f' % (tot_c / tot_n))
    print('平均定位清除时间 %.1f 秒' % (tot_t / tot_c))
    print('按局平均: 比例 %.4f  平均定位清除时间 %.1f 秒' % (sum(per_ratio) /
    ↪  len(per_ratio), sum(per_avg) / len(per_avg)))
```

```
if __name__ == '__main__':
    main()
```

reproduce.py　一键复现：按顺序运行四问的求解程序

```
# -*- coding: utf-8 -*-
"""一键复现：按论文的顺序运行四问的求解程序，结果写到上一级的 out/ 目录与 kernel/ 目录。

  python reproduce.py            四问全跑，局数取论文里的值，批量演练以小时计
  python reproduce.py --quick    每问只跑少量局数，用来核对程序能否运行

各程序也可以单独运行，用法见各文件开头的说明。多进程数由各程序的 --procs 控制，默认按 19
↪  核。
"""
import os
import subprocess
import sys

HERE = os.path.dirname(os.path.abspath(__file__))
QUICK = "--quick" in sys.argv
PROCS = "4" if QUICK else "19"


def run(*args):
    cmd = [sys.executable] + list(args)
    print(">>", " ".join(cmd))
    subprocess.run(cmd, cwd=HERE, check=True)


def main():
    # 问题一：随机构型统计，两点构型核对
    run("q1_diameter.py", "--trials", "2000" if QUICK else "60000", "--procs",
    ↪  PROCS)
    run("q1_twopoint.py", "--trials", "5000" if QUICK else "200000")
    # 问题二：闭式直径与只做垂直平移的数值验证，按期望耗时在网格上选第二检测点
    run("q2_second_point.py")
    run("q2_redo.py", "--samples", "100" if QUICK else "800", "--step", "50" if
    ↪  QUICK else "25",
        "--tmin", "-100", "--tmax", "1100", "--hmax", "600", "--procs", PROCS,
        ↪  "--tag", "new")
    # 问题四扫描点设计：三角点阵、贪心挪点补漏、模拟退火、独立核验
    run("q4_cover.py", "--lattice", "950", "--out", "kernel/q4_lattice_950.json")
    run("q4_design3.py", "--r", "970", "--ang", "89", "--out",
    ↪  "kernel/q4_design_r970.json",
        "--procs", PROCS, "--variants", "4" if QUICK else "48")
    run("q4_design_sa.py", "--init", "kernel/q4_design_r970.json", "--out",
    ↪  "kernel/q4_design_sa_c8.json",
        "--iters", "500" if QUICK else "120000", "--chains", "2" if QUICK else "16",
        ↪  "--procs", PROCS)
    run("q4_verify.py", "--pts", "kernel/q4_design_sa_c8.json", "--n", "100000" if
    ↪  QUICK else "2000000")
    # 问题三与问题四：最终策略的离线批量演练，与逐步贪心对照，全知路线对照
    run("q34_batch2.py", "--problem", "3", "--cases", "40" if QUICK else "2000",
    ↪  "--start", "50000",
        "--tag", "final3", "--procs", PROCS)
```

```
    run("q34_batch2.py", "--problem", "4", "--cases", "20" if QUICK else "1000",
    ↪  "--start", "50000",
        "--tag", "final4new", "--procs", PROCS)
    run("q34_batch.py", "--problem", "3", "--cases", "40" if QUICK else "800",
    ↪  "--start", "70000",
        "--tag", "c3old", "--procs", PROCS)
    run("q34_oracle.py", "--problem", "3", "--rows", os.path.join("..", "out", "q3",
    ↪  "plan_final3.jsonl"),
        "--limit", "20" if QUICK else "400")


if __name__ == "__main__":
    main()
```